TEORIA

전체 문서 통합본 · 인쇄용

AI-Native Metadata & Ontology Platform

문서 기준일 2026-10-16 · 수집 상태 관찰 15:46 KST

Markdown 53 개 + YAML 참조·템플릿 3 개 · 원문 56 개 전체 수록

A4 세로 · 파일명/경로 표시 · 목차/페이지 번호 · 양면 인쇄 가능

이 문서 읽기

각 장은 원문 파일명과 전체 경로를 표시한다. 목차의 장 번호는 본문과 같다. 문서 내부 링크의 § 번호는 이 통합본의 해당 장이며, 저장소 경로는 원본 코드·계약의 위치다.

날짜별 검증 기록은 당시 결과를 보존한다. 최신 수집의 성공과 과거 데이터의 완전성은 구분한다. EC2 K3s 구성은 로컬 Compose 실배포 검증과 다르다.

수정은 원문 Markdown/YAML 에서 수행한다. 이 Word/PDF 는 2026-10-16 시점의 인쇄 사본이다. 인쇄는 A4·실제 크기·양면 긴 쪽 넘김을 기준으로 한다.

목차

수록 파일 목록 8

01. index.md — Teoria 문서 13

02. platform-guide.md — Teoria 시작과 역할별 작업 17

03. roadmap.md — Teoria product roadmap 19

04. overview.md — Teoria architecture 22

05. repository-structure.md — 저장소 구조와 소유권 25

06. admin_ui.md — Admin UI 사용과 개발 29

07. context.md — Context 의 의미 조회·계획과 실행 32

08. development.md — Platform 개발 안내 35

09. activity_aggregation.md — 공공조달 활동·참여 집계 39

10. contract_aggregation.md — 공공조달 계약 집계 기준 43

11. market_analysis.md — 공공조달 시장 분석 구현 48

12. notice_status.md — 입찰공고 현재 상태 기준 51

13. procurement_classification.md — 공공조달 분야 분류 기준과 확장 계획 52

14. asset_assessment.md — 자산 진단과 제안 적용 55

15. asset_context.md — 자산 맥락과 샘플·프로파일 조회 58

16. law_evidence.md — 법령 근거 검색 60

17. suggestions.md — 변경 제안의 검토·적용과 복구 61

18. ontology.md — Ontology 와 Semantic Binding 64

19. capabilities.md — Capability 실행 계약 66

20. guide.md — Registry guide 67

21. mappings.md — Mapping 값 변환과 객체 생성 69

22. source_authoring.md — Source Registry 작성과 검증 70

23. runtime.md — Runtime 실행과 artifact 배포 72

24. database_source.yaml — 계약 참조·템플릿 74

25. source.yaml — 계약 참조·템플릿 75

26. connectors.md — Connector 와 수집 계약 작성 77

27. database.md — Data DB 와 migration 79

28. deployments.md — Pipeline Deployment 목록 81

29. development.md — Pipelines 개발 안내....91
30. legal_information.md — 법령 수집 및 버전 관리....94
31. public_procurement.md — 공공조달 데이터 수집....97
32. openmetadata.md — OpenMetadata 메타데이터 수집....100
33. prefect.md — Prefect 실행과 장애 대응....102
34. writing_pipelines.md — PostgreSQL 변경분을 DW 에 반영하는 ETL 만들기....106
35. compose.md — Compose 배포와 운영....112
36. configuration.md — 환경 설정과 인증정보....116
37. experimental_k3s.md — K3s 배포 안내 이전....129
38. k3s.md — EC2 단일 서버 K3s 배포....130
39. openmetadata.md — OpenMetadata Server 배포와 업그레이드....136
40. gateway.md — Teoria MCP Gateway....138
41. provider_api.md — Teoria Provider API....140
42. bid-check-service.md — 입찰체크 서비스 연동....141
43. procurement_exploration_contract.md — 입찰체크 조달 탐색 연동 계약....143
44. bid_check_capabilities.yaml — 계약 참조·템플릿....147
45. analyze_metadata_asset.md — 자산 진단 Skills....149
46. source-registry-author.md — Source Registry 작성 Skill....152
47. asset_assessment_2026_10_15.md — 자산 진단 실행 검증: 2026-10-15....154
48. async_deployment_collection_2026_10_16.md — 비동기 배포와 수집 상태 검증: 2026-10-16....155
49. bid_document_archive_2026_10_16.md — 첨부문서·입찰요건 기능 보류 검증....160
50. bid_notice_vertical_verification.md — Bid notice Context vertical 검증 보고서....161
51. compose_pipeline_resources_2026_10_16.md — Compose 수집 Worker 재배포 검증: 2026-10-16....163
52. index.md — 검증 기록....165
53. openmetadata_upgrade_2026_10_13.md — OpenMetadata 업그레이드 검증....167
54. procurement_collection_2026_10_12.md — 공공조달 수집 점검: 2026-10-12....170
55. procurement_exploration_2026_10_13.md — 조달 탐색 연동의 단회 관측: 2026-10-13....171
56. vertical_slice_verification.md — Context vertical 검증 보고서....172

		수록 파일 목록

장

원문 파일 경로

구분

01

docs/index.md

문서

02

docs/platform-guide.md

문서

03

docs/roadmap.md

문서

04

docs/architecture/overview.md

문서

05

docs/architecture/repository-structure.md

문서

06

docs/platform/admin_ui.md

문서

07

docs/platform/context.md

문서

08

docs/platform/development.md

문서

09

docs/platform/domains/public_procurement/activity_aggregation.md

문서

10

docs/platform/domains/public_procurement/contract_aggregation.md

문서

11

docs/platform/domains/public_procurement/market_analysis.md

문서

12

docs/platform/domains/public_procurement/notice_status.md

문서

13

docs/platform/domains/public_procurement/procurement_classification.md

문서

14

docs/platform/metadata/asset_assessment.md

문서

15

docs/platform/metadata/asset_context.md

문서

16

docs/platform/metadata/law_evidence.md

문서

17

docs/platform/metadata/suggestions.md

문서

18

docs/platform/ontology.md

문서

19

docs/platform/registry/capabilities.md

문서

20

docs/platform/registry/guide.md

문서

21

docs/platform/registry/mappings.md

문서

22

docs/platform/registry/source_authoring.md

문서

23

docs/platform/runtime.md

문서

24

docs/platform/registry/templates/database_source.yaml

참조

25

docs/platform/registry/templates/source.yaml

참조

26

docs/pipelines/connectors.md

문서

27

docs/pipelines/database.md

문서

28

docs/pipelines/deployments.md

문서

29

docs/pipelines/development.md

문서

30

docs/pipelines/domains/legal_information.md

문서

31

docs/pipelines/domains/public_procurement.md

문서

32

docs/pipelines/openmetadata.md

문서

33

docs/pipelines/prefect.md

문서

34

docs/pipelines/writing_pipelines.md

문서

35

docs/deployment/compose.md

문서

36

docs/deployment/configuration.md

문서

37

docs/deployment/experimental_k3s.md

문서

38

docs/deployment/k3s.md

문서

39

docs/deployment/openmetadata.md

문서

40

docs/mcp/gateway.md

문서

41

docs/packages/provider_api.md

문서

42

docs/integration/bid-check-service.md

문서

43

docs/integration/procurement_exploration_contract.md

문서

44

docs/integration/bid_check_capabilities.yaml

참조

45

docs/skills/analyze_metadata_asset.md

문서

46

docs/skills/source-registry-author.md

문서

47

docs/verification/asset_assessment_2026_10_15.md

기록

48

docs/verification/async_deployment_collection_2026_10_16.md

기록

49

docs/verification/bid_document_archive_2026_10_16.md

기록

50

docs/verification/bid_notice_vertical_verification.md

기록

51

docs/verification/compose_pipeline_resources_2026_10_16.md

기록

52

docs/verification/index.md

기록

53

docs/verification/openmetadata_upgrade_2026_10_13.md

기록

54

docs/verification/procurement_collection_2026_10_12.md

기록

55

docs/verification/procurement_exploration_2026_10_13.md

기록

56

docs/verification/vertical_slice_verification.md

기록

01. index.md — Teoria 문서

원문 파일: docs/index.md

Teoria 는 OpenMetadata 의 데이터 자산과 Business Ontology·Capability 를 연결하는 Metadata & Ontology Platform 이다. 처음 사용할 때는 시작 안내를 읽고, 기능을 고치거나 운영할 때는 해당 프로젝트의 문서로 이동한다. 명령은 별도 안내가 없으면 저장소 루트에서 실행한다.

문서 대조·취합 기준일은 **2026-10-16** 이다. 인쇄용 Word [docs/teoria_documentation_2026_10_16.docx]와 인쇄용 PDF [docs/teoria_documentation_2026_10_16.pdf]는 이 폴더의 Markdown 전체와 YAML 예제·템플릿을 파일명·원문 경로·목차·페이지 번호와 함께 취합한 시점별 사본이다. 수정의 기준은 원문 문서이며, 배포·수집의 실제 확인 범위와 남은 결손은 최신 검증 기록 [§48]을 따른다.

처음 시작하기

시작과 역할별 작업 [§02]: 로컬 실행, 데이터 운영·메타데이터 관리·개발의 순서

전체 아키텍처 [§04]: 구성요소, 데이터 흐름과 권위 경계

저장소 구조 [§05]: 코드·계약의 소유권과 의존 규칙

제품 로드맵 [§03]: 다음 결과와 완료 조건

		작업별 문서 찾기

하려는 작업

먼저 읽을 문서

다음 문서

전체 스택 실행·설정

Compose [§ 35]

환경변수·인증 [§ 36]

처음 Prefect 파이프라인 작성

PostgreSQL ETL·검증·배포 따라 하기 [§ 34]

개발 안내 [§ 29], Connector [§ 26]

수집 기능 추가·수정

Pipelines 개발 [§ 29]

Connector 작성 [§ 26], Data DB [§ 27]

수집 실행·실패 복구

Prefect 운영 [§ 33]

배포 목록 [§ 28], 조달 수집 [§ 31]

메타데이터 수집

OpenMetadata Job [§ 32]

Server 배포·업그레이드 [§ 39]

법령 수집·버전 조회

법령 수집 [§ 30]

진단용 법령 검색 [§ 16]

자산 탐색·설명 변경

Admin UI [§ 06]

자산 맥락·샘플·프로파일 [§ 15]

자산 진단·제안 적용

자산 진단 [§ 14]

제안 검토·적용 [§ 17], Codex Skills [§ 45]

Ontology·Binding 변경

의미 모델과 이관 [§ 18]

Runtime bundle 배포 [§ 23]

Capability·Source·Mapping 개발

Platform 개발 [§ 08]

Registry 작성 [§ 20]

질문 계획·실행

Context [§ 07]

Runtime [§ 23]

MCP 연결

MCP Gateway [§ 40]

외부 서비스 연동 [§ 42]

Provider wire 처리 수정

Provider API 라이브러리 [§ 41]

각 소유 프로젝트의 Source·Connector 작성 문서

업무 정책과 외부 연동

공공조달의 의미·계산 기준은 Platform, 수집·API 보강 절차는 Pipelines 가 소유한다. 첨부문서 처리와 문서 기반 참가자격 판단은 보류되어 활성 기능에 포함하지 않는다.

계약 집계 기준 [§ 10]과 활동·참여 집계 [§ 09]

공고 상태 [§ 12]

분야 분류와 확장 계획 [§ 13]

시장 분석 구현 [§ 11]

입찰체크 연동 [§ 42], 탐색 계약 [§ 43]

Source 작성 Skill [§ 46]

검증 기록과 배포 범위

검증 기록 목차 [§ 52]는 특정 날짜·version 에서 확인한 결과와 한계를 보존한다. 현재 설정·스케줄·기능 수로 읽지 않는다. Kubernetes 는 EC2 K3s 자동 배포 [§ 38]와 로컬 Compose 를 구분한다.

문서 작성과 정리 기준

문서 위치는 소유 프로젝트를 따른다. Platform 의 조회·진단과 Pipelines 의 수집은 같은 외부 시스템을 사용해도 구분한다. 전체 설계는 `architecture/`, 서비스 조립은 `deployment/`, 외부 호출 계약은 `integration/`, Skill 사용은 `skills/`에 둔다.

각 문서는 독자가 해결할 질문을 먼저 밝힌다. 개념 설명, 실행 절차, 계약 참조를 구분해 작성하고 같은 작업을 설명하는 짧은 문서는 합친다. 변경 이유·독자·검증 범위가 다른 정책은 독립 문서로 유지한다. 프로젝트 개발 안내는 코드 위치와 수정·검증 순서를 설명하고 상세 계약은 링크한다. 새 파일·폴더는 내용이 생길 때 만들며 빈 분류를 미리 만들지 않는다.

같은 규칙·설정·스케줄은 기준 문서 한 곳에서 관리한다. 실행 계약·Provider 원문·SQL 은 소유 프로젝트에 유지하고 문서는 그 위치를 안내한다. 배포 목록은 YAML 에서 생성한다. 시점별 수치·실행 ID·검증 환경은 `verification/`에, 다음 구현 범위는 로드맵에 둔다.

문서를 이동·통합할 때는 본문 링크, AGENTS·Skill 지침, 템플릿 참조, 문서 생성 코드와 테스트의 경로를 함께 바꾸고 링크를 검증한다. 이번 소유권별 재구성에서는 기존 `projects/`·`ingestion/`·`registry/`·`domains/` 문서를 해당 프로젝트로 옮겼다. 기존 저장소 내부 참조는 새 기준 문서를 가리키도록 갱신했으며, 외부에서 저장한 이전 경로는 Git 이력에서 찾을 수 있다. 루트 README 를 제품 입구로 유지하고 하위 README 는 만들지 않는다.

문서 변경 검증

문서를 추가·이동·통합하면 링크·제목 anchor 와 목차 도달 여부를 검사한다. 자동 생성 배포 목록은 직접 편집하지 않고 YAML 변경 후 재생성한다.

```
python3 -m unittest discover -s tests/documentation
python3 tests/documentation/check_docs.py
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs --check
docs/pipelines/deployments.md
```

`tests/documentation/check_docs.py` 는 README·AGENTS·docs·저장소 Skill 의 Markdown 에서 로컬 링크를 검사하고 `docs/index.md` 에서 모든 문서에 도달하는지 확인한다. 외부 URL 은 요청하지 않는다. 기본 ATX 제목(# 형식)의 fragment 도 검사한다. CI 의 `documentation` Job 이 같은 명령을 실행한다. 법령 수집 문서의 이전 `pipelines/law_collection.md` 경로는 `pipelines/domains/legal_information.md` 로 이동했고 내부 참조를 함께 갱신했다. 계약 집계에서 옮긴 활동·참여 기준은 별도 문서로 연결한다.

EC2 자동 배포는 K3s 배포 [§ 38]를 따른다.

02. platform-guide.md — Teoria 시작과 역할별 작업

원문 파일: docs/platform-guide.md

Teoria 는 데이터를 수집하고, OpenMetadata 로 자산을 설명하고, Ontology·Binding 으로 업무 의미를 연결한 뒤 Capability 와 Context 로 요청을 실행한다. 공공조달은 이 경로를 검증하는 첫 업무 도메인이다. 구성요소의 책임은 전체 아키텍처 [§04]를 따른다.

로컬 스택 시작

저장소 루트에서 환경 파일을 복사하고 의존성을 설치한다.

```
cp deploy/compose/.env.example deploy/compose/.env
uv sync --locked --all-packages --all-groups
```

`.env` 의 필수 token·DB 암호·Prefect 인증정보를 준비한다. 현재 기본 배포는 첨부문서 처리용 S3 호환 저장소를 요구하지 않는다. 설정 이름과 기본값은 환경 설정 [§36], 서비스 시작 조건은 Compose [§35]를 따른다.

	`docker compose --env-file deploy/compose/.env \ -f deploy/compose/compose.yaml up --build -d`

대상

기본 주소

Admin UI

`http://localhost:8081/`

Admin API 문서

`http://localhost:8081/admin-api/docs`

Runtime API 문서

`http://localhost:8081/runtime-api/docs`

Prefect UI

`http://localhost:8081/prefect/`

OpenMetadata 가 필요하면 `--profile metadata` 를 추가한다. Server 는 기본 `http://localhost:8585/`에서 확인한다. Server 시작만으로 메타데이터 수집이 실행되지는 않는다. OpenMetadata 배포 [§39] 후 수집 Job [§32]을 실행한다.

데이터 운영자: 수집부터 확인하기

수집 코드는 Compose 와 K3s 가 공유한다. K3s 에서는 작업별 Job 과 자원 예산·동시 실행 상한으로 병렬 실행을 제어한다. Compose 는 Worker 전체 자원 상한을 사용하며 최신 수집 우선·Backfill 동시 실행 제한은 양쪽에 적용한다. 공통 정책은 Prefect 운영 [§33], K3s 설정과 검증 범위는 K3s 배포 [§38]를 따른다.

배포 목록 [§28]에서 수집 계약·Flow·파라미터를 찾는다.

Prefect [§33]에서 실행하고 실패 Task 와 재시도를 확인한다.

수집 정의의 원본 보존 정책과 정규 적재 결과를 확인한 뒤 checkpoint 이동을 확인한다.

OpenMetadata Job 을 실행해 수집된 테이블·view 를 catalog 에서 찾는다.

조달의 최신·보정·Backfill 수집은 공공조달 수집 [§31], 법령 원문·시행 버전은 법령 수집 [§30]을 따른다. Flow 를 수정할 때는 Pipelines 개발 안내 [§29]에서 구현과 검증 위치를 찾는다.

첨부파일 다운로드·문서 입찰요건 추출·참가자격 판단은 보류되어 archive [archive/bid_document_processing/README.md]에 보관한다. 현재 배포와 API 에서 제공하지 않는다.

Metadata 관리자: 자산을 이해하고 변경하기

Admin 데이터 카탈로그 [§06]에서 자산과 컬럼을 찾는다.

자산 맥락 [§15]에서 기술·운영·의미·거버넌스 정보를 확인한다.

설명 변경안을 직접 작성하거나 자산 진단 [§14]을 요청한다.

제안 검토·적용 [§17]에서 현재 값·근거·대상 version 을 비교한 뒤 승인하고 적용 결과를 확인한다.

OpenMetadata entity 는 Teoria DB 로 복제하지 않는다. AI 결과는 Suggestion 이며 승인과 적용 성공 전에는 권위 metadata 가 아니다. 개방 진단은 법령 근거를 가진 검토 초안이며 법률 적용성이나 공개 승인을 확정하지 않는다. Codex 로 작업하려면 진단 Skills [§45]의 별도 실행·제출 절차를 따른다.

Ontology 담당자: 의미를 연결하고 게시하기

자산과 Glossary 를 확인하고 Stable Concept 에 Binding 을 작성한다. 기본 경로는 `Physical Asset → OpenMetadata Glossary Term → Teoria Ontology` 다. 직접 자산 Binding 은 보조 연결이며 용어 경로보다 우선하지 않는다.

Binding Draft 를 검토·승인하고, Ontology 변경은 현재 Published version 에서 새 Draft 를 만들어 검토·승인·게시한다. Ontology 와 이관 [§18]을 따른다. 변경된 의미를 운영 실행에 반영하려면 Runtime bundle [§23]을 구성·활성화하고 재배포한다.

개발자: 실행 기능 추가하기

요청 시 직접 호출하는 API 는 Source [§22], 지속 수집하는 API 는 Connector [§26]로 작성한다. Pipeline 이 적재한 DB 를 조회할 때는 Database Source 와 Mapping 을 작성한다. 같은 API 를 양쪽에 중복 등록하지 않는다.

Platform 개발 안내 [§08]에서 기능의 Domain 과 진입점을 찾고, Registry 안내 [§20]에 따라 Mapping·Capability 를 정의한다. 조달 계산을 바꿀 때는 집계 기준 [§10]을 먼저 확인한다. 검증한 계약은 bundle 로 발행하고 API version 을 확인한다.

클라이언트 개발자: 기능 발견과 호출

MCP Gateway [§40] 또는 Runtime HTTP API 로 공개 Capability 를 발견한다. 질문의 의미 조회·계획·실행은 Context [§07]를 참고한다. 입력 schema 는 discovery 에서 가져오고 결과의 provenance·페이지·부분 실패를 확인한다. 외부 서비스의 권장 호출 순서는 입찰체크 연동 [§42]을 따른다. MCP 에는 Source 키나 DB 권한을 주지 않는다.

변경 검증과 다음 단계

프로젝트별 테스트와 계약 검증은 Platform [§08], Pipelines [§29], MCP [§40], Provider API [§41]의 개발 안내를 따른다. 과거 종단 실행 결과는 검증 기록 [§52], 남은 제품 결과는 로드맵 [§03], 전체 문서 탐색은 목차 [§01]에서 확인한다.

EC2 의 K3s 배포와 main push 자동 배포는 K3s 배포 안내 [§38]를 따른다.

03. roadmap.md — Teoria product roadmap

원문 파일: docs/roadmap.md

로드맵은 완료된 migration 일지가 아니라 남은 제품 결과와 완료 조건만 관리한다. 현재 구현 사실은 코드, Registry 와 Architecture [§04]가 기준이다.

현재 제공하는 기반

Pipelines [§29]: Connector · Prefect 수집, 원본 · 정규 적재와 Data DB

Prefect 실행 정책 [§33]: 두 배포의 공통 등록 · Queue 우선순위 · 동시 실행 상한. Compose 는 Worker 전체 제한, K3s [§38]는 작업별 Job · 자원 예산을 적용한다. 실제 EC2 에서의 병렬 실행 · 자원 포화 검증은 남아 있다.

Platform Runtime [§23]: Source · Mapping · Capability 실행, Context 계획 · 실행과 OPA 정책 연계

Ontology 와 Binding [§18]: 작성 · 검토 · 게시, Stable Concept 와 immutable bundle

Metadata 관리 [§14]: OpenMetadata 조회, AI 제안 · 검토 · 적용 · 복구

Admin UI [§06]와 MCP [§40]: 관리 · 실행 기능의 사용자 진입점

HTTP · DB native async 실행과 연결 상한: 계산 · 검증은 일반 함수로 유지하고 승인 · 적용 · checkpoint 순서와 취소 시 rollback 을 보존한다. 로컬 Compose 반영과 실제 실행은 2026-10-16 검증 [§48]에서 확인했다.

구현 파일 · 이관 이력은 각 프로젝트의 개발 안내와 Git 이력에서 확인한다. 실제 질문과 배포 환경의 검증 범위는 검증 기록 [§52]에 남긴다.

다음 결과

1. Context Engine vertical slice

현재 API · 지원 질문 · 실행 제약은 Context [§07]를 따른다.

승인된 Ontology, Binding 과 Capability 를 사용해 질문을 실행계획으로 변환한다.

완료 조건:

계획에 사용한 concept, capability, source 와 artifact version 을 설명한다.

접근권한과 시간 범위를 실행 전에 검증한다.

부분 실패와 불확실성을 결과에 보존한다.

현재 공고 검색과 계약금액 질문의 해석 · 집계는 공공조달 패키지에 분리되어 있다. 이 구조는 범용 자연어 planner 의 완료를 뜻하지 않는다. 다른 실제 업무 Domain 을 추가할 때 공통 Runner · Context Engine 수정 없이 handler 를 연결할 수 있는지 검증하고, 그때 확인된 재사용만 공통 계약에 반영한다.

2. 검증된 Capability 와 사용자 평가

대표 질문에 필요한 소수 Capability 의 Ontology, OpenMetadata, Binding, Mapping 과 provenance 를 끝까지 완성하고 실제 사용자 평가로 가치를 검증한다. 검증되지 않은 Capability 는 운영 준비 상태와 명확히 구분한다.

대표 질문의 종단 검증 결과와 당시 metadata 결손은 Context vertical 검증 보고서 [§56]와 Bid notice vertical 검증 보고서 [§50]에 기록한다. 현재 준비도는 배포된 계약 · 실행 결과 · 근거의 최신 상태를 대조해 평가한다.

3. 필요한 Admin 관리 UX

Context vertical 에서 반복적으로 확인된 병목만 기존 관리 surface 에 추가한다. Workspace 를 별도 권위 lifecycle 로 만들지 않고 Capability, Suggestion, Binding, Ontology 와 OpenMetadata 의 권위 상태를 직접 보여준다. Authoring

Workspace 를 사용할 경우에도 이는 이 상태들을 조율하는 작업 단위일 뿐 release readiness 나 승인 상태의 권위가 아니다. 범용 Mapping builder 나 Source editor 를 제품 가치 검증보다 먼저 만들지 않는다.

Suggestion 적용은 자동 승인이 아니다. 사람이 최소 변경 단위로 승인한 뒤 대상 version 을 재검증하고, OpenMetadata 변경 또는 Teoria Draft 생성을 수행하며 결과를 `applied` 또는 `failed` 이력으로 남긴다. Ontology 변경은 게시가 아닌 Draft 생성까지만 자동화한다. 데이터 자산 상세의 수동 description 편집도 같은 Suggestion 경로를 사용한다. 제출자는 인증 principal 로 남기고, 원천의 현재 값과 version 을 근거로 고정해 승인 전 변경과의 충돌을 차단한다.
owner · domain · classification · Glossary 편집 UI 는 각 OpenMetadata 참조 선택과 제거 의미, stale evidence 계약을 갖춘 뒤 이 흐름으로 확장한다.

메타데이터 인텔리전스의 사용자 작업명은 자산 진단이다. 개발 단계에서는 Data Catalog 에서 대상 FQN 을 포함한 세 Skill 요청을 복사한다.

메타데이터 정비 (`metadata_enrichment`, `$enrich-metadata-asset`): 설명 · Glossary 제안

업무 의미 연결 (`semantic_alignment`, `$align-metadata-semantics`): 기존 Published Concept Binding 제안

개방 적합성 진단 (`disclosure_assessment`, `$assess-data-disclosure`): 정보 유형 · 법령 근거 · 개방 상태 · 보호조치 검토 초안

정비와 의미 연결은 기존 Suggestion→Review→Application 경로를 사용한다. 새 분석 유형 ID 는 설계 용어이며 저장된 Suggestion enum 과 API 를 변경하지 않는다. 신규 업무 Ontology 제안은 보류한다. 비동기 첫 구현은 `intelligence.asset_assessments` 와 `metadata-assessment-worker` 를 사용한다. 수동 시작 · 상태 조회 · 설명 및 기존 Concept 직접 Binding 제안 등록 · 법령 검토 findings 를 제공한다. 모델 gateway 설정 후 실행할 수 있으며 실제 모델 품질 평가는 별도다. 법령 진단의 구조화된 Suggestion 계약 · 태그 적용기, 원천 문서 자동 검색, 자동 Glossary 연결 · 주기 진단 · 재분석 · 품질 지표는 후속 범위다.

4. Semantic MCP/SDK

Capability 목록을 그대로 노출하는 수준을 넘어 semantic discovery 와 provenance 를 안정된 client 계약으로 제공한다.

완료 조건:

MCP 와 SDK 가 같은 Runtime API 계약을 사용한다.

schema 를 client 코드에 복제하지 않고 discovery 로 협상한다.

pagination, timeout, retry 와 오류 의미가 문서화되고 회귀 테스트된다.

5. Production identity and scale

Admin OIDC 와 세분화된 role 을 OPA principal 계약에 연결

Runtime service identity, token rotation, quota 와 영속 decision/execution audit

Data/Control plane 의 독립 backup · restore 검증

Kubernetes workload 와 secret 계약 완성

suggestion 품질 평가셋과 승인 성과 측정

수집 처리량과 데이터 완전성을 분리한 운영 지표, 개찰 보강 잔량 · 부분완료 · 조회 실패 및 계약 미해결 구간의 복구. 현재 잔량은 날짜별 검증 기록으로 관리한다.

의도적으로 미루는 항목

Ontology canvas, 범용 graph editor, 자동 승인, OpenMetadata entity 복제, Airflow 추가, 다중 workflow engine 은 위 vertical slice 의 운영 가치가 검증되기 전에는 구축하지 않는다.

공공조달의 버전형 분야 분류체계는 별도 Domain 기능으로 구축한다. 그 전까지 물품과 공사는 원천 공식 분류번호·분류명의 단일 단계를 사용하고, 코드 접두어로 상위 계층을 추정하지 않는다. 수집원, 기간 유효 분류와 최신 체계 재분류의 완료 조건은 공공조달 분야 분류 기준과 확장 계획 [§13]을 따른다.

법령 기반 데이터 거버넌스

법령 근거 저장소는 Pipeline 의 시행일별 원문 수집·구조화·재가공 기능을 사용한다. 다음 결과는 승인된 법령-정보 유형 매핑 규칙, 내부 처리와 외부 개방의 분리 평가, 법령 변경 영향 검토 및 Suggestion→Review→Application 을 통한 OpenMetadata 반영이다. 업무 Ontology 는 실제로 사용하는 보호대상 유형·처리 목적·개방 시나리오부터 정의하며 조문 원문이나 개별 평가 응답을 Business Object 로 만들지 않는다.

현재 Admin 진단 기반은 OpenMetadata 저장 샘플·프로파일의 제한된 조회와 법령의 키워드 검색·snapshot 원문 조회를 제공한다. 기본 메타데이터 ingestion 은 샘플과 프로파일을 자동 수집하지 않는다. 다음 검증 범위는 대상 테이블을 명시한 관측 수집 workflow, AI 의미 분석·근거 생성과 태그 제안 및 검색 평가다. 배포 호환성 결과는 OpenMetadata 검증 기록 [§53], 현재 제안 비교·적용 이력과 화면 사용법은 Admin UI [§06]를 따른다.

조달 탐색의 고정 snapshot·과거 시점 재현·관계 확장과 성능 검증의 후속 범위는 조달 탐색 연동 계약 [§43]을 따른다.

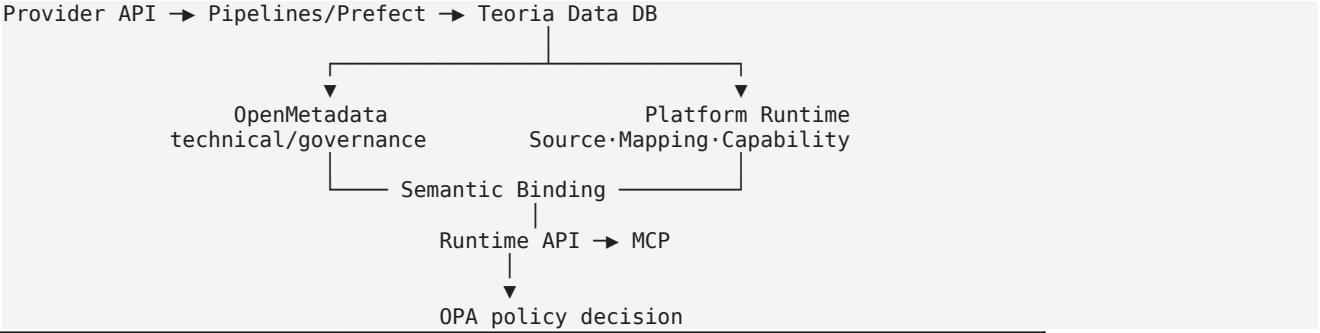
EC2 1 대의 K3s 배포와 GitHub CD 경로는 배포 안내 [§38]에 구현되어 있다. 실제 EC2 연결·배포 검증은 서버·도메인·Secret 설정 후 수행한다.

04. overview.md — Teoria architecture

원문 파일: docs/architecture/overview.md

Teoria 는 수집 시스템, metadata catalog, 업무 의미와 실행 계약을 분리한다. 이 문서는 현재 구조의 기준이며 프로젝트별 파일 소유권은 Repository structure [§05]를 따른다.

Teoria 의 제품 경계는 특정 조달 서비스가 아니라 여러 업무 도메인의 데이터와 의미를 연결하는 AI-native metadata/ontology platform 이다. 공공조달은 Source, Ontology, Binding, Capability 와 Context 실행을 끝까지 검증하는 첫 vertical domain 이다. 조달 고유 규칙은 Domain Registry 와 기능 패키지 안에 두고 Platform 공통 계층으로 끌어올리지 않는다.



		권위 경계

시스템

권위 데이터

두지 않는 것

Prefect

workflow 실행, schedule 과 retry

Ontology 와 metadata 의미

Teoria Data DB

raw 관찰과 정규화된 업무 데이터

governance metadata

OpenMetadata

asset, schema, glossary, owner, lineage, quality, usage

Business Ontology

Teoria App DB

Ontology, Binding, Suggestion, Review, context 설정

OpenMetadata entity 본문

Runtime Registry

Source, Mapping, Capability 실행 계약

authoring 상태와 수집 workflow

OpenMetadata entity 는 복제하지 않고 `MetadataTargetRef` 의 system, entity ID, entity type, FQN 과 필요한 version 만 저장한다. Capability 는 독립된 knowledge source 이며 `CapabilityTargetRef` Binding 으로 Ontology 와 연결한다.

실행과 데이터 흐름

Pipeline 은 Provider API 를 지속 수집하고 정의된 원본 보존·정규 적재 단계를 완료한 뒤 checkpoint 를 갱신한다. 계약·공고·업종은 Raw 를 보존하고, 법령은 전용 XML 보관 구조를 사용한다. 낙찰·개찰은 `retain_raw_records: false` 계약에 따라 성공 응답 본문을 보존하지 않으며 정규 행의 원천 hash 와 실행 이력을 남긴다.

Pipeline 의 Flow·Task·client·정규화는 업무 기능별 패키지에 모은다. 실행 기록과 checkpoint Task 는 `execution/`, 저장은 `persistence/`, OpenMetadata ingestion Job orchestration 은 `metadata/`가 소유한다.

Runtime 은 직접 호출 Source 또는 Data DB 의 read-only relation 을 Mapping 으로 runtime object 에 변환하고 Capability 를 실행한다.

Context Engine 은 질문을 versioned Semantic Query Plan 으로 만들고 계획과 실행을 분리한다. 두 단계 모두 OPA 에 동일한 principal·action·resource 계약으로 정책을 질의하며 정책 서비스가 응답하지 않으면 운영 요청을 허용하지 않는다.

OPA 정책은 이미지에 직접 적재하지 않는다. 전용 build job 이 versioned bundle 을 RSA 로 서명하고 Policy Control Plane 이 이를 배포한다. OPA 는 공개키 검증에 성공한 bundle 만 활성화하고 decision log 와 agent status 를 Application DB 로 전송한다.

MCP 는 Runtime HTTP API 만 호출하며 DB 권한이나 Provider credential 을 갖지 않는다.

Provider wire 계약과 HTTP 실행은 `teoria-provider-api` 만 담당한다.

Pipeline 실행 코드는 Platform Capability 나 Mapping Runtime 을 호출하지 않는다.

공통 Capability Runner 는 명시적으로 등록된 processor 와 query 후처리를 실행한다. Company 와 공공조달 구현은 `platform/src/teoria/domains/`에 두고 composition 에서 조립한다. Context Engine 은 의미 조회와 정책 검증을 소유하며 조달 질문 해석· 집계는 공공조달 query handler 가 담당한다.

저장소와 배포

HTTP·DB I/O 는 API·Service·Repository 와 Prefect DB Task 까지 native async 로 연결한다. 계산·정규화·검증은 일반 함수를 사용한다. DB 연결은 event loop 별·접속 URL 별로 제한하며 같은 transaction 의 순서와 승인·적용·checkpoint 경계는 유지한다. 연결을 빠져나가면 성공 시 commit, 예외·취소 시 rollback 후 닫는다. 자세한 상한과 구현 위치는 Platform [§08]과 Pipelines [§29] 개발 안내를 따른다.

로컬 Compose 는 Data Plane 인 `teoria_data` 를 별도 PostgreSQL 에 두고, `teoria_app`, `prefect`, `openmetadata_db` 는 database 와 role 을 분리해 Control/Application PostgreSQL 에 둔다. database 간 SQL join 은 허용하지 않는다. 부하, 장애 격리 또는 backup 정책이 달라지면 connection URL 을 변경해 별도 인스턴스로 분리한다.

수집 Flow·Task 는 두 배포가 공유한다. Compose 는 Process Worker, K3s 는 별도 `teoria-pipelines` namespace 의 실행별 Kubernetes Job 을 사용한다. ResourceQuota 와 work pool 동시 실행 제한으로 정해진 자원 예산 안에서 병렬 실행한다. 설정은 K3s 배포 [§38]를 따른다.

OpenMetadata ingestion 일정과 재시도는 Prefect 가 소유한다. 공식 ingestion image 를 ephemeral Job 으로 실행하며 일반 Prefect Worker 에 ingestion dependency 를 설치하지 않는다. 지원되는 Compose 는 격리된 Docker worker 를 사용

한다. K3s 에서는 전용 metadata Worker 가 네임스페이스 권한으로 공식 ingestion image 의 Kubernetes Job 을 생성한다. 이미지 allowlist 와 Secret 참조 범위를 제한하며, EC2 배포 [§ 38]의 실제 환경 검증은 서버 설정 후 수행한다.

변경 불변식

AI 결과는 Suggestion 이며 review, approval, 적용 성공 전에는 권위 정보가 아니다.

Admin 쓰기는 인증 principal 을 actor 로 기록한다.

Published Ontology revision 은 직접 수정하지 않는다.

Runtime 은 Published version 에서 생성한 checksum 검증 가능한 immutable artifact 를 사용한다. 운영 환경은 authoring Registry 경로를 직접 읽지 않으며 `TEORIA_RUNTIME_ARTIFACT_PATH` 또는 atomic active pointer 를 사용하는 `TEORIA_RUNTIME_ARTIFACT_STORE` 를 반드시 설정한다.

Application DB 와 Data DB migration 은 이미 적용된 파일을 수정하지 않고 새 순번을 추가한다.

Source 와 Connector 를 같은 API 에 중복 등록하지 않는다.

운영에서는 Admin 과 Runtime 에 별도 bearer credential 을 사용한다.

운영에서는 `TEORIA_POLICY_MODE=opa` 를 사용한다. `disabled` policy adapter 는 개발·테스트에서만 허용한다.

OPA readiness 는 최초 서명 bundle 활성화와 management plugin 상태를 포함한다.

Ontology 모델은 Ontology [§ 18], Registry 계약은 Registry guide [§ 20], 배포 방법은 Deployment [§ 35]를 참고한다.

05. repository-structure.md — 저장소 구조와 소유권

원문 파일: docs/architecture/repository-structure.md

새 기능을 추가하거나 기존 동작을 수정할 때 어느 프로젝트가 책임지는지 정하는 문서다. 전체 데이터 흐름과 권위 경계는 아키텍처 [§04]를 먼저 참고한다.

		프로젝트 선택

변경하려는 내용

소유 프로젝트

개발 안내

업무 의미, Ontology, Binding, 메타데이터 진단, 사용자 요청 실행

`platform/`

Platform [§08]

외부 데이터 수집, 정규화, 적재, Prefect, Data DB migration

`pipelines/`

Pipelines [§29]

MCP Tool, protocol 변환, Runtime HTTP client

`mcp/`

MCP Gateway [§40]

Provider API wire 계약과 HTTP 요청·응답 처리

`packages/provider_api/`

Provider API [§41]

서비스 조립, 이미지, 네트워크와 배포 설정

`deploy/`

Compose [§35]

공공조달은 첫 vertical domain 이다. 조달 고유 정책은 해당 Domain Registry 와 이름 있는 기능 패키지에 두며, 두 번째 Domain 에서 실제 재사용이 확인되기 전에는 Platform 공통 계층으로 승격하지 않는다.

통신과 의존 방향

```
pipelines ──SQL write──▶ Teoria Data DB ◀──SQL read── platform
platform  ──REST──────▶ OpenMetadata
mcp       ──HTTP──────▶ Platform Runtime API
platform  ──import────▶ teoria-provider-api ◀──import── pipelines
```

Pipeline 실행 코드는 Platform Capability 나 Mapping Runtime 을 호출하지 않는다. MCP 는 Runtime HTTP API 만 호출하고 Source 키·DB 권한·embedded Runtime 을 갖지 않는다. Pipeline 의 별도 통합 검증 진입점만 Platform Registry 를 지연 import 할 수 있다.

`packages/`에는 둘 이상의 프로젝트가 실제 사용하는 설치형 라이브러리만 둔다. 각 패키지는 독립 `pyproject.toml`, 테스트와 단일 책임을 가지며 소유 프로젝트를 역으로 import 하지 않는다. 미래 재사용을 위한 빈 패키지나 `common`, `shared`, `utils` 묶음을 만들지 않는다.

	계약과 설정의 기준 위치

목적

기준 위치

Runtime 이 직접 호출하는 API·DB

`platform/registries/sources/`

Runtime Object·Mapping·Capability

`platform/registries/domains/<domain>/`

Provider API 원문과 근거

각 프로젝트의 `references/providers/`

지속 수집 API 계약

`pipelines/connectors/`

수집 대상·Operation·output 계약

`pipelines/definitions/<domain>/`

수집 업무 입력

`pipelines/config/`

수집·Metadata Job 배포와 스케줄

`pipelines/prefect.yaml`, `pipelines/prefect.metadata.yaml`

두 배포의 Prefect 등록·Queue·동시 실행 정책

`pipelines/src/teoria_pipelines/deployment.py`

Data DB schema

`pipelines/database/migrations/`

Application DB schema

`platform/database/migrations/`

Ontology 이관 입력과 적용 이력

`platform/ontology_migrations/pending/`, `applied/`

로컬 개발·통합 배포

`deploy/compose/`

EC2 K3s 배포·GitHub CD

`deploy/k3s/`, `.github/workflows/deploy_k3s.yaml`

같은 수집 API 를 Source 와 Connector 에 중복 등록하지 않는다. Pipeline 이 적재한 DB 를 Runtime 이 읽으면 Database Source 와 Mapping 을 작성한다. 적용한 SQL migration 과 게시한 Ontology·Runtime artifact 는 수정하지 않고 새 migration 또는 version 을 만든다.

기능별 코드 배치

실행 코드·타입·저장소 구현은 소유 Domain 의 기능 패키지에 모은다. 공통 계층은 실제 재사용하는 기반 코드만 소유하고 Domain 구현을 역으로 import 하지 않는다. Flow 와 HTTP·CLI 진입점은 실행 순서와 조립을 보여주며, 큰 업무 책임은 이름 있는 모듈로 분리한다. 파일 수나 줄 수만을 이유로 나누지 않는다.

Platform 의 Runtime·Admin·Context 조립과 Domain 등록은 Platform 개발 안내 [§08], Flow·Task·정규화·repository 의 배치는 Pipelines 개발 안내 [§29]를 따른다.

테스트 배치

테스트는 검증 대상의 소유권을 따른다. 프로젝트 단독 단위·DB 통합 검증은 각 프로젝트의 `tests/`에 두고, 루트 `tests/`에는 저장소 공통 검사와 프로젝트 사이 계약·연동 검증을 둔다. DB 를 사용한다는 이유만으로 루트 통합 테스트로 분류하지 않는다. 배포 설정 검증은 `deploy/compose/tests/`가 소유한다. 기능별 테스트 경로는 `unit/domains/<domain>/<feature>/`처럼 소유 코드와 대응하며 공통 기반 검증은 `unit/`에 유지한다. 프로젝트 간 테스트가 다른 프로젝트의 테스트 모듈을 import 하지 않는다.

이름과 문서 배치

Teoria 소유 경로와 내부 ID 의 기본 형식은 `snake_case` 다. 클래스·React 컴포넌트는 `PascalCase`, 환경변수는 `UPPER_SNAKE_CASE`, CLI·Docker·배포 리소스는 `kebab-case` 다. 도구가 고정한 이름과 외부 Provider 의 wire 표기는 유지한다. 기존 문서의 `kebab-case` 이름은 호환 계획 없이 이름만 바꾸지 않는다.

설명 문서는 `docs/`에 모으고 코드 디렉터리에 README 를 분산하지 않는다. `docs/`에는 문서와 문서용 예제·템플릿만 둔다. 문서 검사기와 회귀 테스트는 `tests/documentation/`에 함께 두며 루트 `tools/`를 추가하지 않는다. 문서는 `platform/`,

pipelines/, mcp/, packages/의 소유권을 따르며 전체 설계는 architecture/, 서비스 배포는 deployment/, 외부 연동은 integration/에 둔다. 현재 계약과 절차는 계속 갱신하고, 날짜·버전에 종속된 결과는 verification/에 보관한다. 문서 작성·통합 기준과 읽는 순서는 문서 목차 [§01]를 따른다.

06. admin_ui.md — Admin UI 사용과 개발

원문 파일: docs/platform/admin_ui.md

Admin UI 는 자산을 탐색하고 업무 의미·연결·변경 제안을 관리하는 화면이다. 전체 스택 실행 후 `http://localhost:8081/`에 접속하면 `/admin/overview`로 이동한다.

		작업에 맞는 화면 찾기

하려는 작업

메뉴와 URL

확인할 내용

연결·최근 활동 확인

현황 `/admin/overview`

자산·연결 상태, 조회 범위 내 최근 검토

테이블·컬럼 탐색

데이터 카탈로그 `/admin/data-catalog`

설명·소유자·태그, 스키마·계보·적재 상태

업무 개념 작성

의미 모델 `/admin/semantic-model/ontology`

version, 개념·관계·규칙, Draft 검토·게시

의미 연결 검토

의미 연결 `/admin/semantic-model/bindings`

자산·Glossary·Concept 참조와 승인 상태

관리 공백 확인

거버넌스 /admin/governance

소유자·분류 등 관리 공백

변경안 승인·복구

검토·적용 /admin/changes/review

현재 값·제안·근거·적용 이력

운영 상태 확인

운영 /admin/operations

원천별 연결과 작업 상태

실행 계약 확인

기능 계약 /admin/developer/capabilities

의미·데이터·실행 준비도

질문 실행 확인

질문 테스트 /admin/developer/ask

계획·실행 결과와 근거

현황의 제안·최근 활동은 조회한 범위의 결과이며 전체 누적 통계가 아니다. 자산의 상세 탭, 원천별 조회 범위와 장애 표시는 자산 맥락 [§15]에 설명한다.

자산을 조회하고 설명 변경안 만들기

데이터 카탈로그에서 스키마를 고르고 자산을 검색한다. 설명·소유자·태그 누락 필터를 사용할 수 있다.

자산을 열어 개요·스키마·의미·거버넌스·품질/프로파일·계보·운영 탭을 확인한다.

테이블의 설명 변경안 또는 컬럼의 설명 변경에서 변경안을 제출한다.

검토·적용에서 원천의 현재 값, 제안 값, 제안 당시 근거 version 을 비교하고 승인한다.

제출은 pending Suggestion 을 만든다. OpenMetadata 반영은 승인 후 application service 가 대상 version 을 재검증한 뒤 수행한다. 최신 자산 조회 실패는 빈 값으로 표시하지 않는다. 목록 검색·스키마·페이지는 URL 에 유지되며 상세를 보고 돌아와도 탐색 상태를 복구한다.

진단과 적용 결과 확인

자산 상세의 자산 진단에서 진단 항목과 최대 30 개 대상 컬럼을 선택하고 실행한다. 상태·제안 링크·법령 검토 findings 를 확인한 뒤 제안을 별도로 검토한다. 비동기 Worker 설정은 자산 진단 [§14], 공통 승인·적용·복구는 제안 처리 [§17]를 따른다.

제안 목록은 서버가 상태와 자산의 정확한 참조를 필터링하고 cursor 로 페이지를 조회한다. 적용 이력 보기에서 시도별 실행자·시각·결과·오류를 확인한다. 종료 기록이 없는 작업을 성공으로 단정하지 않고 서버의 재시도·기존 결과 확인·재분석 안내를 따른다. Binding Draft 와 Ontology Draft 는 생성된 정확한 초안으로 이동하며 별도 승인을 거친다.

개발용 Skill 요청 복사도 제공한다. 정비·의미 연결·개방 진단 요청의 범위와 제출 방법은 진단 Skills [§45]를 따른다. 비동기 Worker 와 Skill 실행은 서로 다른 진입점이다.

관리자 연결

상단 관리자 연결에서 Admin token 을 입력하면 /v1/admin/auth/session 으로 확인한다. 공개 metadata status 응답을 인증 성공으로 사용하지 않는다. token 은 브라우저 sessionStorage 에만 보관하고 로그아웃하면 제거한다. 입력은 기본 마스킹되며 토큰 보기를 누르는 동안만 표시한다. 인증 실패 시 연결 상태를 확인하고 다시 입력한다.

사용하는 값은 TEORIA_ADMIN_API_TOKEN 이며 OpenMetadata bot token 과 다르다. 설정 줄·따옴표·Bearer 접두사를 붙여 넣은 경우 입력 형식을 정리한다. 운영 API 는 bearer 인증과 principal 별 역할을 사용한다. 토큰 없는 카탈로그 조회 여부와 샘플 조회 권한은 환경 설정 [§36], 자산 맥락 [§15]을 참고한다.

프론트엔드 개발

UI 는 `platform/admin_ui/`, Admin API 는 `platform/src/teoria/admin/`가 소유한다.

```
Admin UI → Admin API → OpenMetadata / Application DB / Data DB read view
MCP → Runtime API → Capability Runner
```

저장소 루트에서 Admin API 를 실행하고 별도 터미널에서 UI 를 시작한다.

```
uv run --locked --package teoria-platform teoria-admin-api
cd platform/admin_ui
npm ci
npm run dev
```

UI 변경은 같은 디렉터리에서 `npm test` 와 `npm run build` 로 검증한다. Compose 는 UI·Admin API 를 직접 공개하지 않고 nginx 가 `/`와 `/admin-api/`로 전달한다. API 문서는 `http://localhost:8081/admin-api/docs`, Runtime 문서는 `http://localhost:8081/runtime-api/docs` 다. 개발 API 의 기본 포트는 8001 이다.

화면·테스트·스타일은 `src/features/<feature>/`에 함께 둔다. `src/api/`는 metadata, suggestions, asset_assessment, ontology, binding, capability, context, registry, auth 별 계약과 호출을 소유한다. HTTP 전송·token·오류 처리는 `http_client.ts` 를 공유한다.

`components/workspace/WorkspaceLayout.tsx` 와 `workspace.css` 는 공통 페이지 틀을 제공한다. 데이터 카탈로그의 여백·중립적 선택 색상·구분선을 공통 기준으로 사용한다. 화면별 API 를 직접 사용하고 기능을 모두 묶는 `admin.ts` 객체를 다시 만들지 않는다.

직접 URL·새로고침·새 탭 접근은 nginx SPA fallback 으로 지원한다. 이전 `/admin/business-concepts`, `/admin/semantic-bindings`, `/admin/review-queue`, `/admin/settings`, `/admin/capabilities`, `/admin/ask` 는 현재 경로로 redirect 한다.

화면 코드의 책임

`features/access/AdminAccess.tsx` 는 관리자 세션을, `AdminLoginDialog.tsx` 는 토큰 입력·정규화와 로그인 요청을 담당한다. 자산 진단의 선택·요청은 `AssetAssessmentPanel.tsx`, 실행 이력·근거·제안 링크 표시는 `AssessmentHistory.tsx` 에 둔다.

`api/http_client.ts` 의 `AdminApiError` 는 HTTP 상태와 서버 오류 코드를 보존한다. 인증·권한·진단 오류 안내는 `status` 와 `code` 로 판단하며 메시지 문자열을 검색하지 않는다.

07. context.md — Context 의 의미 조회·계획과 실행

원문 파일: docs/platform/context.md

Context 는 업무 표현을 승인된 의미·Capability 와 연결하고, 허용된 질문을 실행계획으로 바꾼 뒤 Runtime 을 호출한다. 현재 기본 handler 는 공공조달의 정해진 질문 형식을 처리한다. 임의 업무 질문을 해석하는 범용 자연어 planner 의 완료를 뜻하지 않는다.

	현재 지원 범위

질문 유형

처리

기관의 최근 N 년 계약금액

기관 식별, 승인된 analytics 경로 선택, 계약 조회·페이지 집계

공고 검색

기간·업무구분·접수 상태·지원 키워드 등을 해석해 `search_bid_notices` 실행

예를 들어 `근로복지공단의 최근 5년 계약금액을 알려줘`는 계약금액 질문 형식이다. 기간은 `as_of` 와 질문에서 계산한다. 의미·기관·승인 경로를 찾지 못하면 실행하지 않는다. 계약 검색 결과 집계는 프로필의 계약사건 집계와 자동으로 같은 의미가 되지 않으므로 계약 집계 기준 [§10]과 혼용하지 않는다.

요청 처리 순서

```
질문·기준일 → context.plan 정책 검사 → 의미·기관·실행 경로 조회 → 계획
→ context.execute 정책 검사 → Runtime Capability 실행 → 집계·근거·불확실성 반환
```

계획에는 `schema_version`, 질문·기준일, concept·ontology, Capability ID, inputs 와 해석·선택 이유·근거·경고가 들어간다. 계획의 정책 결정과 실행의 정책 결정은 구분한다. 실행 결과는 부분 실패, freshness 와 metadata quality 상태를 보존한다.

계획 작성 중 기관 식별·지원 계약 확인을 위해 Runtime 조회를 할 수 있다. `plan` 을 외부 호출이 전혀 없는 순수 계산으로 간주하지 않는다.

	Admin API 사용

API

입력과 동작

`GET /v1/admin/context/resolve?term=계약금액&purpose=analytics`

게시된 의미·승인된 연결과 목적별 경로 조회

`POST /v1/admin/context/plan`

질문과 선택적 `as_of` 로 계획 작성

`POST /v1/admin/context/execute`

질문으로 계획을 새로 작성하고 정책 검사 후 실행

`POST /v1/admin/context/query`

execute 와 같은 동작의 deprecated 호환 경로

plan·execute 는 관리자 인증을 사용한다. 요청 예는 다음과 같다.

```
{
  "question": "근로복지공단의 최근 5 년 계약금액을 알려줘",
  "as_of": "2026-10-16"
}
```

`execute` 는 앞서 받은 plan JSON 이나 plan ID 를 입력받지 않는다. 같은 질문도 실행 시점의 참조·데이터 상태에 따라 계획이 달라질 수 있다. 저장된 계획의 승인·재실행 계약은 현재 API 로 제공하지 않는다. 전체 요청·응답과 인증 조건은 배포된 Admin OpenAPI 를 확인한다.

준비할 연결과 실패 확인

	Admin 의 Context repository 는 명시한 Runtime bundle path·store 를 우선하고, 해당 설정이 없으면 Application DB 를 사용한다. 운영 의미·계약을 고정하는 배포는 Runtime bundle [§ 23] 안내를 따른다. 질문 실행에는 Runtime URL·service token 이 필요하다. OpenMetadata 연결은 근거 조회에 사용하며 entity 본문을 Teoria DB 에 복제하지 않는다.

상태

먼저 확인할 내용

`context_engine_unavailable`

bundle 또는 Application DB 연결

`context_runtime_unavailable`

Runtime client 설정·접속

`context_not_resolved`

지원 질문 형식·기간·기관·승인된 의미 경로

`policy_denied`

principal·역할과 정책 결정 이유

`policy_unavailable`

OPA 연결·준비 상태

운영은 OPA 정책 장애 시 요청을 허용하지 않는다. 기간·최대 페이지·Runtime timeout 은 환경 설정 [§ 36]에 설명한다. UI 의 질문 테스트 사용법은 Admin UI [§ 06]를 참고한다.

수정할 코드와 검증

`platform/src/teoria/admin/context_api.py` 는 HTTP, `context/service.py` 는 의미 조회·정책, `context/query.py` 는 handler 계약, `context/runtime_client.py` 는 Runtime HTTP 호출을 소유한다. `context/composition.py` 가 기본 handler 를 조립한다. `domains/public_procurement/context.py` 는 질문 파싱·계획 구성·결과 집계를 담당한다.

Domain 질문을 늘릴 때는 handler 를 확장하고 공통 Engine 에 Domain 별 분기를 추가하지 않는다. 회귀 테스트는 `platform/tests/unit/context/test_context_engine.py` 와 `platform/tests/unit/admin/test_context_api.py` 에 있다. 코드 변경과 전체 검증은 Platform 개발 [§08], 시점별 실행 증거는 Context 검증 기록 [§56]을 참고한다.

08. development.md — Platform 개발 안내

원문 파일: docs/platform/development.md

Platform 은 업무 의미를 정의하고 사용자 요청을 실행한다. OpenMetadata 자산을 조회하고, Ontology 와 연결하며, AI 제안을 검토·적용하고, 게시된 계약으로 Capability 를 실행한다. 데이터를 계속 수집하는 작업은 Pipelines [§29]가 담당한다.

먼저 찾을 위치

	아래 경로는 `platform/` 기준이다.

목적

위치와 책임

실행 명령

`pyproject.toml` 의 scripts, `src/teoria/entrypoints/`

관리 명령

`src/teoria/cli/`: 인자 정의와 기능별 실행 함수

Admin HTTP API

`src/teoria/admin/`: 인증, Router 와 서비스 조립

Ontology 작성·게시

`src/teoria/ontology/`, `src/teoria/authoring/`

Semantic Binding

`src/teoria/binding/`

자산 맥락·OpenMetadata client

`src/teoria/metadata/`

자산 진단·제안 처리

`src/teoria/intelligence/asset_assessment/`, `suggestions/`

Source·Mapping·Capability 실행

`src/teoria/runtime/`

질문 계획·실행 정책

`src/teoria/context/`

OPA 정책 연계

`src/teoria/policy/`

업무별 구현

`src/teoria/domains/`

Registry 로딩·검증·발행

`src/teoria/registry/`

선언형 실행 계약

`registries/`

Application DB migration

`database/migrations/`

관리자 화면

`admin_ui/src/features/`

		실행 진입점

명령

용도

내부 진입점

`teoria`

검증, Registry 발행, bundle 구성, 이관·관리

`entrypoints/cli.py`

`teoria-admin-api`

자산·Ontology·Binding·제안 관리

`entrypoints/admin_api.py`

`teoria-runtime-api`

Capability discovery 와 실행

`entrypoints/runtime_api.py`

`teoria-metadata-assessment-worker`

비동기 자산 진단

`entrypoints/asset_assessment_worker.py`

`teoria-policy-control-plane`

서명 정책 bundle·로그·상태 처리

`entrypoints/policy_control_plane.py`

CLI 는 `cli/parser.py` 에서 인자를 정의하고 `entrypoints/cli.py` 에서 명령을 연결한다. Registry·artifact 는 `cli/registry.py`, Ontology 는 `cli/ontology.py`, Binding 은 `cli/binding.py`, DB migration 은 `cli/database.py`, Source 검증은 `cli/source_verification.py` 가 소유한다. 조달 데이터 진단 명령은 해당 Domain 의 `cli.py` 에 둔다.

Admin 의 `api.py` 는 HTTP 앱을 만들고 `composition.py` 의 `create_admin_services()`가 서비스를 생성한다. `install_admin_routers()`는 구성한 서비스를 Router 에 연결한다. 새 API 는 기능별 Router 에 추가하고 모든 업무 로직을 앱 진입점에 모으지 않는다.

I/O 와 동시 실행

HTTP·DB 호출은 API → Service → Repository 까지 `await` 로 연결한다. PostgreSQL 은 `persistence/postgres.py` 의 `connect()`와 Psycopg `AsyncConnection` 을 사용한다. 연결을 빠져나가면 성공 시 commit, 예외·취소 시 rollback 하고 연결을 닫는다. 한 transaction 의 SQL 실행 순서와 승인·효과 기록 경계를 유지한다.

`TEORIA_DATABASE_CONCURRENCY_LIMIT` 는 event loop 별·접속 URL 별 활성 연결 상한이며 기본값은 10 이다. 프로세스가 여러 개면 각 프로세스의 상한이 합산되므로 DB 전체 연결 예산에 맞춰 설정한다. 연결을 재사용하는 pool 은 아니며 연결을 요청·Task 사이에 공유하지 않는다.

독립 조회만 제한된 범위에서 병렬 실행한다. 정규화·계산·검증은 일반 함수로 유지한다. 동기 SDK 는 제한된 thread 또는 별도 Job 으로 격리하며 `async def` 안에서 직접 실행하지 않는다. CLI 는 프로세스 진입점에서만 `asyncio.run()`으로 비동기 작업을 실행한다. Psycopg 비동기 연결 [참고 1]의 connection·cursor context 와 취소 동작을 따른다.

Domain 기능을 추가하는 방법

`src/teoria/domains/company/identity/`는 기업 식별을, `public_procurement/`는 조달 질문 해석·시장 분석·분류·탐색을 소유한다. `legal_information/`은 수집된 법령을 Admin 진단에서 검색한다.

Domain 의 `processors.py` 에서 Capability handler 와 query 후처리를 등록하고 `runtime/composition.py` 에서 조립한다. Registry 문자열을 임의 Python import 경로로 실행하지 않는다. Context 의 기본 query handler 는 `context/composition.py` 가 선택한다. 공통 Context Engine 은 의미 조회·계획 검증·실행 정책을, Domain handler 는 질문 해석과 결과 집계를 담당한다. 새 기능을 위해 공통 Runner 에 Domain 별 분기를 추가하지 않는다.

기존 `assessment.*`, `market_context.*`, `company_identity.*` processor ID 는 게시된 계약의 식별자다. Python 모듈을 옮긴다는 이유로 바꾸지 않는다. 업무 Domain 과 Runtime 계약 namespace 의 차이는 Registry 안내 [§ 20]를 참고한다.

값 변환 위치도 구분한다. Provider 응답을 정규 DB 행으로 만드는 코드는 Pipelines 의 기능별 `normalization.py`, Source record 를 Runtime 값으로 만드는 codec 은 `src/teoria/runtime/mapping/transforms/`에 둔다.

개발 실행과 검증

저장소 루트에서 의존성과 환경 설정 [§ 36]을 준비한다. Admin API 의 기본 포트는 8001, Runtime API 는 8000 이다. 전체 스택을 사용하려면 시작 안내 [§ 02]를 따른다.

```
uv run --locked --package teoria-platform teoria-admin-api
```

변경 후 단위·통합 테스트와 Registry 검증을 실행한다.

```
uv run --locked --package teoria-platform pytest platform/tests
uv run --locked --package teoria-platform teoria validate platform/registries
```

Application DB 통합 테스트는 운영과 분리된 DB 의 `TEORIA_TEST_APP_DATABASE_URL` 을 사용한다. 설정이 없으면 DB 의존 테스트가 건너뛰어질 수 있다. CI 의 `platform-app-integration` Job 은 migration·동시 승인·게시·제안 적용 복구를 검증한다. Application DB migration 은 새 순번 파일을 추가하고 Compose 설정도 검사한다.

```
uv run --locked --package teoria-platform pytest platform/tests/integration
docker compose -f deploy/compose/compose.yaml config --quiet
```

작업별 상세 안내

의미 모델을 바꾸려면 Ontology 와 Binding [§ 18]을 읽는다.

실행 계약을 바꾸려면 Registry [§ 20], 실행·게시 경로는 Runtime [§ 23]을 읽는다.

질문 계획·정책·실행은 Context [§ 07]를 읽는다.

자산 조회는 자산 맥락 [§ 15], 분석은 자산 진단 [§ 14], 승인·적용·복구는 제안 [§ 17]을 읽는다.

화면 개발은 Admin UI [§ 06], 조달 계산은 시장 분석 [§ 11]을 읽는다.

외부 참고 주소

[1] https://www.psycopg.org/psycopg3/docs/advanced/async.html

09. activity_aggregation.md — 공공조달 활동·참여 집계

원문 파일: docs/platform/domains/public_procurement/activity_aggregation.md

공고·개찰 참여·낙찰·계약을 함께 조회할 때의 사건일, 중복 제거, 공고 묶음과 관측 범위를 설명한다. 대상은 `search_procurement_activity`와 기관·업체 프로필의 활동·참여 지표다. 계약사건 병합·최신 금액·공동수급 귀속은 계약 집계 기준 [§10]을 함께 적용한다.

같은 기간을 조회해도 지표별 사건일은 다르다. 각 지표의 근거 범위와 집계 단위를 먼저 확인하고, 참여 기록이 없는 것을 미참여나 실패로 해석하지 않는다.

`yearly_activity`의 지표별 연도 기준

		`yearly_activity`는 하나의 공통 활동일을 사용하지 않는다. 지표마다 다음 날짜를 독립적으로 적용한다.

지표

연도 기준

중복 제거 단위

`notice_count`

`notice_published_at`

`bid_notice_id`

`participation_count`

한국시간 개찰일; 없으면 최종낙찰일·입찰시각 순서

업체 + 공고번호 + 공고차수 + 분류번호 + 재입찰차수

`award_event_count`

한국시간 `opening_at`, 없으면 `final_award_date`

공고번호 + 공고차수 + 분류번호 + 재입찰차수

`contract_event_count`

최초 계약일

계약사건 키

`attributed_contract_amount`

최초 계약일

계약사건별 업체 귀속금액

`company_count`

최초 계약일

해당 연도 계약업체의 사업자등록번호

공고는 낙찰·계약 행에 반복해서 나타나더라도 게시연도에 한 번만 센다. 기관 프로필은 해당 기관이 기간 안에 게시한 공고 목록을 기준으로 하고, 업체 프로필은 업체 활동과 연결된 공고 중 기간 안에 게시된 공고를 기준으로 한다. 따라서 동일한 필터와 기간에서 다음 불변식을 만족해야 한다.

```
sum(yearly_activity[*].notice_count) == summary.notice_count
```

낙찰일과 개찰일이 모두 없는 낙찰행은 연도별 낙찰 집계에 포함하지 않는다. 계약의 최근 변경일은 여전히 연도별 계약 지표에 사용하지 않는다.

조달 활동 검색과의 일관성

`search_procurement_activity` 도 단계마다 동일한 사건일을 사용한다.

공고 단계: `notice_published_at`

낙찰 단계: `final_award_date`, 없으면 `opening_at`

계약 단계: `first_contract_date`

계약 단계는 공고 생애주기 단위가 아니라 계약사건 단위로 한 레코드를 반환한다. 동일 공고에서 계약이 여러 건 체결되면 각 계약사건을 별도로 반환하고, 변경계약은 계약사건 키로 병합한다. 따라서 기관·기간·업무구분·분야 필터가 같고 추가 검색어 또는 업체 필터가 없으면 다음 불변식을 만족해야 한다.

```
search_procurement_activity(stage="contract").pagination.total_items
== analyze_organization_procurement_profile.summary.contract_event_count
```

계약 레코드의 `latest_activity_date` 는 기간 귀속 기준인 최초 계약일이다. 최신 변경일과 최신 계약금액은 레코드 내부의 계약 상세로 제공하며 기간 귀속을 바꾸지 않는다.

공고 중심 그룹 조회

`search_procurement_activity` 의 기본 `view_mode=flat` 은 기존과 같이 단계별 사건을 반환한다.

`view_mode=notice_grouped` 는 기간·업무구분·분야 필터를 적용한 사건을 다음 키로 그룹화한 뒤 그룹 단위로 상태 집계, 정렬과 페이지네이션을 수행한다.

공고 연결 활동: `bid_notice_id`

공고 미연결 계약: `contract_event_id`, 없으면 `unified_contract_number`

계약 계보와 근거 수준

`search_procurement_activity(view_mode=notice_grouped)` 의 `contracts[]` 는 계약사건 병합과 서로 다른 계약사건 사이의 계보를 구분한다. 같은 `contract_event_id` 에 속한 원계약·변경본은 기존대로 하나의 계약사건으로 병합하고 `contract_version_count` 로 원천 버전 수를 표시한다. 서로 다른 계약사건을 하나의 원계약 계보로 연결하는 판단은 별도의 `contract_family_id` 와 근거 수준으로 표현한다.

`long_term_continuation_type=장기`: `long_term_continuing`

`long_term_continuation_type=계속비`: `installment`

`long_term_continuation_type=신규`: `single`

그 밖의 값 또는 결측: `unknown`

원천에는 명시적인 부모 계약 ID 와 구조화된 차수 번호가 없다. 따라서 장기·계속비 계약에서 같은 공고에 연결되고, 차수 표기를 제거한 계약명이 정확히 같은 계약사건은 하나의 후보 계보로 묶되 `relationship_status=inferred` 로만 반환한다. 이때 계약명에서 명시적으로 확인되는 `N차` 만 `phase_number` 로 추출한다. 제목이나 금액이 비슷하다는 이유만으로 `confirmed` 를 부여하지 않는다. 추론 근거는 `relationship_basis` 에 공고 ID, 원천 장기계속구분, 정규화된 계약명과 차수 표기를 남긴다. 같은 후보 계보에서 동일한 차수 표기가 여러 계약사건에 반복되면 날짜순 첫 사건은 `phase`, 이후 사건은 앞 사건을 부모로 하는 `amendment` 후보로 표시하되 이 관계 역시 `inferred` 다. 동일 `contract_event_id` 안의 변경본 병합과 이 추론 관계를 혼용하지 않는다.

계약상세 URL 에 조달청이 제공한 `ctrtNo` 와 `ctrtChgOrd` 가 있고, 두 값을 결합한 번호가 `confirmed_contract_number` 또는 `contract_reference_number` 와 정확히 일치하면 예외적으로 원천 식별자가 확인된 변경계약 계보로 본다. 같은 `ctrtNo` 를 `contract_family_id` 로 사용하고 변경차수 `00` 은 `original`, 이후 연속 차수는 `amendment` 로 연결한다. 이때만 `relationship_status=confirmed` 를 반환한다. 중간 변경차수가 조회 결과에 없으면 존재하지 않는 부모를 추정하지 않고 부모 ID 를 `null` 로 둔다.

변경계약 가족에서는 가장 큰 변경차수의 레코드만 `is_current_record=true` 이며 이전 레코드는 `superseded_by_contract_event_id` 로 다음 변경본을 가리킨다. 각 변경본의 금액은 누적 합산하지 않는다. 최신 레코드 하나만 `include_in_family_total=true` 로 표시한다.

`total_contract_amount` 는 최신 계약 버전의 원천 `total_amount`, `phase_contract_amount` 는 최신 계약 버전의 원천 `current_contract_amount` 다. 두 값의 0 과 `null` 은 구분한다. 원천 `total_amount=0` 은 총액 0 원의 증거로 사용하지 않고 미제공 값으로 취급한다. 단일계약에서 유효한 `current_contract_amount` 가 있으면 이를 `total_contract_amount` 로 보완하고 `total_contract_amount_basis=current_contract_amount_for_single_contract` 를 함께 제공한다. 현재 계약 API 에는 부가세 포함 여부가 없으므로 `amount_tax_basis` 는 `unknown` 이다. 총계약금액과 금차계약금액은 의미가 다르고, 동일 공고의 원천 계약 기록끼리도 중복될 수 있으므로 화면에서 단순 합산하지 않는다.

계약별 `amount_record_type`, `effective_contract_amount`, `include_in_family_total` 은 금액의 역할과 합산 여부를 나타낸다. 공고 그룹의 `result_summary` 는 가족별 유효 레코드만 사용해 `effective_contract_amount`, `effective_contract_event_id`, `contract_family_count` 와 `amount_aggregation_status` 를 제공한다. 모든 가족이 원천 식별자로 확정되면 `confirmed`, 추론 가족이 포함되면 `partially_confirmed`, 유효 레코드를 하나로 정할 수 없거나 계보가 불완전하면 `unresolved` 다.

장기계속계약의 교차 가족 계보

나라장터가 연도나 차수 전환 시 서로 다른 기초 계약번호를 발급하면 계약 가족 내부의 변경차수만으로 전체 장기계속계약을 표현할 수 없다. 이 경우 계약 가족과 별도로 다음 교차 가족 계보를 제공한다.

`contract_series_id`: 같은 장기계속계약일 가능성이 있는 가족들의 계보 식별자

`previous_contract_family_id`: 직전 계약 가족

`original_contract_family_id`: 후보 계보의 최초 계약 가족

`family_relationship_type`: `continuation` · `independent` · `unknown`

`family_relationship_status`: `confirmed` · `inferred` · `unresolved`

`family_relationship_basis`: 공고 ID, 장기계속 구분, 원천 요청번호, 명시적 차수와 직전 총액 승계 여부 등 판단 근거

공통된 원천 장기계속 식별자가 없으면 같은 공고, 정확히 같은 정규화 계약명, 명시적 차수와 직전 총액 승계가 모두 관찰되더라도 `confirmed` 로 승격하지 않는다. 이러한 관계는 `inferred` 이며 `family_relationship_basis.authoritative_series_identifier` 를 `null` 로 반환한다. 제목 · 업체 · 금액 · 날짜 유사성만으로 확정 관계를 만들지 않는다.

장기계속 계보가 둘 이상의 계약 가족을 포함하면서 관계가 확정되지 않은 경우 다음 안전 불변조건을 적용한다.

```
amount_aggregation_status == "unresolved"
effective_contract_amount == null
effective_contract_event_id == null
```

이때 확인 가능한 계약 가족의 금액을 버리지는 않는다. `latest_confirmed_contract_amount` 와 `latest_confirmed_contract_event_id` 에는 가장 최근 확정 가족의 유효 금액과 이벤트를 제공하고, `included_contract_family_count` 와 `included_contract_event_ids` 에는 확인된 부분집합을 명시한다. 이 값은 전체 계약금액이 아니며 클라이언트가 임의 합산해서는 안 된다. `amount_aggregation_reason` 은 `long_term_continuation_relationship_unresolved` 처럼 전체 금액을 확정할 수 없는 이유를 제공한다.

서로 독립된 계약 가족임이 원천으로 확인된 경우에만 가족 금액을 합산한다. 이때 여러 이벤트가 합산되므로 단일 `effective_contract_event_id` 대신 `included_contract_event_ids` 를 사용한다.

같은 공고에 계약사건이 여러 개 있으면 공고 그룹은 하나지만 `contracts` 에는 각 계약사건을 별도로 유지한다. 같은 계약사건의 변경본은 하나로 병합하고 최초 계약일, 최신 변경 확인일, 조회 종료일 이전 최신 계약금액과 버전 수를 구분한다. 공고명이나 업체명 유사성은 그룹 관계의 근거로 사용하지 않는다.

`stage_counts` 는 그룹의 최신 단계 기준이며 `pagination.total_items` 도 그룹 수를 센다. 전체 상태 조회에서는 다음 불변식을 만족해야 한다.

```
sum(stage_counts[scheduled, open, closed, award, contract, failed, cancelled])
== stage_counts.all
== pagination.total_items
```

참여 분석과 수집 상태

기관·업체 프로필의 `outcome.participation_analysis` 를 참여 실적 표시 계약으로 사용한다. `summary` 와 `yearly_activity` 는 같은 업체·입찰 사건 집합에서 `observed_participation_count`, `result_confirmed_participation_count`, `observed_award_count` 를 집계한다. `observed_success_rate` 는 마지막 두 값의 비율이며 분자·분모도 `success_rate_numerator` 와 `success_rate_denominator` 로 반환한다. 최종 낙찰업체가 비어 있는 결과는 확정된 탈락으로 처리하지 않는다.

`independent_award_event_count` 는 참여 기록 수집 여부와 무관한 별도 낙찰 실적이다. 이를 성공률의 분자로 사용하지 않는다. 기존 `summary·yearly_activity` 의 `award_event_count` 도 이 별도 낙찰 실적이며, 기존 응답 구조의 호환을 유지한다. 참여 카드에는 새 분석의 `observed_award_count` 를 사용해야 한다. 계약금액은 계속 최초 계약 연도로 집계하므로 낙찰금액과 구분한다.

수집 대기·실행·재시도 상태는 `collection_status=collecting` 과 `collection_status_label=수집 중` 으로 제공한다. 이 상태에서 관측된 참여가 없으면 표시용 `participation_count` 는 `null`, 관측 건수는 0, 성공률은 `null` 이다. `complete` 는 알려진 입찰 사건의 **보존 대상** 수집 완료만 의미한다. 실패와 확인할 수 없는 상태는 각각 `failed`·`unknown` 이며 완료로 추정하지 않는다. 상태는 Data DB 의 제한된 조회 view 로 제공하고 Runtime 에는 `ingestion` 스키마 조회 권한을 부여하지 않는다.

현재 보존 범위는 개찰 순위 1~10 위와 원천 응답에 있는 최종 낙찰업체다. `basis.maximum_opening_rank=10`, `retain_available_final_winner_regardless_of_rank=true`, `all_participants_retained=false` 로 범위를 제공한다. 10 위 밖 참여를 알 수 없으므로 `overall_success_rate` 는 항상 `null` 이며 알려진 낙찰 공고의 수집 완료만으로 업체 전체 참여 이력의 완전성을 주장하지 않는다. 도움말과 화면 표시는 입찰체크에서 구현한다.

수정과 검증 위치

활동 조회는 `platform/src/teoria/domains/public_procurement/market_analysis/activity/`, 프로필의 참여 분석은 `profiles/participation_analysis.py`, 관측 참여 조회는 `participation/`에서 찾는다. 각 기능의 테스트는 `platform/tests/unit/domains/public_procurement/market_analysis/`의 같은 기능 아래에 있다.

정책 변경 시 계약 집계와 활동 조회의 사건 수·기간 기준을 함께 검증한다. 수집 완료·부분 수집의 의미는 Pipelines 공공조달 수집 [§31]을 따른다.

10. contract_aggregation.md — 공공조달 계약 집계 기준

원문 파일: docs/platform/domains/public_procurement/contract_aggregation.md

이 문서는 Teoria 의 기관·업체 조달 프로필에서 계약 건수와 계약금액을 산정하는 현재 기준을 정의한다. 화면 문구나 개별 서비스 구현보다 이 문서를 우선하며, 대상 Capability 는 다음과 같다.

`analyze_organization_procurement_profile`

`analyze_company_procurement_profile`

원천 계약 검색 결과를 그대로 세는 규칙이 아니라, 변경계약을 하나의 계약사건으로 병합하고 업체별 귀속금액을 계산하는 분석 규칙이다. 일반 계약 검색 Capability 의 페이지 집계 기준과 혼용하지 않는다.

핵심 원칙

계약 건수는 최초 계약일 기준이며, 금액은 조회 종료일 현재 확인되는 최신 계약금액이다.

최신 계약금액도 원계약의 최초 계약 연도에 귀속한다. 최근 변경일은 최신 버전을 선택하고 최근 계약 관계를 정렬하는 데 사용하지만 연도별 계약 건수나 금액의 귀속 연도를 바꾸지 않는다.

	용어와 식별 기준

용어

정의

계약 버전

원계약과 변경계약 등 원천에 별도 행으로 존재하는 계약 기록

계약사건

같은 계약으로 확인되는 계약 버전을 합친 분석 단위

최초 계약일

조회 종료일 이전 계약 버전의 `concluded_date` 최솟값

최신 계약 버전일

조회 종료일 이전 계약 버전의 `concluded_date` 최댓값

최신 계약금액

조회 종료일 이전 최신 버전의 `current_contract_amount`

업체 귀속 계약금액

최신 계약금액 중 해당 계약업체에 귀속할 수 있는 금액

계약사건 키는 다음 순서로 선택한다.

값이 있는 `confirmed_contract_number`

값이 있는 `contract_reference_number`

`unified_contract_number`

기관 프로필에서는 기관 코드와 계약사건 키를 함께 사용해 버전을 묶는다. 업체별 관계와 금액은 같은 계약사건 안에서도 사업자등록번호 단위로 구분한다. 따라서 한 공고에서 계약이 여러 건 체결되면 공고 수가 아니라 확인된 계약사건 수를 센다.

계약 분야 분류

기관 프로필과 조달 활동 검색의 계약 단계는 동일한 계약 분야 식별 정책을 사용한다. 계약의 원천 분류번호·분류명을 우선 사용하고, 연결된 공고에 같은 분류번호의 대분류· 중분류가 없으면 해당 분류번호가 확인된 최근 공고의 공식 분류 계층으로 보완한다. 분야 필터는 이 보완이 끝난 계약사건에 적용한다. 따라서 동일한 기관·기간·업무구분· 분야 조건에서는 다음 불변식을 만족해야 한다.

```
analyze_organization_procurement_profile.summary.contract_event_count
== search_procurement_activity(stage=contract).pagination.total_items
```

공고의 표시용 분야를 계약사건의 분야보다 우선하지 않는다. 연결 공고가 조회 기간에 포함되어 같은 생애주기 레코드로 합쳐져도, 계약 단계 필터와 결과의 분야 값은 계약에 적용된 공통 분야 식별 결과를 사용한다.

기간 기준

내부 조회는 시작일 이상, 종료일 미만인 반개구간 `[period_from, period_to_exclusive)`을 사용한다. 응답의 `analysis_basis.period_to`는 사용자에게 표시할 포함 종료일이다.

`period_from_year`와 `period_to_year`를 지정하면 해당 달력연도 전체를 대상으로 한다. 종료 연도가 현재 연도이면 오늘까지만 포함한다.

연도를 지정하지 않으면 `period_years`의 가장 오래된 포함 연도 1 월 1 일부터 오늘까지 조회한다.

계약사건의 기간 포함 여부는 최초 계약일로만 판단한다.

공고일은 최초 계약일의 대체값으로 사용하지 않는다.

예를 들어 원계약이 2023 년 2 월 1 일이고 2025 년 6 월 1 일에 변경된 계약은 2023 년 계약사건이다. 2025 년만 조회하면 이 사건은 2025 년 계약 건수로 다시 들어가지 않는다.

계약 버전 병합과 금액 선택

조회 종료일 이전에 확인된 버전만 계약사건 후보로 사용한다. 같은 계약사건에서는 다음 순서로 최신 버전을 선택한다.

`concluded_date` 내림차순 (`NULL`은 마지막)

`updated_at` 내림차순

`unified_contract_number` 내림차순

선택된 최신 버전의 `current_contract_amount`를 현재 계약금액으로 사용한다. 변경계약 때문에 금액이 1 억 원에서 1 억 3 천만 원으로 바뀌었다면 계약 건수는 한 건이고, 1 억 3 천만 원을 최초 계약 연도에 반영한다. 버전 수는 별도 `contract_version_count`로 제공한다.

공동수급과 업체별 금액 귀속

		관계별·분야별·연도별 금액에는 전체 계약금액을 업체마다 반복 합산하지 않고 업체 귀속 계약금액을 사용한다.

조건

업체 귀속 계약금액

완전성

통화가 KRW 이고 지분율이 있음

최신 계약금액 × 지분율 ÷ 100

`complete`

통화가 KRW 이고 계약업체가 1 개

최신 계약금액 전액

`complete`

계약업체가 여러 개이고 지분율이 없음

산정하지 않음 (`null`)

`partial`

계약금액이 없거나 통화가 KRW 가 아님

산정하지 않음 (`null`)

`unknown`

`total_attributed_contract_amount` 는 산정 가능한 업체 귀속 계약금액의 합이다. 금액이 `null` 인 사건을 0 원 계약으로 확정한다는 뜻은 아니다. 응답의 `amount_completeness` 와 `data_completeness.missing_reasons` 를 함께 확인해야 한다.

최초 계약일 누락

최초 계약일이 없는 계약사건은 공고일이나 변경일로 보정하지 않는다.

기간·연도별 계약 건수와 계약금액에서 제외한다.

`summary.missing_first_contract_date_count` 에 고유 계약사건 수를 기록한다.

`data_completeness.missing_reasons` 에 `some_contract_events_missing_first_contract_date` 를 추가한다.

따라서 `missing_first_contract_date_count` 가 0 보다 크면 화면의 계약 합계는 확인 가능한 최초 계약일을 가진 사건에 대한 부분 집계다.

공급업체 진입 목록의 검색과 페이지네이션

`search_organization_supplier_entries` 는 기관·대상연도·진입상태·업무구분·분야 필터를 적용한 계약업체를 사업자등록번호 기준으로 한 번만 집계한다. 업체명 또는 사업자등록번호의 부분 일치 `company_query` 를 적용한 다음 안정적인 보조키와 함께 정렬하고 마지막에 페이지네이션한다. 따라서 `pagination.total_items` 는 검색어까지 적용된 고유 업체 수다.

`supplier_entry` 는 검색 목록과 의미가 다르다. 이 요약은 `company_query` 에 영향받지 않으며 선택한 기관·연도·업무구분·분야 전체의 신규 관측·재진입·기존 업체 수를 유지한다. 검색 결과가 없더라도 요약을 0 으로 바꾸지 않는다.

정렬 기준은 `contract_amount_desc`, `contract_count_desc`, `latest_contract_desc`, `company_name_asc` 다. 기존 호출 호환을 위해 `first_contract_desc` 도 유지한다. 모든 정렬은 사업자등록번호를 최종 보조키로 사용해 페이지 경계에서 누락이나 중복이 생기지 않게 한다.

업체 귀속금액은 계약사건의 최신 금액과 공동수급 지분율을 사용한다. 확인 가능한 귀속금액의 합계가 실제 0 원이면 `0`, 모든 계약사건의 귀속금액을 확인할 수 없으면 `null` 을 반환하며 `amount_completeness` 를 함께 제공한다.

업체별 발주기관 관계 진입 상태

`analyze_company_procurement_profile` 은 `target_year` 에 해당 업체와 계약한 발주기관을 조회 가능한 과거 계약 이력과 비교한다. 대상연도 이전 계약이 없으면 `first_observed`, 직전 3 개 역년 안에 계약이 있으면 `incumbent`, 더 오래된 계약만 확인되면 `reentering` 으로 분류한다. 이는 법적 의미의 최초 거래가 아니라 Teoria 가 조회할 수 있는 이력에서의 최초 관측이다. 업체 방향의 기관 관계 분류는 Capability 가 지원하는 2000 년부터 대상연도 직전까지의 계약 이력을 조회해 2020 년 이전 거래도 판정에 포함한다.

`organization_entry` 에는 상태별 전체 기관 수와 신규 관측 비중을 반환한다. `organization_entry_status` 를 지정하면 전체 프로필 집계와 상태별 요약은 유지하고 `organization_relationships` 에만 상태 필터를 적용한다. 그 다음 `organization_query`, 정렬, 페이지네이션을 적용하므로 `pagination.total_items` 는 검색 조건까지 적용된 전체 고유 기관 수이며 현재 페이지의 행 수가 아니다.

	연도별 집계 예시

원천 상황

집계 결과

2023 년 원계약 1 억 원, 2025 년 변경계약 1 억 3 천만 원

2023 년 1 건·1 억 3 천만 원

같은 공고에서 서로 다른 계약번호로 2 건 체결

계약 2 건

2 억 원 공동수급, A 60%, B 40%

A 1 억 2 천만 원, B 8 천만 원

2 억 원 공동수급, 업체별 지분율 없음

건수는 포함, 업체 귀속금액은 미산정

최초 계약일 없음

기간 집계 제외, 누락 건수에 1 건 추가

API 가 반환하는 기준 정보

클라이언트는 표시 문구를 추측하지 않고 `outcome.analysis_basis` 를 확인한다. 현재 계약은 다음 필드를 반환한다.

```
{
  "contract_event_date_basis": "first_contract_date",
  "contract_amount_basis": "latest_version_at_or_before_period_end",
  "contract_amount_year_attribution": "first_contract_year",
```

```
  "contract_version_deduplication": "merged_by_contract_event",
  "notice_year_basis": "notice_published_at",
  "award_year_basis": "final_award_date_or_opening_at",
  "contract_year_basis": "first_contract_date"
}
```

화면에는 최소한 다음 안내를 계약 추이와 계약금액 근처에 표시한다.

계약 건수는 최초 계약일 기준이며, 금액은 조회 종료일 현재의 최신 계약금액입니다.

`latest_contract_date`는 최근 변경을 포함한 최신 계약 버전일이므로 최근 관계 정렬과 표시에는 사용할 수 있지만 계약 추이의 연도 기준으로 사용하지 않는다.

변경 관리와 검증

이 기준을 변경할 때는 다음 항목을 한 변경에서 함께 갱신한다.

Runtime 조회·집계 구현과 단위 테스트

기관·업체 프로필 Capability 설명과 Registry version

이 문서와 입찰체크 표시 문구

`analysis_basis` 계약 및 실제 Runtime 응답 검증

최소 회귀 사례는 변경계약의 최초 연도 귀속, 조회 종료일 이후 버전 제외, 공동수급 배분, 최초 계약일 누락, 동일 공고의 복수 계약사건이다.

관련 집계

공고·참여·낙찰·계약을 함께 조회할 때는 활동·참여 집계 [§09]를 따른다. 계약사건 식별·최신 버전·금액 귀속은 이 문서가 기준이며, 활동 문서에서도 같은 규칙을 참조한다.

11. market_analysis.md — 공공조달 시장 분석 구현

원문 파일: docs/platform/domains/public_procurement/market_analysis.md

`platform/src/teoria/domains/public_procurement/market_analysis/`는 조달 원천을 읽어 기관·업체·공고 관계를 계산한다. DB 접근과 판정 정책을 섞지 않는 것을 기본 원칙으로 한다.

모듈과 위치

	Capability 등록은 상위 `public_procurement/processors.py`가 기능 모듈을 직접 참조한다. Registry의 `market_context.*` ID와 응답의 provenance 이름은 게시된 계약의 호환성을 위해 유지한다. Python 구현 위치와 Registry ID는 다르다.

책임

위치

기관·기업 프로필 실행·집계·시장 구조

`profiles/`

기관·기업 관계·분야별 근거

`relationships/`

계약 변경·차수·계열·조회 보강

`contracts/`

공고 관계·참여 맥락·벤치마크·업체 선정

`bid_context/`

유사 실적·관련 사업 검색·필터·유사도

`related_projects/`

참여·경쟁사 조회

`participation/`

낙찰·계약 결과·단계별 활동

`outcomes/`, `activity/`

기업 근거·분야 식별·기간·신호 상수

`company_evidence.py`, `field_identity.py`, `periods.py`, `signal_policy.py`

SQL 문장·Data DB 조회 경계

각 기능의 `sql.py`, `reader.py`

프로필·참여 행의 내부 타입 검증

`profiles/models.py`, `participation/models.py`, `row_validation.py`

호출은 기능 모듈을 직접 참조한다. 이전 `market_context/`와 재노출 `processor.py`는 제거했다. 프로필 실행은 `profiles/service.py`가 소유한다. 기능 모듈은 순환 import 하지 않는다. 각 기능의 메모리 cache는 해당 실행 모듈에 두고, 순수 정책은 Registry·DB connection·Runtime cache와 분리한다. 기능 테스트는 `platform/tests/unit/domains/public_procurement/market_analysis/`에 둔다.

변하지 않아야 하는 기준

계약과 업체를 먼저 고유 사건으로 병합한 뒤 필터와 페이지네이션을 적용한다.

신규·재진입 및 반복 판정은 각 계약일 직전 이력만 사용한다.

공동계약의 상태는 업체별로 판정하되 목록 건수는 계약 사건 기준이다.

사용자 표시 사유와 내부 전체 근거를 분리한다.

상대적인 상위 25% 계약은 절대적인 `대형 계약`으로 표현하지 않는다.

판정 임계값과 정렬 우선순위는 응답의 basis에 함께 반환한다.

변경 순서

정책 모듈의 단위 테스트를 먼저 추가한다.

Capability 회귀 테스트로 응답 계약을 확인한다.

Registry 입력 또는 의미 계약이 바뀌면 버전과 release를 갱신한다.

운영 공고로 고유 사건 수, 정렬, 캐시 결과를 검산한다.

Reader의 SQL import는 명시적으로 선언한다. 프로필과 참여 행은 날짜·Decimal 금액·nullable 값을 검증한 뒤 기존 dict 형태와 추가 원천 필드를 유지한다. 검증 오류에는 필드와 오류 유형만 담고 원본 행의 값은 노출하지 않는다.

참여 분석은 `profiles/participation_analysis.py`에서 같은 업체·입찰 차수의 분자·분모를 집계하며 별도 낙찰 실적과 구분한다. API의 `participation_analysis` 계약과 수집 상태·상위 10위 범위는 활동·참여 집계 기준 [§09]에 정의한다.

활동 조회의 `activity/service.py`는 실행을 조립한다. 사건 병합은 `aggregation.py`, 단계·확장은 `lifecycle.py`, 결과 필터는 `filters.py`, 공고 묶음은 `grouping.py`, 묶음 선택·집계·정렬·페이지는 `result_selection.py`, Runtime 응답은 `presentation.py`가 담당한다.

각 기능은 실행과 Reader·SQL·행 타입을 같은 폴더에서 찾을 수 있게 배치한다. 분류 보강은 여러 Reader가 실제 사용하는 `classification.py`·`classification_sql.py`, 공통 행 검증은 `row_validation.py`가 담당한다. 프로필 Reader 등 실제 재사용하는 코드는 소유 기능을 명시해 참조한다. 테스트는 같은 기능 폴더를 따른다.

인접 기능 찾기

`public_procurement/context.py`는 조달 질문의 해석·계획·집계를, `exploration/`은 자동완성·네트워크·분야 시장 탐색을 소유한다. 탐색의 `reader.py`는 Data DB 조회, `filters.py`는 입력 정책, `pagination.py`는 cursor, `cache.py`는 Runtime cache 연결, `presentation.py`는 응답·근거 변환을 담당한다. `discovery.py`, `network.py`, `field_market.py`가 각 기능의 실행을 조립한다.

첨부문서 요건 추출과 참가자격 판정은 보류되어 archive로 이동했다. 현재 Runtime에는 해당 Capability와 processor를 등록하지 않는다.

`classification/`은 조달 분류 정책, `data_profile.py`·`identity_validation.py`는 데이터 품질과 identity 검증, `metadata_operations.py`는 제한된 운영 read view 조회를 소유한다. Domain SQL을 범용 Ontology 패키지에 두지 않는다.

외부 호출 흐름과 노출 범위는 조달 탐색 계약 [§43], 공고·계약·낙찰 원본 수집은 Pipelines 수집 [§31]을 참고한다.

12. notice_status.md — 입찰공고 현재 상태 기준

원문 파일: docs/platform/domains/public_procurement/notice_status.md

입찰공고의 현재 상태는 공고명이나 자유문 텍스트로 추론하지 않고 구조화된 원천과 조달 사건을 기준으로 산정한다.

상태 필드

`notice_status`: 기존 공고 계보 생명주기인 `active`, `cancelled`, `superseded` 를 유지한다.

`current_status`: 화면에서 사용할 현재 조달 상태다. 값은 `scheduled`, `open`, `closed`, `awarded`, `contracted`, `failed`, `cancelled` 다.

`status_source`: 상태 판정에 사용한 원천 Operation 또는 정규 relation 이다.

`status_confirmed_at`: 원천 상태를 확인한 시각이다. 일정으로 산정하는 일반 예정·진행·마감은 원천 확인시각이 없을 수 있다.

호환성을 위해 기존 `notice_status` 의 의미를 변경하지 않는다. 새 화면은 `current_status` 를 우선 사용한다.

우선순위와 근거

`cancelled`: 나라장터 공고종류가 `취소공고`인 경우다. 취소 사유는 원천 `chgNtceRsn`, 취소 시각은 원천 변경시각 또는 취소공고 게시시각을 사용한다.

`contracted`: 해당 공고에 연결된 계약 원천이 존재하는 경우다.

`awarded`: 해당 공고에 연결된 최종낙찰 원천이 존재하는 경우다.

`failed`: 나라장터 낙찰정보서비스의 `개찰결과 유찰 목록 조회`에서 `opengRsltDivNm=유찰`로 확인된 경우다. `nobidRsn` 을 유찰 사유로 사용한다.

`scheduled`, `open`, `closed`: 위 사건이 없을 때 구조화된 게시·마감시각으로 산정한다. 유찰 원천이 확인되지 않은 마감 공고는 `closed` 를 유지한다.

`search_procurement_activity` 의 `stage` 와 `stage_counts` 도 유찰 `failed` 와 취소 `cancelled` 를 별도로 센다. 모든 stage 의 합은 `stage_counts.all` 및 필터 전 그룹 수와 일치해야 한다.

정정·취소 계보

`original_notice_id`: 계보의 최초 공고

`current_notice_id`: 계보의 최신 공고

`revision_number`: 게시순서를 0 부터 센 정정 순번

`is_latest_revision`: 현재 레코드가 계보의 최신 공고인지 여부

계보는 나라장터의 이전 공고번호 원천 필드로 연결하며 제목 유사성은 사용하지 않는다.

13. procurement_classification.md — 공공조달 분야 분류 기준과 확장 계획

원문 파일: docs/platform/domains/public_procurement/procurement_classification.md

이 문서는 Teoria 의 공공조달 분석에서 물품·용역·공사의 분야를 표시하고 필터링하는 현재 기준과, 별도 기능으로 구축할 버전형 분류체계의 범위를 정의한다. 공공조달 고유 정책이며 Platform 공통 분류 추상화로 승격하지 않는다.

현재 제공 범위

		현재 Runtime 은 원천에서 확인되는 공식 분류를 사용하며, 서로 다른 업무구분의 분류를 하나의 임의 계층으로 합치지 않는다.

업무구분

현재 표시 단위

계층 탐색

물품

대표 구매대상물품의 세부품명번호·세부품명

단일 단계

용역

공공조달 분류의 대분류·중분류·분류번호

원천 계층 사용

공사

공공조달 분류번호·분류명

단일 단계

외자·기타

원천에서 확인되는 공공조달 분류

확인된 범위만 제공

물품은 상위 물품분류 마스터를 아직 수집하지 않으므로 10 자리 세부품명번호의 접두어를 잘라 대·중·소분류를 추정하지 않는다. 대표 구매대상물품을 공사 분야와 같은 단일 단계로 제공한다. 따라서 코드와 세부품명이 확인된 물품을 상위 분류가 없다는 이유로 미분류에 포함하지 않는다.

여러 구매대상물품이 있는 공고·계약은 원천 순번이 가장 앞선 품목을 대표 분야로 사용하고 전체 품목 목록은 상세 근거로 유지한다. 대표 분야 기준 집계가 전체 품목별 금액 배분을 의미하지는 않는다.

`work_type` 은 물품·용역·공사를 구분하는 업무구분이며 분야가 아니다. 사업 대상 분야와 참가자격의 면허·업종도 서로 다른 의미이므로 같은 분류 축으로 합치지 않는다.

미분류 의미

현재 미분류는 해당 사건에서 사용할 수 있는 공식 분류번호와 분류명이 모두 없는 경우에만 사용한다. 다음 상태는 장기 분류체계 기능에서 분리해 제공한다.

원천 품목 코드 없음

원천 코드는 있으나 최신 분류로 매핑할 근거 없음

공식 변경 이력으로 최신 분류 매핑 완료

별도 분류체계 기능

향후 기능은 각 업무구분의 공식 분류체계와 변경 이력을 수집하고 과거 조달 데이터를 최신 분류체계로 정규화한다. 단순한 코드·명칭 사전이 아니라 다음 정보를 갖는 기간 유효 분류 레지스트리가 필요하다.

```
procurement_classification
- taxonomy_id
- taxonomy_version
- work_type
- code
- name
- parent_code
- valid_from
- valid_to
- status

procurement_classification_transition
- taxonomy_id
- old_code
- new_code
- transition_type
- effective_date
- source_reference
```

원천 코드와 명칭은 덮어쓰지 않는다. 분석용 최신 분류를 별도로 계산하고 공식 변경 이력, 적용일과 근거를 함께 보존한다. 명칭 변경과 일대일 이동·통합은 최신 코드로 정규화할 수 있지만, 하나의 과거 코드가 여러 최신 코드로 분리된 경우에는 품목식별번호 등 추가 근거 없이 임의 배정하지 않는다.

목표 사용자 기준

장기 기능이 도입되면 분석과 필터의 기본값은 업무구분별 최신 공식 분류체계다. 계약 상세에서는 당시 원천 분류와 최신 정규화 분류를 함께 제공한다.

분야는 업무구분별 최신 공식 분류체계로 재분류하여 집계하며, 매핑 근거가 없는 사건은 별도로 표시합니다.

프로필 분석과 조달 활동 검색은 같은 정규화 결과를 사용해야 한다. 동일한 기간, 업무구분과 분야 필터가 서로 다른 계약 집합을 만들면 안 된다.

구현 경계

공식 분류 마스터·변경 이력의 지속 수집과 적재는 `pipelines/`가 소유한다.

분류체계와 원천 코드의 의미 계약은 공공조달 Domain Registry 가 소유한다.

Runtime 은 Data DB 에 적재된 분류와 매핑을 읽으며 운영 중 Provider 화면을 크롤링하지 않는다.

입찰체크 등 클라이언트는 자체 분류 변환표를 만들지 않고 Runtime 이 반환한 코드, 명칭, 기준 version 과 매핑 상태를 표시한다.

도입 전 완료 조건

업무구분별 권위 분류원과 이용 조건을 확인한다.

현재 마스터와 공식 신설·변경·삭제·분류이동 이력을 수집한다.

원천 분류와 최신 정규화 분류를 함께 보존하는 Data DB migration 을 추가한다.

일대일 이동, 통합, 분할, 코드 없음과 매핑 불가 회귀 사례를 검증한다.

프로필 분석과 조달 활동 검색이 동일한 분류 집합을 사용하는지 검증한다.

14. asset_assessment.md — 자산 진단과 제안 적용

원문 파일: docs/platform/metadata/asset_assessment.md

`intelligence.asset_assessments` 는 사용자가 시작한 자산 진단의 요청·실행·결과를 소유한다. 자산의 상태는 OpenMetadata 에서 읽고 `MetadataTargetRef` 와 관찰 version 만 저장한다. 실행 상태는 `queued → running → completed / partial / failed` 다.

		제공하는 진단

진단

입력

결과

`metadata_enrichment`

테이블·컬럼의 현재 설명·타입

설명 변경 Suggestion

`semantic_alignment`

메타데이터·게시된 Property 후보·기존 Binding

기존 Concept 에 대한 직접 자산 Binding Suggestion

`disclosure_assessment`

메타데이터·평가일·개방 상황·로컬 법령 검색

법령 근거가 붙은 검토 findings

새 Glossary 생성·할당의 자동 추론, 용어를 통한 Binding 확장, 원천 문서 자동 검색, 프로파일·원본 샘플 사용, 신규 Ontology 와 태그 자동 적용은 이 Worker 의 현재 범위에 포함하지 않는다. 기존 Codex Skill 은 설명·Glossary 정비와 용어 경유 의미 연결을 계속 지원한다. Worker 의 직접 Binding 은 보조 연결이며 기본 의미 경로인 `Physical Asset → Glossary Term → Ontology` 보다 우선하지 않는다.

원천 업무 문서를 대조하지 않은 제안은 추정임을 명시하며 confidence 를 0.69 이하로 제한한다. 법령 검색은 기준일의 키워드 검색이며 적용 조건·예외의 검토를 대신하지 않는다. 진단은 법적 적합성을 확정하지 않고 태그 후보도 미검증으로 표시한다. 법령 근거가 없으면 공개 가능 판정 대신 `review_required` 로 남긴다.

API 와 화면

`POST /v1/admin/intelligence/asset-assessments` 는 `metadata_admin` principal 만 호출한다. 요청 body 의 actor 는 허용하지 않는다.

```
{
  "table_fqn": "service.database.schema.table",
  "scopes": ["metadata_enrichment", "semantic_alignment"],
  "column_names": ["amount"],
  "disclosure_context": "public_release"
}
```

`assessment_date` 는 선택 사항이며 기본값은 한국 날짜다. `column_names` 가 비면 전체 컬럼을 분석하며 최대 30 개다. `disclosure_context` 는 `public_release` 또는 `restricted_sharing` 이다. API 는 실제 자산 identity 와 컬럼을 확인하고 202 와 진단 레코드를 반환한다. 같은 대상·version·범위·기준일의 활성 요청은 중복 등록하지 않는다.

`GET /v1/admin/intelligence/asset-assessments?table_id=<uuid>`: 자산의 최근 진단

`GET /v1/admin/intelligence/asset-assessments/<assessment_id>`: 상태·제안 ID·findings

진단 조회는 인증이 필요하다. 자산 상세의 `자산 진단`에서 항목과 컬럼을 고르고 `진단 실행`을 누르면 상태를 주기적으로 조회한다. 실패한 진단은 이력을 유지하며 새 진단을 요청한다. 별도 자동 대상 스캔이나 예약 진단은 없다.

Worker 와 결과 검증

Worker 의 내부 진입점은 `platform/src/teoria/entrypoints/asset_assessment_worker.py` 이며 `intelligence/asset_assessment/`의 실행 서비스를 구성한다. 입력 구성·법령 근거·지시문· 결과 검증·제안 변환·응답 변환은 각각 이름 있는 모듈로 분리한다. Prefect 는 데이터·메타데이터 수집을 계속 담당한다.

PostgreSQL `SKIP LOCKED` 선점, claim token 과 10 분 lease 를 사용한다. 진단 실행은 8 분으로 제한하며 lease 가 만료된 실행은 최대 3 회 선점한다. timeout·모델 오류 등 명시적 실패는 `failed` 로 종료한다. 프로세스 종료로 만료된 작업만 자동 선점한다. 만료된 Worker 는 결과를 저장할 수 없다. 강제 종료 후 재실행은 모델 호출 비용을 중복시킬 수 있지만 제안 저장은 실행권과 트랜잭션으로 보호한다.

모델은 서버가 고정한 gateway 만 호출한다. 메타데이터·법령의 텍스트는 비신뢰 입력으로 취급하며 모델에 임의의 Tool 실행이나 외부 쓰기를 허용하지 않는다. 모델은 변경 후보와 근거 ID 만 반환한다. 서버는 선택 범위·컬럼·게시된 Concept 후보·근거 ID 를 검증하고 제안의 적용 payload 를 직접 구성한다. 분석 전후 자산 version 이 다르면 저장하지 않는다.

검증된 Suggestion, 진단 결과, 연결 ID 를 한 트랜잭션으로 기록한다. 동일한 기존 pending 제안은 재사용한다. 결과 검증에서 일부가 제외되거나 법령 버전이 없으면 `partial` 로 표시한다. 기존 `Suggestion → Review → Application` 을 그대로 사용하며 Worker 는 승인·태깅·게시를 실행하지 않는다. DB 에는 원천 자산 본문, 샘플, 전체 법령 본문, 모델의 원문 응답을 저장하지 않는다.

설정과 실행

```
TEORIA_ASSESSMENT_ENABLED=true
TEORIA_ASSESSMENT_MODEL_URL=<full chat-completions endpoint>
TEORIA_ASSESSMENT_MODEL_NAME=<model identifier>
TEORIA_ASSESSMENT_MODEL_TOKEN=<secret if required>
TEORIA_ASSESSMENT_MODEL_MAX_TOKENS=6000
```

gateway 는 `messages`, `model`, `temperature`, `max_tokens`, `response_format: {"type":"json_object"}` 요청과 `choices[0].message.content` JSON 문자열 응답을 지원해야 한다. 모델 토큰은 Worker 에만 주입하며 기본 외부 endpoint 나 모델을 임의로 선택하지 않는다. 출력 토큰 한도는 모델의 실제 문맥 한도에 맞춘다. 입력 메타데이터·법령과 출력 토큰의 합이 모델의 문맥 한도를 넘지 않도록 설정한다. 작은 문맥 모델은 소수 컬럼부터 진단하고, 대량 컬럼은 입력 분할이나 더 큰 문맥이 필요하다.

Application DB migration 을 적용하고 새 authoring·UI 이미지를 빌드한 뒤 시작한다.

```
docker compose -f deploy/compose/compose.yaml --profile metadata --profile intelligence \
  up -d --build admin-api admin-ui metadata-assessment-worker
```

Compose 의 `app-postgres-migrate` 가 신규 migration 을 적용한다. 모델 설정이 비어 있으면 Admin API 는 `asset_assessment_not_configured` 를 반환하며 Worker 도 시작하지 않는다. 운영 Admin API 는 `bearer` 와 별도 관리자 토큰을 사용한다.

내부 작업과 API 응답 계약

`models.py` 는 Worker 의 `AssessmentJob`·근거 타입과 분석 결과 `AssessmentAnalysis`, 외부 조회용 `AssessmentResponse`·`AssessmentFindingResponse`·`AssessmentPage` 를 구분한다. 모델 후보의 `Finding` 과 서버 검증 후 finding·진단 오류 응답은 별도 계약이다. 생성·목록·단건 조회 API 는 같은 응답 모델을 사용하며 OpenAPI 에 노출한다.

`claim_token`과 `request_key`는 응답 모델에 없고 목록에서도 반환하지 않는다. 응답 변환은 원천 레코드를 변경하지 않으며 기존 nullable 값과 원래 존재한 finding 필드만 유지한다. UI는 요청·상태·finding·근거·조회 응답 타입을 명시한다.

진단 결과 검토

완료된 진단의 근거·신뢰도·대상 version을 확인한다. 설명·Binding 제안은 제안 검토·적용 [§17]의 공통 승인 경로를 따른다. 개방 진단 findings는 검토 결과이며 자동 태깅·공개 승인으로 적용되지 않는다.

구현 위치와 관련 문서

`platform/src/teoria/intelligence/asset_assessment/`에서 `service.py`는 실행, `context_builder.py`는 자산·Concept 후보, `law_evidence.py`는 근거, `prompts.py`는 모델 지시문, `validation.py`는 검증, `proposals.py`는 제안 변환, `presentation.py`는 조회 응답을 소유한다.

제안 생성 이후의 승인·적용·복구 구현은 공통 제안 서비스 [§17]를 참고한다.

자산 맥락과 샘플·프로파일 [§15]

법령 근거 검색 [§16]

Codex 진단 Skills [§45]

2026-10-15 실행 검증 [§47]

15. asset_context.md — 자산 맥락과 샘플·프로파일 조회

원문 파일: docs/platform/metadata/asset_context.md

Teoria Admin API 와 Data Assets 테이블 상세 화면에서 OpenMetadata 가 이미 보관한 샘플·프로파일을 조회한다. 실제 데이터 값과 OpenMetadata entity 상태를 Teoria Application DB 에 저장하지 않는다. 조회 결과를 그대로 Suggestion evidence 나 로그에 기록하지 않는다.

통합 자산 맥락

	데이터 자산의 상세에는 기술·운영·의미·거버넌스 정보를 함께 보여주는 통합 메타데이터 패널이 있다. `GET /v1/admin/metadata/asset-context/{table_id}`는 원천을 조회하며 응답을 Teoria DB 에 저장하지 않는다. 데이터 자산(`/admin/data-catalog`)에서 Table 을 선택하면 전용 상세 페이지를 볼 수 있다. `TEORIA_ADMIN_METADATA_PUBLIC_READ=true` 면 카탈로그·상세·통합 패널·계보·프로파일을 토큰 없이 조회할 수 있다. 기본값은 false 다. 샘플 데이터 조회와 변경 작업은 관리자 인증을 유지한다.

영역

원천과 표시 내용

기술

OpenMetadata 의 스키마·컬럼·설명, 요청 시 물리적 계보

운영

Data DB 의 명시적 Pipeline output 계약, 최근 적재 실행·성공·체크포인트

업무

OpenMetadata Glossary 와 Teoria 의 직접·용어 기반 Ontology Binding 및 상태

거버넌스

OpenMetadata 소유자·태그, 자산별 제안·최근 검토·적용 상태

운영 조회에는 `084_asset_collection_context.sql` 의 제한된 read view 가 필요하다. 공공조달의 서비스·DB·스키마·테이블이 정확히 일치할 때만 연결하며 테이블명 유사성이나 FQN prefix 로 Pipeline 을 추정하지 않는다. 미연결 자산은 미연결로, 미실행 Pipeline 은 실행 기록 없음으로 표시한다. 현재 운영 패널은 적재 audit 이며 Prefect schedule·전체 Flow 상태를 복제하거나 대신하지 않는다. checkpoint 와 최근 성공만으로 전체 과거 데이터의 수집 완전성을 보장하지 않는다.

Binding 은 자산·컬럼의 실제 FQN 과 실제 Glossary term 으로 조회한다. draft·deprecated 도 상태를 표시하며 승인된 연결로 간주하지 않는다. 최근 50 개 Binding·Suggestion 을 표시하고 잘린 결과를 명시한다. Suggestion 원문·근거 excerpt·데이터 샘플은 이 API 에서 반환하지 않는다. 검토·승인·적용은 의미 연결과 제안 검토 화면에서 수행한다.

OpenMetadata 는 기술·자산 메타데이터의 원천이며 해당 조회가 실패하면 오류를 반환한다. Data DB 나 Application DB 장애는 해당 영역만 unavailable 로 표시한다. DB 연결 미설정, 필요 migration 미적용, 명시적 연결 없음은 빈 정상 결과와 구분한다. 자산 전환 후 이전 요청의 늦은 결과를 다른 자산에 표시하지 않으며 응답은 no-store 다.

샘플·프로파일 API

Admin bearer 인증은 기존 `TEORIA_ADMIN_API_TOKEN` 을 사용한다.

```
GET /v1/admin/metadata/tables/{table_id}/sample-data?limit=10
GET /v1/admin/metadata/tables/{table_id}/sample-data?limit=10&include_values=true
GET /v1/admin/metadata/tables/{table_id}/profile
```

샘플은 기본적으로 모든 비 NULL 값을 `[REDACTED]`로 반환한다. 원천 OpenMetadata 의 PII masking 을 해제하지 않는다. `include_values=true` 는 인증된 Admin principal 에 `metadata_sample_reader` 역할이 있을 때만 허용한다. 이 역할은 기본 역할에 넣지 않았으며, 필요한 계정의 `TEORIA_ADMIN_API_ROLES` 에 명시적으로 추가한다. 현재 Admin bearer 는 하나의 token/역할 집합 모델이므로 일반 사용자와 원문 조회자를 같은 token 으로 운영하면 권한이 분리되지 않는다.

행 수는 최대 50, 컬럼은 최대 100, 문자열 값은 최대 500 자로 제한한다. 중첩 객체·배열 값은 표시하지 않는다. `stored_row_count` 는 저장된 샘플의 행 수이며 원천 테이블의 전체 행 수가 아니다. 줄인 결과는 `truncated` 로 표시한다. 응답은 `Cache-Control: no-store` 이며 UI 는 이전 테이블의 값을 다른 테이블에 표시하지 않도록 요청 결과를 구분한다.

프로파일은 최신 table/column profile 의 행·컬럼 수, null·distinct·unique 건수/비율과 길이 지표만 반환한다. min/max, 빈도값, distinct 값 목록과 custom SQL metric 은 원천 값 노출을 막기 위해 응답에 포함하지 않는다. 프로파일도 원천 전체 데이터의 안전성을 보장하는 결과가 아니다.

`available=false, reason=not_collected` 는 샘플 또는 프로파일이 수집되지 않았다는 뜻이다. 수집된 샘플이 0 행이면 `available=true` 이다. OpenMetadata 의 403 은 권한 오류로, 404 는 자산 부재로 반환하며 수집되지 않은 상태로 숨기지 않는다.

수집과 진단 경계

현재 기본 PostgreSQL ingestion 은 `DatabaseMetadata` 이며 스키마 메타데이터만 수집한다. 조회 기능은 운영 DB 에서 원문 샘플 수집이나 프로파일링 Job 을 자동 실행하지 않는다. 실제 샘플이 필요하면 대상 테이블, 샘플 수, 접근 범위가 정해진 별도 OpenMetadata workflow 를 Prefect 가 실행하도록 구성해야 한다. 일반 Pipeline Worker 에 OpenMetadata ingestion 패키지를 설치하지 않는다.

Codex Skill 에서 허용된 샘플·프로파일을 추가 근거로 사용할 수 있다. 비동기 자산 진단 Worker 는 현재 원문 샘플을 모델 입력으로 사용하지 않는다. 샘플에서 미탐지한 것을 전체 테이블에 위험 정보가 없다는 뜻으로 해석하지 않는다. 실제 원문 값을 모델에 전달할 범위는 Agent 실행 정책에서 정한다.

버전별 호환성 검증 결과는 OpenMetadata 업그레이드 검증 [§ 53]을 참고한다.

구현 위치

통합 조회는 `platform/src/teoria/metadata/asset_context.py`, OpenMetadata 연계는 `metadata/openmetadata/`, HTTP 는 `admin/metadata_api.py` 가 소유한다. 공공조달의 운영 조회는 `domains/public_procurement/metadata_operations.py`, output 계약과 read view 는 Pipelines 가 소유한다. 화면은 `platform/admin_ui/src/features/metadata/`에서 찾는다.

스키마 메타데이터를 수집하려면 OpenMetadata Job [§ 32], 조회 결과로 진단하려면 자산 진단 [§ 14]을 참고한다.

16. law_evidence.md — 법령 근거 검색

원문 파일: docs/platform/metadata/law_evidence.md

법령 검색은 Platform **Admin 진단 API** 다. 별도 Ontology, 임베딩 모델, 벡터 DB 와 Neo4j 를 사용하지 않는다. Runtime Capability 나 MCP Tool 로 등록하지 않았으며, 해당 경로를 추가할 때 Database Source · Mapping · Published artifact 계약을 별도로 작성해야 한다. 검색 데이터는 Pipeline migration `083_legal_information_search_views.sql` 의 읽기 전용 view 두 개로 제공한다. Runtime/Admin 공용 읽기 계정에는 view 만 허용하고 원본 테이블 · Raw 에는 권한을 부여하지 않는다.

```
POST /v1/admin/legal-information/search
GET /v1/admin/legal-information/documents/{snapshot_id}?path=...
```

검색 요청 예:

```
{
  "query": "개인정보 민감정보",
  "assessment_date": "2026-10-12",
  "version_scope": "as_of",
  "match": "any",
  "limit": 10
}
```

`law_id` 를 추가하면 법령 범위를 제한한다. 검색어는 최대 8 개 키워드, 200 자이며 `match=any` 는 OR, `match=all` 은 AND 다. SQL wildcard 는 리터럴 문자로 처리한다. 조문 제목 · 본문(하위 항 · 호 · 목 포함) · 법령명을 검색하고 제목/본문 일치를 먼저 정렬한다. 형태소 분석, 동의어 확장이나 의미 검색은 하지 않는다.

`as_of` 는 공포일 · 시행일을 이용해 법령별 후보 버전을 고른다. 기준일 이후 버전과 그 날짜에 아직 시행하지 않은 조항을 제외한다. 최신 후보의 본문이 미수집이면 이전 버전으로 대체하지 않고 `unavailable_versions` 에 표시한다. 폐지된 후보를 제외할 때도 이전 버전을 다시 선택하지 않는다. 부칙 · 개별 적용례와 다른 법령의 예외까지 자동 판단하는 기능은 아니므로 `legal_applicability_confirmed=false` 를 항상 반환한다. `version_scope=all` 은 수집된 연혁 · 현행 · 예정 후보를 함께 검색한다.

결과는 시행 버전, snapshot ID, 원문 hash, parser version, 조항 path 와 1,000 자 발췌를 포함한다. 발췌는 잘림 여부를 표시한다. 공포본의 외부 화면 링크는 `source_publication_url` 이며 분리시행 slice 의 근거는 snapshot · path 로 고정한다. 문서 조회는 같은 snapshot · path 를 사용하므로 이후 원문이 정정되어도 기존 근거를 조회할 수 있다. 기본 20,000 자, 최대 50,000 자를 반환하며 잘림 여부를 표시한다.

구현과 데이터 준비

`platform/src/teoria/domains/legal_information/search.py` 가 Data DB read view 를 조회하고 `admin_api.py` 가 HTTP API 를 제공한다. Admin 서비스 조립은 `admin/composition.py` 가 담당하며 `TEORIA_ADMIN_DATA_DATABASE_URL` 의 read-only 계정을 사용한다.

검색 전 법령 수집 [§30]과 migration 083 을 적용한다. 법령 본문은 Application DB 로 복제하지 않는다. 자산 진단은 snapshot · path · hash 를 근거로 참조한다. 자산 진단 [§14]의 findings 는 법률 적용성 확정이나 자동 공개 승인을 대신하지 않는다.

17. suggestions.md — 변경 제안의 검토·적용과 복구

원문 파일: docs/platform/metadata/suggestions.md

설명 변경, Codex Skill 과 자산 진단은 모두 변경 후보를 제안할 수 있다. 이 문서는 생성된 Suggestion 을 사람이 검토하고 원천에 적용하는 공통 절차를 설명한다. 진단 Worker 의 분석·근거 검증은 자산 진단 [§ 14]이 담당한다.

제안에서 적용까지

```
제안 생성 → pending → 검토 → approved → 대상 재검증·적용 → applied / failed
                          ↳ rejected / changes_requested
```

제안 생성은 `metadata_admin`, 검토·재시도·복구는 `metadata_reviewer`, `metadata_admin`, `ontology_owner` 중 허용 역할이 필요하다. Ontology 변경 검토에는 `ontology_owner` 가 추가로 필요하다. 변경 주체는 인증 principal 이며 요청 body 의 reviewer·actor 를 신뢰하지 않는다.

자산의 현재 값, 제안 값과 제안 당시 근거를 비교한다.

대상·의미·근거가 맞으면 승인하고, 부족하면 거절하거나 변경을 요청한다.

서비스가 원천의 version 과 필요한 참조를 다시 검증한 뒤 적용한다.

결과와 적용 이력을 확인한다. 승인되었다는 사실만으로 적용 성공을 판단하지 않는다.

적용 대상별 결과

		진단 완료는 원천 변경 완료를 뜻하지 않는다. UI 에서 결과의 근거·신뢰도·대상 version 을 검토하고, 설명·Binding 제안은 검토·적용에서 승인한다. 개방 진단 findings 는 검토 보고서이며 자동 태깅이나 공개 승인으로 적용되지 않는다.

제안 대상

승인 후 수행하는 작업

추가 절차

설명·Glossary·quality test

OpenMetadata API 에 적용

대상 version·생성 결과 재검증

Semantic Binding

Binding Draft 생성

별도 Binding 승인

Ontology

새 Draft 에 변경 적용

Ontology 검토·승인·게시

적용 시도는 `intelligence.application_attempts` 에 남긴다. 실행권 token 과 효과 시작·성공 기록으로 중복 적용을 막고 metadata entity 본문은 저장하지 않는다.

실패 후에는 서버가 반환한 다음 조치를 따른다. 변경 시작 전 실패는 기존 승인으로 재적용하고, 변경 시작 후 결과가 불확실하면 기존 결과를 먼저 확인한다. stale evidence 는 재분석한다. 재시도는 새 승인을 만들거나 실행 중인 작업을 중복 호출하지 않는다.

조회와 변경 API

	아래 경로의 공통 prefix 는 `/v1/admin/intelligence` 다. 자세한 요청·응답은 배포된 Admin OpenAPI 를 따른다. 목록·단건·비교·이력 조회에는 관리자 인증이 필요하다.

API

용도

`GET /suggestions?status=pending&table_id=<uuid>`

상태·자산별 목록, `after` cursor 로 다음 페이지

`POST /suggestions`

검증된 제안 입력 제출

`POST /description-suggestions`

현재 설명·version 을 근거로 설명 변경안 생성

`POST /binding-suggestions`

자산 컬럼의 Binding 후보 제안

`GET /suggestions/{id}/target`

현재 대상과 제안 비교

`POST /suggestions/{id}/reviews`

`approve`, `reject`, `request_changes`, `supersede` 검토

`GET /suggestions/{id}/applications/attempts`

시도별 실행자·시각·결과 조회

`POST /suggestions/{id}/applications/retry`

기존 승인으로 안전한 적용 재시도

`POST /suggestions/{id}/applications/reconcile`

외부 변경을 재실행하지 않고 기존 결과 확인

재시도·복구 가능 여부는 서버의 현재 실행권·효과 기록과 응답을 따른다. `stale_evidence` 는 근거 갱신과 재분석이 필요하다. 실행 중인 작업을 중복 호출하지 않는다. 목록과 이력은 페이지 단위이며 조회한 범위를 전체 누적 결과로 해석하지 않는다.

구현 위치

제안 기능은 `platform/src/teoria/intelligence/suggestions/`에 둔다. `generation.py` 는 생성, `application.py` 는 승인·재시도·복구, `execution.py` 는 실행권과 효과 기록, `appliers/openmetadata.py`·`appliers/drafts.py` 는 대상별 쓰기를 담당한다. `contracts.py` 에서 생성한 schema 를 Skill bundle 에도 사용한다.

HTTP 와 서비스 경계

`admin/intelligence_api.py` 는 인증·역할 검사와 HTTP 오류 변환을 담당한다. `intelligence/suggestions/service.py` 는 자산 기준 제안 조회, 원천 대상 비교, 제안 생성·조회·적용 이력과 재시도 조건을 소유한다. OpenMetadata 자산 식별 확인과 대상 참조 구성은 서비스에서 수행한다. API → Service → Repository → PostgreSQL 은 native async 호출을 `await` 로 연결한다. 승인·실행권 선점·효과 시작·성공·적용 결과 기록의 순서를 유지하며 같은 transaction 의 SQL 을 무분별하게 병렬 실행하지 않는다. 연결 상한과 취소 처리는 Platform 개발 안내 [§ 08]를 따른다.

현재 화면은 Admin UI [§ 06], Codex 제출 경로는 진단 Skills [§ 45]를 참고한다.

18. ontology.md — Ontology 와 Semantic Binding

원문 파일: docs/platform/ontology.md

Business Ontology 에는 여러 시스템과 시간에 걸쳐 유지되는 업무 의미만 둔다. API 응답, Capability 출력, 화면 projection 과 평가 결과는 Runtime 계약이지 Business Object 가 아니다.

Teoria 의 실행 의미는 네 경계로 분리한다.

Ontology 는 Object, Property 와 Relationship 으로 업무 세계를 정의한다.

Capability 는 Ontology 를 이용하는 query, compute, decision, action 능력이다.

CapabilityVersion 은 semantic requirement, 입출력, 구현과 정책을 고정한 불변 계약이다.

Runtime Artifact 는 호환성이 검증된 Ontology, Binding, Registry 와 CapabilityVersion 집합을 checksum 으로 고정한다.

CapabilityVersion 은 Ontology revision 자체에 pin 하지 않고 필요한 stable concept 를 선언한다. Artifact compile 은 이를 Published Ontology 의 immutable concept ID 로 해석하며 선언된 필수 요구사항을 해석할 수 없으면 publish 를 차단한다. Lifecycle 과 특정 Artifact 에 대한 compatible/incompatible 판정은 서로 다른 상태로 관리한다.

입찰체크 Ontology 변경은 현재 Published version 에서 새 Draft 를 생성해 수행한다. 업무 의미가 같은 stable concept 는 승계하고, 화면 projection 이나 Capability 응답은 Business Object 로 추가하지 않는다. 검토·승인·게시된 새 version 만 Runtime bundle 에 포함하며 새 의미 계약을 사용하는 배포는 새 bundle 을 사용한다. 이전 bundle 은 복구를 위해 보존한다.

모델 경계

	공공조달 모델은 하나의 shared ontology 안에서 다음 경계를 유지한다.

영역

대표 개념

Identity

`LegalEntity`, `BusinessRegistration`, `Organization`

Procurement lifecycle

`ProcurementCase`, `BidNotice`, `BidOpening`, `Award`, `Contract`

Qualification

`Qualification`, `Industry`, `RegisteredIndustry`, `Sanction`

Observation

Source 별 관찰, 식별 assertion, provenance

동명이거나 이름이 비슷하다는 이유로 객체를 합치지 않는다. Provider 가 제공한 식별자와 검증된 연결을 사용하고, 서로 다른 원천의 충돌은 값을 덮어쓰기보다 원 관찰과 authority policy 를 보존한다. 계약 snapshot 을 계약 사건이나 version history 로 오해하지 않는다.

Semantic path

기본 연결은 다음 순서다.

```
Physical Asset → OpenMetadata Glossary Term → Stable Ontology Concept
Capability ──────────────────────────────────→ Stable Ontology Concept
API Field ───────────────────────────────────→ Stable Ontology Concept
```

Data Asset 직접 Binding 도 가능하지만 Glossary Term Binding 보다 우선하지 않는다. API Field Binding 은 활성 Source Registry 에서 Operation 과 원본 필드가 검증된 경우만 승인한다.

식별자와 version

Stable Concept ID 는 version 간 의미 식별자다.

revision ID 는 특정 Ontology version 의 편집 단위다.

Published version 은 불변이며 변경은 그 version 에서 새 Draft 를 생성한다.

lifecycle 은 `draft → in_review → approved → published → deprecated` 다.

publish 전에 구조 검증, Binding 호환성 검사와 deterministic artifact compile 을 통과해야 한다.

Binding 은 `draft → approved/rejected → deprecated` 상태와 불변 review 기록을 갖는다.

Authoring 과 Runtime

Authoring DB 모델을 Runtime contract 로 직접 사용하지 않는다. Publish 가 생성한 artifact 는 stable concept, property 와 relationship 을 포함하며 checksum 으로 식별한다. Runtime Registry 의 Object/Link 는 Capability 입출력 materialization 계약으로 계속 독립적으로 존재한다.

개발 환경은 빠른 authoring 검증을 위해 원본 Registry 를 직접 읽을 수 있다. 운영 Runtime bundle 은 Registry release, Published Ontology artifact 와 compile 시점의 Approved Binding snapshot 을 함께 고정한다. Loader 는 version 디렉터리의 `manifest.json`, `bundle.json` 과 각 구성요소 checksum 을 대조한 뒤에만 bundle 을 적재한다. 원본 Registry 에는 release 파일을 쓰지 않으며 Artifact Store 의 `manifest.json` 이 Registry release provenance 의 단일 권위다. Artifact store 의 `active.json` 교체는 atomic rename 으로 수행하며 rollback 은 검증된 이전 version 을 다시 activate 하는 방식으로 처리한다.

Ontology 변경과 이관

Published version 에서 새 Draft 를 만들고 필요한 개념·속성·관계를 수정한다. 검토·승인·게시 후 Runtime bundle [§ 23] 을 구성해 배포한다. Stable Concept ID 를 의미 식별자로 유지하고 version 별 revision ID 를 Binding 의 의미 식별자로 사용하지 않는다.

구형 YAML Object 를 추가하거나 제거하는 이관에는 `platform/ontology_migrations/pending/<migration>/ontology_v2.yaml` 분류를 함께 작성한다. 이관 묶음 이름은 `YYYY_MM_<purpose>`를 사용한다. manifest 를 명시해 보고서를 실행한다.

```
uv run --locked --package teoria-platform teoria ontology-migration-report \
  --manifest platform/ontology_migrations/pending/2026_10_runtime_contract_classification/
ontology_v2.yaml
uv run --locked --package teoria-platform pytest platform/tests
```

위 경로는 현재 보관된 분류 manifest 의 예다. 새 변경에는 그 변경의 manifest 를 지정한다. 적용 후 대상 Ontology version 과 artifact checksum 을 기록하고 묶음을 `applied/`로 옮긴다. `applied/`는 이미 검토·게시된 입력과 결정 기록이므로 수정하거나 현재 구성처럼 재실행하지 않는다. 현재 권위는 Application DB 에서 게시한 Ontology 와 immutable artifact 다.

초기 통합은 `applied/2026_10_initial_unification/`, 공고 문맥과 Stable Key 교정은 `applied/2026_10_bid_notice_context/`, `applied/2026_10_bid_notice_stable_key_correction/`에 보존되어 있다. 시점별 수치와 중간 입력은 이력으로 읽고 현재 계약으로 복제하지 않는다.

Ontology 작성 코드는 `platform/src/teoria/ontology/`, Binding 은 `binding/`, 관리 API 는 `admin/ontology_authoring_api.py` 와 `admin/binding_api.py` 에서 찾는다.

19. capabilities.md — Capability 실행 계약

원문 파일: docs/platform/registry/capabilities.md

Capability 는 검증할 입력, 실행 단계와 반환할 Runtime Object·Link 타입을 선언한다. 업무 의미는 Published Ontology 의 Concept 요구사항과 Semantic Binding 으로 연결한다. MCP 에 공개할 때 Tool 이름은 Capability ID 와 같다.

아래는 구조를 설명하기 위해 일부 필드만 보인 예다.

```yaml
capability:
  id: get_company_profile
  inputs:
    corporate_registration_number:
      property: company.legal_entity.corporate_registration_number
      required: true
  steps:
    - call: fsc_company_basic.get_company_overview
  returns:
    - company.legal_entity
```

`property`: `<runtime_contract>.<object_type>.<property>`

`call`: `<source>.<operation>`

`returns`: `<runtime_contract>.<object_type|link_type>`

실행 순서는 `steps` 순서와 같다.

입력→요청은 Mapping 의 request binding 과 `encode`, 응답→객체는 response binding 과 `decode` 를 사용한다. Capability 전용 입력은 `data_type + field` 로 직접 연결할 수 있다.

Runtime 은 timeout, retry, deadline 과 최대 페이지를 적용하고 결과를 identity 로 병합한 Object·Link 와 provenance 로 반환한다. 원본 응답은 명시적으로 요청한 진단 상황에서만 포함한다.

실행 종류와 외부 노출

`kind` 는 `query`, `compute`, `action` 으로 실행 성격을 구분한다. `effects.reads`·`effects.produces` 는 조회 대상·비영속 계산 결과, `effects.creates`·`effects.updates` 는 action 의 업무 상태 변경을 선언한다. 기존 조회 Capability 의 기본 종류는 `query` 다.

`exposure: public` 인 Capability 만 Runtime discovery 와 MCP Tool 에 노출한다. 조합 실행용 지원 계약은 `internal` 로 유지한다. 입력 바인딩은 Ontology 와 Runtime 의 타입·collection 호환성을 검증한다. Domain 구현과 등록은 Platform 개발 [§ 08], 발행과 실행은 Runtime [§ 23]을 따른다.

20. guide.md — Registry guide

원문 파일: docs/platform/registry/guide.md

Registry 는 Platform Runtime 이 검증하고 실행하는 선언형 계약이다. Business Ontology authoring 과 Pipeline orchestration 을 담지 않는다.

	구성

Registry

역할

Core

Data Type 과 Value Set

Domain

Runtime Object, Link, Mapping, Capability

Source

Runtime 이 직접 호출하는 API 또는 Database relation

ID 는 `snake_case`, 참조는 namespace 를 포함한 완전한 ID 를 사용한다. 외부 Provider 의 wire field 와 code 는 원문 표기를 유지한다. 변경 가능한 상태를 모두 별도 Object 로 만들지 않고, 업무상 독립된 이력일 때만 observation/event 객체로 모델링한다.

Source 와 Connector 선택

Capability 가 요청 시 직접 호출하는 API 는 `platform/registries/sources/`에 둔다.

`capability.exposure: public` 인 계약만 Runtime discovery 와 MCP Tool 에 노출한다. 다른 Capability 가 조합 실행에 사용하는 지원 계약은 `internal` 로 유지한다.

Prefect 만 지속 수집하는 API 는 `pipelines/connectors/`에 둔다.

수집 DB 를 Runtime 이 읽으면 Database Source 와 DB-to-Ontology Mapping 을 만든다.

같은 API 를 Source 와 Connector 양쪽에 등록하지 않는다.

Source 의 세부 작성 절차와 verification case 규칙은 Source authoring [§22]을 따른다.

Runtime 계약

`domains/<domain>/`은 업무 소유권을 나타내며 독립 배포 경계가 아니다. Runtime 계약 namespace 는 YAML 의 `runtime_contract.id` 로 식별한다. 하나의 업무 Domain 이 여러 Runtime 계약을 소유할 수 있다.

아래 트리는 `platform/registries/` 기준이다.

```
domains/
├── company/
│   ├── runtime_contracts/company.yaml
│   ├── capabilities/
│   └── mappings/
```

```
└── public_procurement/
    ├── runtime_contracts/public_procurement.yaml
    ├── capabilities/
    └── mappings/
```

파일명은 계약 ID 와 일치해야 하며 소유 Domain 폴더명과 계약 ID 를 같게 만들 필요는 없다. 로더는 이전에 발행한 `domains/<namespace>/runtime_contract.yaml` 구조도 읽지만 새 계약은 소유 Domain 의 `runtime_contracts/<contract_id>.yaml` 에 작성한다. 중복 계약 ID 는 위치와 무관하게 거부한다.

Object Type 은 identity property 와 업무 property 를, Link Type 은 source 와 target 을, Mapping 은 Source record 변환을, Capability 는 검증된 입력·출력과 실행 조합을 정의한다.

Registry checksum 에는 파일 경로도 포함되므로 위치만 바꿔도 checksum 이 달라진다. 새 구조를 운영에 사용하려면 새 Registry release 와 검증된 Runtime bundle 을 만든다. 기존 Published release·artifact 경로와 내용은 직접 수정하지 않는다.

Breaking change 는 새 Registry version 으로 발행한다. Runtime 은 검증된 immutable bundle 과 checksum 을 사용하며 Git 작업 파일을 운영 중에 임의로 읽어 바꾸지 않는다.

원본 `platform/registries/`는 사람이 작성하는 draft 계약만 보관하며 release 상태나 `.release.json` 을 저장하지 않는다. 발행 명령은 커밋된 원본을 변경하지 않고 지정한 Artifact Store 에 version 디렉터리와 `manifest.json` 을 생성한다.

```
uv run --locked --package teoria-platform teoria publish \
  platform/registries \
  --version YYYY.MM.DD.REVISION \
  --output /path/to/registry_artifacts \
  --git-commit "$(git rev-parse HEAD)"
```

따라서 Registry 변경은 한 번만 커밋한다. 생성된 manifest 와 Runtime bundle 은 Git 이 아니라 Artifact Store 가 보관하고, 운영 전환은 검증된 bundle 의 `active.json` 을 원자적으로 교체한다.

검증

```
uv run --locked --package teoria-platform pytest platform/tests
uv run --locked --package teoria-platform teoria validate platform/registries

uv run --locked --package teoria-pipelines pytest pipelines/tests
uv run --locked --package teoria-pipelines teoria-pipelines validate pipelines
uv run --locked --package teoria-pipelines --group validation \
  teoria-pipelines validate-integration pipelines \
  --platform-registries platform/registries
```

검증기는 schema, 참조, Mapping 타입, Capability graph, Source request/response 와 Database Source relation 을 확인한다. Provider 계약 변경은 정적 검증 외에 모든 Operation 의 Build 검증을 수행하고 credential 이 있으면 Live 검증도 수행한다.

세부 작성 안내

Capability [§ 19]: 입력, 실행 단계와 결과 계약

Mapping [§ 21]: Source 필드 변환과 객체 materialization

Source 작성과 검증 [§ 22]: 원문 대조와 Operation 별 Build·Live 입력

21. mappings.md — Mapping 값 변환과 객체 생성

원문 파일: docs/platform/registry/mappings.md

Mapping 은 Source record 의 필드를 Runtime Object 의 Property 로 변환한다. 필드·타입·identity 는 Runtime 계약을 따르고, 지속 가능한 업무 의미는 Published Ontology 가 정의한다. YAML 의 `ontology: company` 는 이 실행 계약의 namespace 를 참조하는 기존 필드명이며 Business Ontology 작성이나 Semantic Binding 을 대신하지 않는다.

아래는 구조를 설명하기 위해 일부 필드만 보인 예다.

```yaml
mapping:
  id: company
  ontology: company
  bindings:
    legal_entity.legal_name:
      - field: fsc_company_basic.get_company_overview.response.corpNm
```

필드 참조:

API: `<source>.<operation>.request|response.<path>`

Database: `<source>.<relation>.<field>`

Target: `<object_type>.<property>`

한 Property 아래 여러 항목은 독립 binding 이다. 여러 필드를 함께 변환할 때만 이름 있는 객체로 묶는다.

```yaml
postal_address.full_address:
  - field:
      base: source.operation.response.baseAddress
      detail: source.operation.response.detailAddress
    decode: company.combine_korean_address
```

`decode`: Source 표현을 Runtime Property 값으로 변환

`encode`: Runtime 입력을 Source 요청 값으로 변환

`role`: 같은 레코드의 동일 Object Type 역할 구분

Codec 은 `platform/src/teoria/runtime/mapping/transforms/`의 실제 함수이며 Registry 에서는 `company.function_name` 처럼 짧게 참조한다.

`materializations` 는 binding 결과로 객체 identity 와 Link endpoint 를 만든다. 내부 ID, 고정값, 수집시각과 링크 생성 규칙은 binding 에 넣지 않는다.

다른 연결과의 구분

이 문서는 Source record 를 Runtime object 로 변환하는 실행 계약이다. Ontology Concept 와 Glossary·자산·API·Capability 를 연결하는 Semantic Binding [§18]과 구분한다. API 응답을 정규 DB 로 적재하는 변환은 Pipelines 기능별 `normalization.py` 가 소유한다.

22. source_authoring.md — Source Registry 작성과 검증

원문 파일: docs/platform/registry/source_authoring.md

외부 API 문서로부터 Runtime 용 Source Registry 를 만드는 절차다. Prefect 전용 API 는 Connector [§26]로 작성한다.

Database Source 는 DB Source 템플릿 [§24]을 참고한다. Pipeline 이 적재한 relation 을 read-only 로 조회하고 DB→Runtime 값은 Mapping 으로 변환한다.

산출물

```
platform/registries/sources/{source_id}.yaml
platform/references/providers/{provider}/{source}/metadata.yaml
platform/references/providers/{provider}/{source}/{document}
platform/registries/sources/verification_cases/{source_id}/{operation_id}.yaml
```

Reference 만 등록한 단계는 `metadata.yaml` 에 `status: draft`, Source 까지 완성되면 `active` 를 사용한다.

작성 절차

원본에서 제공기관, 버전, base URL, 인증, Operation, 요청·응답 필드, pagination 과 오류를 확인한다.

`{provider}_{service}` 형식의 안정적인 `snake_case` ID 를 정한다.

원본 문서와 `metadata.yaml` 을 Reference 경로에 둔다.

Source 템플릿 [§25]으로 Source 를 작성한다.

실제 secret 대신 `credential_env` 를 정의한다.

반복 객체는 Component 로 만들고 Operation 에서 `ref` 한다.

모든 Operation 에 공개 가능한 verification case 를 만든다.

정적·Build·Live 순서로 검증하고 계약 오류를 수정한 뒤 전체를 재실행한다.

추측으로 필드, 타입, 필수 여부나 코드값을 만들지 않는다. 문서와 실제 응답이 다르면 둘을 구분하고 Live 검증 근거를 남긴다.

Reference metadata

```yaml
status: draft
provider: 제공기관
source: provider_service
title: API 참고자료
retrieved_at: "2026-07-31"
official_url: null
files:
  - path: api-document.docx
    format: docx
registry: registries/sources/provider_service.yaml
```

`registry` 는 `platform/` 기준 상대 경로다. Reference 에는 API 키, 개인정보와 비공개 응답을 저장하지 않는다.

원문 경로는 `platform/references/providers/{provider}/{source}/`다. metadata 에는 `target: source`, Source ID, 수집일과 원문 파일을 기록한다. 허용 형식은 `pdf`, `docx`, `xlsx`, `csv`, `json`, `xml`, `yaml`, `html`, `markdown`, `text` 다. 원본은 수정하지 않고 변환본이 필요하면 별도 파일로 함께 등록한다. `specification.source_document` 는 등록된 원본 파일명과 일치해야 한다.

핵심 규칙

Source ID 와 파일명은 같아야 한다.

Teoria 가 정하는 Source·Object·Operation ID 는 `snake_case` 다.

원본 필드와 파라미터 ID 는 대소문자까지 보존한다.

`specification.source_document` 는 Reference 파일명과 일치해야 한다.

필드는 `data_type`, `ref`, `type` 중 하나만 사용한다.

요청 필수값은 query/header/body 컨테이너의 `required` 에 둔다.

`GET`, `HEAD` 에는 body 를 선언하지 않는다.

응답 `data` 는 `ref` 또는 인라인 `fields` 중 하나만 사용한다.

조회를 안전하게 반복할 수 있을 때만 `idempotent: true` 를 둔다.

공통 Data Type 이 없으면 재사용 가능성을 확인한 뒤 Core Registry 에 추가한다.

페이지 응답 예:

```yaml
pagination:
  type: page_number
  page: {request: query.pageNo}
  page_size: {request: query.numOfRows}
  total_count: response.body.totalCount
response:
  data:
    record_path: response.body.items[]
    ref: record
```

Verification case 는 `platform/registries/sources/verification_cases/{source_id}/{operation_id}.yaml` 에 둔다. 내용은 Operation 별 요청 section 만 포함한다.

```yaml
query:
  pageNo: 1
  numOfRows: 10
```

검증

계약 정적 검증 → 요청 Build → 실제 응답 Live 순서로 실행한다. 모든 Operation 을 검증하고, credential 이 있으면 안전한 입력으로 Live 까지 실행한다.

```
uv run --locked --package teoria-platform teoria validate platform/registries

uv run --locked --package teoria-platform teoria verify source \
  --profile build \
  --source {source_id} \
  --operation {operation_id} \
  --input platform/registries/sources/verification_cases/{source_id}/{operation_id}.yaml
```

credential 을 주입한 실제 검증은 `--profile live` 를 사용한다. 원본 전체 응답 대신 필요한 필드 단위의 안전한 fixture 만 남긴다.

완료 조건

모든 ID, path, method, field, type 과 pagination 이 문서와 일치한다.

Reference 와 Source 가 서로 연결된다.

모든 Operation 에 verification case 가 있다.

Static · Build 가 통과하고 가능한 Operation 은 Live 도 통과한다.

API 키, 개인정보와 비공개 원본 응답이 커밋되지 않는다.

23. runtime.md — Runtime 실행과 artifact 배포

원문 파일: docs/platform/runtime.md

Runtime 은 Source record 를 Mapping 으로 Runtime Object·Link 에 변환하고 Capability 를 실행한다. MCP 와 외부 서비스는 이 HTTP API 를 사용한다. Ontology 편집·승인과 지속 수집은 각각 Admin 과 Pipelines 가 담당한다.

요청이 처리되는 순서

	HTTP 요청 → Bearer 인증·OPA 정책 → 입력 검증 → Capability Runner → Source API 또는 Data DB 읽기 → Mapping → Object·Link 와 provenance

API

용도

`GET /health`

프로세스 상태

`GET /v1/version`

적재한 Registry·Runtime version 확인

`GET /v1/capabilities`

공개 Capability 와 입력 계약 발견

`POST /v1/capabilities/{capability_id}:execute`

검증된 입력으로 실행

실행 결과에는 식별자로 병합한 Object·Link 와 원천 근거가 포함된다. Runner 는 Source timeout, 재시도, 최대 페이지와 Capability deadline 을 적용한다. 입력 계약과 실행 종류는 Capability [§19], 값 변환은 Mapping [§21]을 참고한다. API 의 전체 요청·응답 schema 는 실행 중인 `/docs` 가 기준이다.

Registry release 와 Runtime bundle

`platform/registries/`는 사람이 작성하는 계약 원본이다. Registry 발행은 그 내용을 검증해 version 별 release 로 고정한다. Runtime bundle 구성은 이 release 와 Application DB 의 Published Ontology, Approved Binding snapshot, CapabilityVersion 을 함께 고정한다. 운영 Runtime 은 checksum 검증을 통과한 bundle 만 읽는다.

게시된 Ontology 와 승인된 Binding, 접근 가능한 Application DB 를 먼저 준비한다. 아래 version 은 예시이며 새 배포마다 고유한 version 을 사용한다.

```
uv run --locked --package teoria-platform teoria validate platform/registries
uv run --locked --package teoria-platform teoria publish platform/registries \
  --version 2026.10.16.1 --output dist/registry
uv run --locked --package teoria-platform teoria compile-runtime-bundle \
  dist/registry/2026.10.16.1 --output dist/runtime \
  --version 2026.10.16.1
uv run --locked --package teoria-platform teoria activate-artifact dist/runtime \
  --version 2026.10.16.1
```

`compile-runtime-bundle` 은 `TEORIA_APP_DATABASE_URL` 을 사용한다. 기본 Ontology namespace 는 `teoria` 이며 `--ontology-namespace` 를 반복해 지정할 수 있다. 필요한 stable concept 를 해석할 수 없거나 호환성이 맞지 않으면 구성이 실패한다.

`manifest.json`, `bundle.json` 과 구성요소 checksum 이 artifact 의 무결성을 확인한다. Registry checksum 에는 파일 경로도 포함되므로 계약 파일 위치 변경도 새 release 가 필요하다. 원본 Registry 에 release 상태나 `.release.json` 을 추가하지 않고 생성물은 Artifact Store 에서 보관한다. 이미 발행한 version 의 내용과 경로는 직접 수정하지 않는다.

실행과 version 전환

```
export TEORIA_RUNTIME_ARTIFACT_STORE=dist/runtime
export TEORIA_ENVIRONMENT=production
uv run --locked --package teoria-platform teoria-runtime-api
```

인증·DB·OPA 연결은 환경 설정 [§ 36]을 먼저 준비한다. 운영은 별도 Runtime bearer token 과 `TEORIA_POLICY_MODE=opa` 를 사용한다. 정책 서비스 장애나 undefined decision 은 요청을 허용하지 않는다.

bundle 선택은 `TEORIA_RUNTIME_ARTIFACT_PATH` 로 version 디렉터리를 직접 지정하거나, `TEORIA_RUNTIME_ARTIFACT_STORE` 의 `active.json` 을 사용한다. 운영에서 둘 다 없으면 시작하지 않는다. 개발 환경에서만 원본 Registry 를 직접 읽는 경로가 허용된다.

활성화 명령은 checksum 을 검증한 뒤 active pointer 를 원자적으로 교체한다. 실행 중인 프로세스는 시작할 때 bundle 을 적재하므로, pointer 변경 뒤 Runtime 과 해당 bundle 을 사용하는 Admin 프로세스를 재시작·재배포하고 `/v1/version` 을 확인한다. 되돌릴 때는 보존한 이전 version 을 다시 활성화하고 같은 순서로 반영한다.

구현과 검증 위치

HTTP 는 `platform/src/teoria/runtime/api.py`, 실행 조립은 `runtime/composition.py`, Runner 는 `runtime/capability/runner.py`, Source 실행은 `runtime/source/`, 값 변환은 `runtime/mapping/transforms/`가 소유한다. Domain handler 는 공통 Runner 에 등록한다.

artifact 검증·구성은 `registry/artifact.py` 와 관련 모듈, App DB snapshot 조회는 `registry/artifact_snapshot.py`, 명령은 `cli/registry.py` 에서 찾는다. Runtime cache 는 `runtime/cache/`가 소유한다. Redis 장애는 실행을 막지 않고 캐시 없이 동작한다. 정책 bundle 빌드·서명·서비스 시작 순서는 Compose [§ 35]를 따른다.

Source DB 실행과 시장 분석 reader 는 native async PostgreSQL 을 사용한다. HTTP 부터 Repository 까지 `await` 를 연결하고 분류 계층 보강 같은 내부 SQL helper 도 같은 연결·transaction 안에서 완료를 기다린다. 동시 실행·취소의 기준은 Platform 개발 안내 [§ 08]를 따른다. 실제 기관 프로필·활동 검색과 분류 보강 경로의 배포 검증은 2026-10-16 검증 기록 [§ 48]에 있다.

24. database_source.yaml — 계약 참조·템플릿

원문 파일: docs/platform/registry/templates/database_source.yaml

다음은 원문 YAML 파일의 전체 내용이다. 예제·템플릿은 실행 중인 Registry 계약과 구분한다.

```yaml
registry:
  version: 1.0.0
  registered_at: "YYYY-MM-DD"

source:
  id: domain_data_store
  name: 도메인 데이터 저장소
  description: Runtime이 직접 조회하는 정규화된 PostgreSQL 데이터 계약
  type: database
  access:
    engine: postgresql
    connection_env: TEORIA_RUNTIME_DATA_DATABASE_URL
  relations:
    - id: records
      name: 레코드
      relation: domain.records
      primary_key: [record_id]
      fields:
        - id: record_id
          name: 레코드 식별자
          data_type: string
```

25. source.yaml — 계약 참조·템플릿

원문 파일: docs/platform/registry/templates/source.yaml

다음은 원문 YAML 파일의 전체 내용이다. 예제·템플릿은 실행 중인 Registry 계약과 구분한다.

```yaml
# Copy this file to platform/registries/sources/{source_id}.yaml and replace every
# provider-specific example value. Files below docs/ are not loaded as registries.
registry:
  version: 1.0.0
  registered_at: "2026-07-31"

source:
  id: provider_service
  name: 제공기관 서비스명
  provider:
    organization: 제공기관
    distribution: 배포기관
  type: api
  specification:
    format: manual
    version: "1.0"
    api_version: "v1"
    source_document: 제공기관_서비스문서.docx
  access:
    base_url: https://api.example.go.kr/service/v1
    authentication:
      type: api_key
      in: query
      name: serviceKey
      credential_env: TEORIA_SOURCE_PROVIDER_SERVICE_API_KEY
  components:
    objects:
      - id: source_record
        name: 원천 레코드
        fields:
          - id: recordId
            name: 레코드 식별자
            data_type: string
          - id: statusCode
            name: 상태 코드
            data_type: string
            values:
              - value: "01"
                label: 정상
  operations:
    - id: list_records
      name: 레코드 목록 조회
      method: GET
      idempotent: true
      path: /records
      description: 검색 조건에 해당하는 레코드 목록을 조회한다.
      pagination:
        type: page_number
        page:
          request: query.pageNo
        page_size:
          request: query.numOfRows
        total_count: response.body.totalCount
      request:
        query:
          fields:
            - id: pageNo
              name: 페이지 번호
              data_type: integer
              default: 1
            - id: numOfRows
              name: 페이지당 결과 수
              data_type: integer
              default: 10
            - id: searchValue
              name: 검색값
```

```
            data_type: string
        required:
          - searchValue
    response:
      content_type: application/json
      http_status: 200
      control:
        record_path: response.header
        success:
          field: resultCode
          equals: "00"
        fields:
          - id: resultCode
            name: 결과 코드
            data_type: string
          - id: resultMessage
            name: 결과 메시지
            data_type: string
      data:
        record_path: response.body.items[]
        ref: source_record
    errors:
      - http_status: 400
        source_code: BAD_REQUEST
        message: 잘못된 요청
      - http_status: 500
        source_code: INTERNAL_ERROR
        message: 제공기관 내부 오류
```

26. connectors.md — Connector와 수집 계약 작성

원문 파일: docs/pipelines/connectors.md

Connector는 Prefect Worker 전용 외부 API 계약이다. Runtime이 직접 호출하지 않으므로 Source Registry에 중복 등록하지 않는다.

아래 경로는 `pipelines/` 기준이다.

```
connectors/{connector_id}.yaml
definitions/{domain}/{pipeline_id}.yaml
verification_cases/{connector_id}/{operation_id}.yaml
references/providers/{provider}/{connector}/
```

현재 조달 데이터 흐름:

```
pps_contract_api → pps_contract_ingestion
→ raw/normalized DB → teoria_public_procurement Database Source
→ Mapping → public_procurement Runtime Object
```

규칙

Connector는 실행에 사용하는 API wire 계약이다. `definitions/`의 Pipeline은 검토·검증용 수집 계약 목록이며 Python 실행 엔진의 설정 파일이 아니다.

`flow_modules`는 해당 수집과 후속 처리를 소유하는 Flow 모듈 목록이다. Deployment entrypoint는 이 목록과 정확히 일치해야 한다.

`operations`와 `sink`는 사용 Operation·적재 대상의 선언이다. Operation 목록은 구현 및 Connector와, Runtime sink는 Platform Source와 일치하는지 검증한다.

`cursor`는 수집 범위의 설명용 기준이고 `provenance`는 구현해야 하는 보존 정책이다. 두 항목을 바꾸는 것만으로 실행 동작이 바뀌지 않는다. 실제 범위와 보존은 Python Flow·Task가 소유하며 실행 파라미터·스케줄은 `prefect*.yaml`을 함께 수정한다.

모든 Connector는 대응 수집 정의를 갖는다. 수집 Deployment의 `pipeline:<definition_id>`·`domain:<domain_id>` tag는 수집 계약과 소유권을 연결한다. 공고의 면허·지역 API 보강 등 후속 처리는 입력 공고를 제공하는 수집 계약에 연결한다.

조회 대상 같은 업무 입력은 `config/`, schedule·work pool·실행 파라미터는 `prefect*.yaml`에 둔다. 선언된 운영 값은 Deployment 목록 [§28]에서 확인한다.

Flow는 Task 조합만 담당한다.

API·DB side effect는 Task, 순수 정규화는 일반 함수로 구현한다.

API→DB 변환은 `src/teoria_pipelines/domains/<domain>/<feature>/normalization.py`, DB→Runtime Object 변환은 Platform Mapping에 둔다. Ontology Concept와 자산의 업무 의미 연결은 별도 Semantic Binding으로 관리한다.

Python Flow·Task·API client는 기능 패키지에 함께 둔다. `pipelines/connectors/`는 YAML 계약이며 Python 호출 코드는 해당 기능의 `client.py`다.

DB SQL과 transaction은 기능별 `repository.py`가 소유한다. Task는 repository를 호출해 실행·재시도 경계를 표현한다. 조달 전체 저장소를 합치는 facade는 두지 않는다.

Raw 보존 수집에는 connector, operation, fetched time, hash와 payload를 남기고 secret은 저장하지 않는다. `retain_raw_records: false`인 낙찰·개찰의 원본 비보존과 정규 이력 정책은 Data DB [§27]를 따른다.

Checkpoint는 적재 성공 후에만 이동한다.

계약기관(`cntrctInstt*`)과 복수 수요기관(`dminsttList`) 역할을 구분한다.

Provider 근거 문서

Connector의 근거 원문과 metadata를 다음 경로에 함께 보존한다.

```
pipelines/references/providers/{provider}/{connector}/
├── metadata.yaml
└── 문서 파일
```

Metadata 는 `target: connector` 와 Connector ID 를 사용하고 `registry` 는 `pipelines/` 기준 상대 경로로 기록한다. API 키, 개인정보와 전체 Live 응답은 저장하지 않는다.

검증

모든 Operation 에 검증 입력을 준비한다. 정적 검증·요청 Build 를 먼저 수행하고, credential 이 있으면 안전한 데이터로 `--profile live` 까지 실행한다.

```
uv run --locked --package teoria-pipelines teoria-pipelines validate pipelines
uv run --locked --package teoria-pipelines --group validation \
  teoria-pipelines validate-integration pipelines \
  --platform-registries platform/registries

uv run --locked --package teoria-pipelines teoria-pipelines verify connector \
  --profile build \
  --connector {connector_id} \
  --operation {operation_id} \
  --input pipelines/verification_cases/{connector_id}/{operation_id}.yaml
```

실제 응답 검증은 `--profile live` 를 사용한다. Reference `registry` 경로는 `pipelines/` 기준으로 기록한다.

법령 근거 저장소

`moleg_law_api` 는 시행일 기준 법령 목록·전체 본문을 직접 수집한다. `moleg_law_ingestion` 은 `type: data_database` sink 로 `legal_information` 에 적재하며 Runtime Source 등록을 요구하지 않는다. 기존 sink 는 기본 `type: runtime_source` 로 Source 와 relation 검증을 유지한다. 자세한 실행·원문 보존·시행 버전 규칙은 법령 수집 [§ 30]을 참고한다.

다음 작업

Flow·Task 를 처음 익힐 때는 PostgreSQL ETL 예제 [§ 34]를 따라 해 본다. 실제 기능의 코드 배치는 개발 안내 [§ 29]를 따른다.

스케줄 등록·재시도는 Prefect 운영 [§ 33]을 따른다.

개찰 보강의 완료 판정은 공공조달 수집 [§ 31]을 따른다.

27. database.md — Data DB 와 migration

원문 파일: docs/pipelines/database.md

Teoria Data Pipelines 가 수집 DB 의 물리 schema 와 migration 을 소유한다. Semantic Platform 은 Database Source Registry 를 통해 정규 relation 을 읽을 뿐 migration 을 실행하지 않는다.

`ingestion`: 실행 감사, 원본 Provider 응답, Checkpoint

`public_procurement`: 정부조달 계약과 기관·공급자 정규 데이터

파일명은 적용 순서를 포함한 `<3 자리 번호>_<설명>.sql` 을 사용한다. 적용된 파일명은 `ingestion.schema_migrations` 에 기록되므로 이미 적용한 migration 파일을 수정하지 않고 새 파일을 추가한다.

```
uv run --locked --package teoria-pipelines \
  teoria-pipelines migrate \
  --migrations pipelines/database/migrations
```

실행에는 `TEORIA_PIPELINE_DATA_DATABASE_URL` 이 필요하다. 로컬 Compose 에서는 `postgres-migrate` 일회성 서비스가 같은 명령을 실행한다.

원본·관찰·정규 데이터

Provider 원본은 `ingestion.raw_provider_payloads` 에 hash 기준으로 한 번 저장하고, 실행별 조회 범위·관찰 시각은 `ingestion.raw_provider_observations` 에 보관한다. 실행 감사는 `ingestion.pipeline_runs`, 업무 데이터는 `public_procurement` 와 `legal_information` 스키마가 소유한다. 법령 XML 은 전용 raw·snapshot 구조를 사용한다.

원본 보존 여부는 `definitions/`의 `provenance.retain_raw_records` 와 실제 Flow 를 함께 확인한다. 현재 계약·입찰공고·업종 수집은 Raw 를 보존하고, 법령은 전용 XML 보관 구조를 사용한다. 낙찰·개찰은 `retain_raw_records: false` 이며 성공 응답 본문은 정규화·검증 후 버리고 정규 행의 `source_record_hash` 와 실행 감사를 남긴다. 따라서 이 경로의 `raw_record_count=0` 만으로 수집 결과가 비어 있다고 판단하지 않는다. 낙찰·참여·공고 결과의 정규 처리 건수와 보강 큐의 검증 상태를 함께 확인한다. 이 경로는 원본 본문을 통한 사후 재현을 제공하지 않는다.

Pipeline writer 와 Runtime reader 의 role 을 분리한다. Runtime 에는 필요한 정규 relation 과 read view 만 허용하고 `ingestion` 작업 테이블·raw 권한을 확장하지 않는다. OpenMetadata entity·설명·소유자·태그를 Data DB 에 복제하지 않는다.

시각 컬럼 규칙

변경 가능한 업무·정규화 테이블은 `created_at`, `updated_at` 을 사용한다.

`created_at` 은 최초 INSERT 시각이며 이후 변경하지 않는다.

`updated_at` 은 실제 저장 값이 변경되는 UPDATE 에서만 갱신한다.

불변 원본과 실행·적용 이벤트는 의미가 명확한 `fetched_at`, `started_at`, `finished_at`, `applied_at` 을 유지한다.

외부 제공기관이 기록한 시각은 `source_*_at` 으로 DB 레코드의 감사 시각과 구분한다.

모든 시각 컬럼은 PostgreSQL `timestamptz` 를 사용한다.

타임존 표기가 없는 제공기관 시각은 제공기관의 명시된 지역 타임존으로 먼저 해석한 뒤 UTC 로 변환한다. 나라장터의 입찰공고 시각은 `Asia/Seoul` 로 해석한다.

기능별 migration 적용 시 확인할 사항

첨부문서·참가요건 추출 테이블은 보류 기능의 기존 데이터로 보존한다. 현재 Worker 는 이 테이블에 새 처리를 등록하지 않는다. `089_detach_notice_view_from_document_extraction.sql` 은 공고 조회에서 추출 테이블 연결을 제거하며 종속 view 의 기존 컬럼 위치는 NULL 로 유지한다. 추출 결과 필드는 활성 Source·Mapping·Runtime 계약에서 제외한다.

`088_version_contract_ledger_refresh_requests.sql` 은 계약 집계 갱신 요청에 버전을 추가한다. Worker 는 자신이 처리한 버전만 완료하므로 갱신 중 새로 요청한 작업을 보존한다.

`081_bid_opening_runtime_collection_status.sql` 은 참여 분석에 필요한 수집 상태를 `public_procurement.runtime_bid_opening_collection_status` view 로 제공한다. Runtime 에는 이 view 의 SELECT 만 허용하고 ingestion 작업 테이블은 노출하지 않는다. 새 Platform 참여 분석을 배포하기 전에 이 migration 을 적용해야 한다.

`082_legal_information_law_collection.sql` 은 법령 ID · MST · 시행일별 목록, 인증값이 제거된 원문 bytes · 관측, 파서 버전별 snapshot, 조항 계층과 별표 정보를 추가한다. 기존 조달 테이블이나 적용된 migration 을 수정하지 않는다. Runtime 읽기 권한은 추가하지 않는다.

`083_legal_information_search_views.sql` 은 수집 상태를 포함한 법령 버전 view 와 조문 · 부칙 하위 내용을 합친 검색 view 를 추가한다. `teoria_runtime` 에는 스키마 USAGE 와 두 view SELECT 만 부여하며 원문 · 스냅샷 · Raw 테이블 권한은 부여하지 않는다.

`084_asset_collection_context.sql` 은 명시한 공공조달 Pipeline output 만 연결하는 `runtime_asset_collection_status` view 와 Pipeline 별 최근 실행 조회 index 를 추가한다. OpenMetadata entity ID · 설명 · 컬럼 상태를 저장하지 않으며 view 의 SELECT 만 허용한다. Prefect schedule 은 복제하지 않고 ingestion 원본 · 작업 테이블 권한도 확장하지 않는다. 새 통합 자산 조회를 배포하기 전에 적용해야 한다. 커스텀 Pipeline ID 또는 파생 relation 은 이름으로 추정하지 않으며 output 계약에 별도로 명시해야 한다.

`085_procurement_entity_search_index.sql` 은 조달 개체 자동완성용 내부 검색 인덱스를 추가한다.
`086_bid_notice_region_filter_indexes.sql` 은 계약에 연결된 공고의 구조화 참가 지역조건을 네트워크 · 시장 조회에서 필터링하기 위한 공고번호 · 지역코드 인덱스를 추가한다.

`087_bound_bid_opening_retries.sql` 은 선택 업체 검증값이 없는 과거 완료 건을 다시 백필 대상으로 전환한다. 불완전 응답과 조회 실패는 각각 제한된 재시도 횟수 이후 `partial_completed` 와 `permanent_failure` 로 종결하며 Runtime 에는 `failed` 로 제공한다.

28. deployments.md — Pipeline Deployment 목록

원문 파일: docs/pipelines/deployments.md

이 표는 `pipelines/prefect*.yaml` 에서 생성한다. 수동으로 수정하지 않는다. 저장소에 선언된 설정이며 실행 중인 Prefect Server 상태를 조회한 결과는 아니다.

```
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs >
docs/pipelines/deployments.md
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs --check
docs/pipelines/deployments.md
```

`openmetadata-postgres-metadata-sync`

		Teoria Data PostgreSQL metadata 를 격리된 OpenMetadata ingestion container 로 동기화한다.

선언: `pipelines/prefect.metadata.yaml`

Flow: `teoria_pipelines.metadata.openmetadata.sync_openmetadata_postgres`

Work pool: `teoria-metadata`

Work queue: `default`

파라미터: `image=teoria-openmetadata-ingestion:2.0.5`, `network=teoria_default`, `timeout_seconds=1800`

Cron

Timezone

예약 상태

`0 5 * * *`

`Asia/Seoul`

활성

`pps-industry-weekly`

		나라장터 표준업종과 근거법규 전체 사전을 매주 일요일 동기화한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.industries.flows.sync_pps_industries`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_industry_ingestion`

파라미터: `pipeline_root=/app/pipelines`

Cron

Timezone

예약 상태

`0 3 * * 0`

Asia/Seoul

활성

pps-bid-notice-ingestion

		신규·변경 나라장터 입찰공고를 수집하고 공고번호별 면허제한과 참가가능지역을 보강한다.

선언: pipelines/prefect.yaml

Flow: teoria_pipelines.domains.public_procurement.bid_notices.flows.sync_pps_bid_notices

Work pool: teoria-ingestion

Work queue: latest

수집 계약: pps_bid_notice_ingestion

파라미터: pipeline_root=/app/pipelines, lookback_days=1, enrichment_batch_size=20

Cron

Timezone

예약 상태

0 * * * *

Asia/Seoul

활성

pps-contract-incremental

		최근 나라장터 계약정보를 4 시간마다 다시 조회하여 신규·변경 데이터를 반영한다.

선언: pipelines/prefect.yaml

Flow: teoria_pipelines.domains.public_procurement.contracts.flows.sync_pps_contract_incremental

Work pool: teoria-ingestion

Work queue: latest

수집 계약: pps_contract_ingestion

파라미터: pipeline_root=/app/pipelines, lookback_days=3

Cron

Timezone

예약 상태

0 */4 * * *

Asia/Seoul

활성

pps-contract-event-ledger-refresh

		변경된 공식 조달분야의 원계약

		· 공동수급 귀속 원장과 인사이트 사전집계를 갱신한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.contracts.flows.refresh_pps_contract_event_ledger`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_contract_ingestion`

파라미터: `batch_size=5`

Cron

Timezone

예약 상태

`15 * * * *`

`Asia/Seoul`

활성

`pps-bid-result-incremental`

		최근 3 일의 최종낙찰과 해당 공고의 개찰 참여업체 결과를 4 시간마다 갱신한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.sync_pps_bid_results_incremental`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=3`

Cron

Timezone

예약 상태

`30 */4 * * *`

`Asia/Seoul`

활성

`pps-bid-notice-reconciliation-3d`

		최근 3 일의 신규·변경 입찰공고와 참가제한 정보를 매일 보정한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_notices.flows.sync_pps_bid_notices`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_notice_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=3`

Cron

Timezone

예약 상태

`10 1 * * *`

`Asia/Seoul`

비활성

`pps-contract-reconciliation-30d`

		최근 30 일 계약정보의 지연 등록과 정정을 매일 보정한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.contracts.flows.sync_pps_contract_incremental`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_contract_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=30`

Cron

Timezone

예약 상태

`0 2 * * *`

`Asia/Seoul`

활성

`pps-contract-reconciliation-90d`

		최근 90 일 계약정보의 장기 지연·정정을 매주 보정한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.contracts.flows.sync_pps_contract_incremental`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_contract_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=90`

Cron

Timezone

예약 상태

`0 3 * * 0`

`Asia/Seoul`

활성

`pps-bid-result-reconciliation-30d`

		최근 30 일 개찰 건의 지연 낙찰 확정과 참여업체 결과를 매일 보정한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.sync_pps_bid_results_incremental`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=30`

Cron

Timezone

예약 상태

`30 2 * * *`

`Asia/Seoul`

활성

`pps-bid-result-reconciliation-90d`

		최근 90 일 개찰 건의 장기 지연 낙찰 확정을 매주 보정한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.sync_pps_bid_results_incremental`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `lookback_days=90`

Cron

Timezone

예약 상태

`30 3 * * 0`

`Asia/Seoul`

활성

`pps-contract-backfill`

		계약정보를 실행당 7 일씩 역순

		Backfill 하며 실패한 날짜 ·Operation 은 결손 큐로 격리해 전체 진행을 계속한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.contracts.flows.sync_pps_contract_backfill`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_contract_ingestion`

파라미터: `checkpoint_id=pps_contract_backfill_2021_2026`, `start_date=2021-01-01`, `end_date=2026-09-13`, `pipeline_root=/app/pipelines`, `batch_days=7`

Cron

Timezone

예약 상태

`20 4 * * *`

`Asia/Seoul`

활성

`pps-contract-backfill-gap-retry`

		계약 Backfill 결손 Operation 을 작은 페이지 크기로 최대 15 회까지 매일 재시도한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.contracts.flows.retry_pps_contract_backfill_gaps`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_contract_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `source_pipeline_id=pps_contract_backfill_2021_2026`, `batch_size=1`, `retry_days=1`, `max_attempts=15`

Cron

Timezone

예약 상태

`20 5 * * *`

`Asia/Seoul`

활성

`pps-bid-notice-backfill`

		2026-09-13 부터 2021-01-01 까지 최신 날짜 우선으로 첨부 파일과 참가제한 보강을 제외한

		기본 입찰공고를 Backfill 한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_notices.flows.sync_pps_bid_notice_backfill`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_notice_ingestion`

파라미터: `checkpoint_id=pps_bid_notice_backfill_2021_2026`, `start_date=2021-01-01`, `end_date=2026-09-13`, `pipeline_root=/app/pipelines`, `batch_days=7`

Cron

Timezone

예약 상태

`50 * * * *`

`Asia/Seoul`

활성

`pps-bid-notice-enrichment-backfill`

		기본 공고와 독립적으로 7 일 이상 지난 미처리 공고의 면허제한과 참가가능지역을 제한된 호출량으로 보강한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_notices.flows.sync_pps_bid_notice_enrichment_backfill`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_notice_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `batch_size=20`, `chunk_size=5`, `minimum_age_days=7`, `lease_minutes=60`

Cron

Timezone

예약 상태

`25 * * * *`

`Asia/Seoul`

활성

`pps-bid-result-backfill`

		현재 연도 포함 최근 5 개 회계연도(2022 년 이후) 낙찰정보를 최신 연도 우선으로 Backfill 한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.sync_pps_bid_results_backfill`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_result_ingestion`

파라미터: `checkpoint_id=pps_bid_result_backfill_5y`, `start_date=2022-01-01`, `end_date=2026-09-13`, `pipeline_root=/app/pipelines`, `batch_days=30`

Cron

Timezone

예약 상태

`40 * * * *`

`Asia/Seoul`

활성

`pps-bid-opening-incremental-enrichment`

		신규 낙찰의 개찰 참여업체 Top 10 과 낙찰업체를 우선 보강한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.enrich_pps_bid_opening_participants`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `batch_size=30`, `lease_minutes=15`, `queue_mode=incremental`

Cron

Timezone

예약 상태

`*/5 * * * *`

`Asia/Seoul`

활성

`pps-bid-notice-outcome-enrichment`

		낙찰이 확인되지 않은 개찰 공고를 조달청 유찰 목록으로 확인해 확정 상태와 사유를 보강한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.enrich_pps_bid_notice_outcomes`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `batch_size=100`

Cron

Timezone

예약 상태

`3-59/5 * * * *`

`Asia/Seoul`

활성

`pps-bid-opening-backfill-enrichment`

		최근 5 개년 개찰 참여업체를 5 분마다 최대 180 개 공고씩 최신 건부터 과거 방향으로 지속 보강한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.enrich_pps_bid_opening_participants`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `batch_size=180`, `lease_minutes=30`, `queue_mode=backfill`

Cron

Timezone

예약 상태

`1-59/5 * * * *`

`Asia/Seoul`

활성

`pps-bid-opening-retry-enrichment`

		실패·부분 수집된 개찰 건을 유형별 상한과 지연 정책으로 재시도한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.public_procurement.bid_results.flows.enrich_pps_bid_opening_participants`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `pps_bid_result_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `batch_size=60`, `lease_minutes=15`, `queue_mode=retry`

Cron

Timezone

예약 상태

`7,22,37,52 * * * *`

`Asia/Seoul`

활성

`moleg-law-daily`

		지정 법령의 전체 버전 목록을 확인하고 신규·변경·현행·시행예정 본문을 수집한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.legal_information.laws.flows.sync_moleg_laws`

Work pool: `teoria-ingestion`

Work queue: `latest`

수집 계약: `moleg_law_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `reconcile_all=False`, `requests_per_second=1`

Cron

Timezone

예약 상태

`15 4 * * 1-6`

`Asia/Seoul`

활성

`moleg-law-weekly-reconciliation`

		지정 법령의 모든 연혁 본문을 재조회해 기존 식별자의 원문 정정도 보존한다.

선언: `pipelines/prefect.yaml`

Flow: `teoria_pipelines.domains.legal_information.laws.flows.sync_moleg_laws`

Work pool: `teoria-ingestion`

Work queue: `backfill`

수집 계약: `moleg_law_ingestion`

파라미터: `pipeline_root=/app/pipelines`, `reconcile_all=True`, `requests_per_second=1`

Cron

Timezone

예약 상태

`15 4 * * 0`

`Asia/Seoul`

활성

29. development.md — Pipelines 개발 안내

원문 파일: docs/pipelines/development.md

Pipelines 는 외부 데이터를 수집하고 원본과 정규 데이터를 Teoria Data DB 에 적재한다. Prefect 는 실행·스케줄·재시도를 관리한다. 업무 의미와 사용자 요청 실행은 Platform 이 담당하며 Pipeline 실행 코드가 Platform Runtime 을 호출하지 않는다.

```
Connector API → 조회 → Raw 저장 → 정규화 → 정규 적재 → Checkpoint
```

위는 Raw 보존 수집의 흐름이다. 낙찰·개찰의 성공 응답은 정규화 후 버리며 원천 hash 와 실행 이력을 보존한다. 기능별 정책은 Data DB [§27]를 따른다.

Prefect 가 처음이라면 PostgreSQL ETL 예제 [§34]를 먼저 실행해 본다. 이 문서는 실제 수집 기능의 코드 위치와 개발·검증 환경을 설명한다.

코드와 설정을 찾는 순서

배포 목록 [§28]에서 실행 이름, Flow entrypoint 와 수집 계약을 찾는다.

`pipelines/definitions/<domain>/<pipeline_id>.yaml` 에서 Connector·Operation·적재 대상을 확인한다.

기능 패키지의 `flows.py` 에서 순서를, Task 모듈에서 재시도와 I/O 경계를 읽는다.

API 요청은 `client.py`, 값 변환은 `normalization.py`, SQL·transaction 은 `repository.py` 를 수정한다.

아래 트리는 `pipelines/` 기준이다.

```
connectors/                          Provider wire 계약
references/providers/                계약의 원문 근거
verification_cases/                  Operation 별 검증 입력
definitions/<domain>/                수집·output 계약
config/                              법령 대상 같은 업무 입력
prefect.yaml                         데이터 수집 Deployment
prefect.metadata.yaml                OpenMetadata Job Deployment
database/migrations/                 Data DB schema
src/teoria_pipelines/
├── cli.py                           검증·migration·worker·law 명령
├── deployment.py                    두 배포의 Prefect 등록·Queue·실행 정책
├── domains/
│   ├── public_procurement/
│   │   ├── contracts/               계약 수집
│   │   ├── bid_notices/             공고·API 면허·지역 정보 수집
│   │   ├── bid_results/             낙찰 목록·개찰 보강
│   │   ├── industries/              업종 사전
│   └── legal_information/laws/      법령 원문·시행 버전
├── metadata/                        격리된 OpenMetadata Job 실행
├── execution/                       실행 기록·Raw 저장·Checkpoint Task
├── checkpoints/                     수집 범위·재개 정책
├── persistence/                     비동기 DB 연결·Raw·실행 기록·Checkpoint 저장
└── flows/pps_industries.py           기존 운영 배포 한 개의 호환 경로
```

기능 구현 규칙

각 기능은 Flow·Task·client·정규화와 전용 `models.py`·`repository.py` 를 함께 둔다. Flow 는 단계 조합을 보여주고 API 호출과 DB 쓰기는 Prefect Task 로 표현한다. 원본 응답과 정규 적재가 모두 성공한 뒤 Checkpoint 를 갱신한다.

기능별 repository 는 독립적으로 생성한다. `public_procurement/repository.py` 는 조달 실행 건수 기록만, `public_procurement/tasks.py` 는 그 완료 기록의 Task 경계만 소유한다. 공통 `LoadSummary` 는 원본 건수, Domain 의 `models.py` 는 조달 집계를 담당한다. 공통 `persistence/`가 조달 구현을 import 하지 않게 유지한다.

입찰공고는 `ingestion_tasks.py` 와 `enrichment_tasks.py` 로 수집과 API 보강을 구분한다. 첨부파일·문서 요건 기능은 `archive/bid_document_processing/`에 보류하며 실행 패키지에 포함하지 않는다.

새 배포는 기능의 `flows.py` 를 참조한다. 이전 업종 배포를 위한 호환 모듈은 운영 entrypoint 전환 뒤 제거한다. 저장소의 선언값과 실제 Prefect 등록 상태는 별개이므로 전환 여부는 운영에서 확인한다.

I/O 와 동시 실행

API·DB Task 는 `async def` 로 정의하고 Flow 에서 `await` 한다. 변환·검증 Task 는 일반 함수로 유지한다. 독립 Operation 을 병렬화할 때도 Provider rate limit 과 재시도 정책을 지키며 Raw 저장 → 정규 적재 → Checkpoint 순서를 바꾸지 않는다.

DB 연결은 `persistence/connection.py` 의 `connect()`가 소유한다. `TEORIA_PIPELINE_DATABASE_CONCURRENCY_LIMIT` 의 기본값은 10 이며 event loop 별·DB 별 활성 연결을 제한한다. 여기서 DB 는 동일한 접속 URL 단위다. 여러 Worker·프로세스의 상한은 합산해서 DB 연결 예산에 맞춘다. 연결을 재사용하는 pool 이 아니며 Prefect Task 사이에 connection·transaction 을 전달하지 않는다. 성공 시 commit, 예외·취소 시 rollback 후 연결을 닫고 대기 슬롯을 반환한다.

동기 SDK 가 필요한 Docker·Kubernetes Job 실행은 별도 동기 Task 경계에서 처리한다. CLI 진입점만 `asyncio.run()`을 사용하며 Flow 내부에 중첩하지 않는다. Prefect Task 호출 [참고 1]과 Psycopg 비동기 연결 [참고 2]을 참고한다.

개발과 검증

명령은 저장소 루트에서 실행한다. Flow 를 로컬 실행할 때는 `orchestration` group, OpenMetadata Job 은 `metadata-orchestration`, Platform 계약 교차 검증은 `validation` group 이 필요하다. 기본 Worker 에 OpenMetadata ingestion 패키지를 설치하지 않는다. K3s 실행 template 와 Prefect 실행 정책 통합 검증에는 `kubernetes-orchestration` group 을 사용한다.

```
uv run --locked --package teoria-pipelines pytest pipelines/tests
uv run --locked --package teoria-pipelines teoria-pipelines validate pipelines
uv run --locked --package teoria-pipelines --group validation \
  teoria-pipelines validate-integration pipelines \
  --platform-registries platform/registries
```

Pipeline 단독 DB 통합 테스트는 `pipelines/tests/integration/`, Pipeline 과 Platform 사이의 계약·조회 통합 테스트는 저장소 루트 `tests/integration/`에 둔다. 법령 snapshot 저장·재파싱은 `pipelines/tests/integration/test_law_persistence.py`, 법령 수집 결과의 Platform 검색과 Runtime 읽기 권한은 루트 `tests/integration/test_law_search_contract.py` 에서 검증한다. XML 입력은 Pipeline 소유 fixture 를 사용하며 각 테스트 모음이 필요한 DB 상태를 직접 준비한다.

조달 단위 테스트는 `pipelines/tests/unit/domains/public_procurement/` 아래에 두며 `contracts/`, `bid_notices/`, `bid_results/`, `industries/`가 실행 코드의 기능과 대응한다. 조달 달력·여러 조달 기능에 걸친 migration 검증은 해당 Domain 바로 아래에, 공통 checkpoint·저장·설정·배포 검증은 `unit/`에 둔다. pytest 는 `importlib` 모드를 사용하므로 기능별 `test_repository.py` 같은 파일명이 충돌하지 않는다.

격리된 PostgreSQL 의 `teoria_data` DB 와 `teoria_pipeline`, `teoria_runtime` role 을 준비하고 `TEORIA_TEST_DATA_DATABASE_URL` 을 설정한다. 기존 migration 의 권한 계약이 DB 이름을 참조하므로 CI 도 같은 이름을 사용한다.

```
uv run --locked --package teoria-pipelines --group validation pytest pipelines/tests/integration
tests/integration
```

Flow 또는 Data DB migration 을 변경하면 Compose 도 검증한다. Deployment 를 변경하면 목록을 재생성하고 선언과 문서의 일치를 확인한다.

```
docker compose -f deploy/compose/compose.yaml config --quiet
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs >
docs/pipelines/deployments.md
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs --check
docs/pipelines/deployments.md
```

Connector 변경의 Operation 별 Build·Live 검증은 Connector 작성 [§ 26]을 따른다. 실행·재시도는 Prefect 운영 [§ 33], 저장 규칙은 Data DB [§ 27], 업무별 처리는 공공조달 수집 [§ 31]과 법령 수집 [§ 30]을 참고한다.

외부 참고 주소

[1] https://reference.prefect.io/prefect/tasks/

[2] https://www.psycopg.org/psycopg3/docs/advanced/async.html

30. legal_information.md — 법령 수집 및 버전 관리

원문 파일: docs/pipelines/domains/legal_information.md

법제처 공식 API 를 직접 호출하는 독립 수집 기능이다. `korean-law-mcp` 는 요청·연혁 처리의 참고 자료이며 실행 의존성이 아니다. 수집·파싱·Data DB 는 `pipelines/src/teoria_pipelines/domains/legal_information/laws/`가 소유한다. 이번 단계는 법령 근거 저장소이며 개인정보 위험등급, 적법성 판정, Ontology 와 OpenMetadata 쓰기는 구현 범위에 포함하지 않는다.

실행

인증값은 `TEORIA_CONNECTOR_MOLEG_LAW_API_KEY` 에 설정한다. Provider 요청의 `OC` 에 전달되며 Registry·로그에는 값 대신 환경변수 이름만 사용한다. Docker 배포는 `deploy/compose/.env`, 로컬 실행은 `.env` 또는 아래처럼 `TEORIA_ENV_FILE` 로 선택한다. Data DB migration 은 운영 변경 전에 별도 검토하고 격리된 DB 에서 먼저 검증한다.

```
export TEORIA_ENV_FILE=deploy/compose/.env
# 로컬 DB 주소 또는 격리된 테스트 DB 주소를 설정한다.
export TEORIA_PIPELINE_DATA_DATABASE_URL='postgresql://.../teoria_data'
uv run --locked --package teoria-pipelines teoria-pipelines migrate
uv run --locked --package teoria-pipelines --group orchestration \
  teoria-pipelines law sync --pipelines pipelines
```

`pipelines/config/law_targets.yaml` 은 수집 범위다. 초기 범위는 개인정보 보호법, 공공데이터법, 정보공개법과 각 시행령 6 개다. 이름은 공식 명칭을 쓰고, 필요하면 `law_id` 를 명시한다. 최초 이름 검색은 현행 목록의 정확한 이름과 유일한 ID 만 허용한다. 최초 등록 이후에는 DB 에 고정한 ID 로 추적하므로 제명 변경에 영향을 받지 않는다. 같은 이름의 폐지·재제정 법령은 서로 다른 ID 를 자동 병합하지 않는다.

```
# 한 대상만 수집: 전체 수집과 별도 checkpoint 를 사용한다.
uv run --locked --package teoria-pipelines --group orchestration \
  teoria-pipelines law sync --pipelines pipelines \
  --target personal_information_protection_act

# 모든 연혁 본문을 재확인한다.
uv run --locked --package teoria-pipelines --group orchestration \
  teoria-pipelines law sync --pipelines pipelines --reconcile-all

# 저장된 원문을 재가공한다. 법제처 호출이나 API 인증값이 필요하지 않다.
uv run --locked --package teoria-pipelines --group orchestration \
  teoria-pipelines law reparse --all

uv run --locked --package teoria-pipelines teoria-pipelines law versions \
  --target personal_information_protection_act

# versions 결과의 정확한 식별자·시행일을 사용한다.
uv run --locked --package teoria-pipelines teoria-pipelines law show \
  --law-id 11357 --mst 283839 --effective-date 2026-09-11 --article 24의2
```

로컬 Flow 실행도 Prefect 설정을 사용한다. 운영 Prefect 에 연결하려면 올바른 `PREFECT_API_URL` 과 `PREFECT_API_AUTH_STRING` 을 함께 설정한다. API URL 을 설정하지 않은 개발 실행은 Prefect ephemeral server 를 사용한다. 개발 server 와 client 의 Basic Auth 설정도 서로 맞아야 한다.

원문과 버전

`law`: 법령 ID 및 수집 대상 ID 의 고정 연결.

`law_version`: `(law_id, mst, effective_date)` 시행일별 식별자와 목록 정보.

`ingestion.law_raw_payloads`: 응답 XML bytes 및 SHA-256. 법제처 응답 링크가 OC 를 되돌려주므로 인증값을 제거한 XML 을 보관하고 이 bytes 의 hash 를 사용한다. 일반 XML 의 순서·속성·CDATA 를 재직렬화하지 않는다.

`ingestion.law_raw_observations`: 실행·인증정보 없는 요청 파라미터·관측 시각.

`law_snapshot`: 원문 hash, 파서 버전, 해당 시점의 기본정보·목록 정보.

`law_provision`: snapshot 에 연결된 장·절 등 제목, 조·항·호·목 및 부칙. `parent_path` 는 계층, `path` 는 원문 위치, `number` 는 Provider 번호다.

`law_attachment`: 동일 본문에 포함된 별표·서식 정보와 파일 링크 XML. 첨부파일 바이너리 다운로드·내용 파싱은 이번 단계에서 수행하지 않는다.

`law_collection_run`: 대상·목록·적재 건수 및 실패한 대상/버전. 실행 상태는 기존 `ingestion.pipeline_runs` 를 참조한다.

같은 원문·파서 버전의 재실행은 snapshot 을 중복 생성하지 않는다. 같은 식별자의 원문이 수정되거나 파서 버전이 바뀌면 새 snapshot 을 추가하며 이전 조항과 원문은 보존한다. 기본 조회는 해당 시행 버전의 최신 snapshot 을 반환하고 근거에 사용할 snapshot ID·hash·parser version 을 함께 제공한다. 과거 snapshot 은 DB 에서 ID 로 직접 조회할 수 있다. 원문 bytes 는 Data DB 의 `bytea` 에 보관되므로 운영에서는 Data DB Docker volume 과 백업에 포함되며 로컬 코드 폴더에 누적하지 않는다.

완전성 및 실행 순서

```
대상 확인 → 법령 ID 확인 → 연혁·현행·예정 목록 전체 수집
→ 목록 XML 보관 → 목록 등록 → 수집할 버전 선택
→ 버전 본문 수집 → 본문 XML 보관 → 버전 검증·조항 파싱
→ snapshot·조항 트랜잭션 적재 → 전체 성공 시 checkpoint 갱신
```

목록은 `LID`, `nw=1,2,3`, `display<=100`, `page` 를 사용한다. 모든 페이지의 전체 건수·페이지 번호·법령 ID 와 `(MST, 시행일)` 중복을 검증한다. 안전한 최대 페이지 수를 넘거나 기존 버전이 목록에서 사라지면 성공으로 축약하지 않고 실패로 기록한다. HTML 오류·잘못된 XML·DTD·빈 본문은 정상 빈 결과로 취급하지 않는다.

본문은 `MST + efYd` 를 고정하고 `JO` 제한 없이 전체를 조회한다. 본문이 항상 MST 를 되돌려주지는 않으므로 법령 ID·시행일·공포일·공포번호를 목록과 대조하고, MST 가 존재하면 추가 대조한다. 요청 날짜가 실패해도 현재 법령이나 MST 단독 본문으로 대신 저장하지 않는다.

Raw 저장 후 구조 검증에 실패한 응답은 근거 추적을 위해 남아 있지만 최신 정규 snapshot 으로 승격되지 않는다. 다른 버전은 계속 수집하고 실패 버전은 다음 실행에 재시도한다. 한 버전이라도 미완료이면 전체 checkpoint 는 갱신하지 않는다.

예약 시각과 활성 여부는 자동 생성 배포 목록 [§28]을 따른다. 일별 수집과 전체 연혁 재확인은 별도 Deployment 다. 매일 전체 목록을 확인하고 신규·파서 변경·목록 정보 변경·현행·시행예정 본문을 수집한다. 일요일에는 모든 본문을 다시 받아 오래된 원문의 정정도 확인한다. 기본 호출 속도는 초당 1 회이며 HTTP 재시도는 기존 Provider Executor 가 담당한다.

시행예정도 보관하지만 자동으로 현재 적용 법령으로 판단하지 않는다. 조항별 시행일과 부칙의 예외를 보존하며, 기준일별 법적 적용성 판단은 후속 정책 평가 기능의 책임이다. 이번 단계는 자동 법률 판단이나 개방 승인 기능을 제공하지 않는다.

Platform 연결 경계

법령 Data DB 만 사용하는 Pipeline sink 는 `type: data_database`, `schema_name: legal_information` 을 사용한다. 기존 Runtime Source sink 는 기본 `type: runtime_source` 이며 기존 Source·relation 검증을 유지한다. Runtime 에서 법령 DB 를 사용하기 시작할 때 Database Source 와 읽기 권한을 별도로 추가한다. Admin 진단용 읽기 계정에는 migration 083 의 검색 view 와 스키마 USAGE 만 허용한다. 원본 테이블·Raw 권한과 Runtime Capability 등록은 추가하지 않는다.

법령 조문 구조 자체는 Ontology 로 발행하지 않는다. 이후 보호대상 정보 유형, 처리 목적, 개방 시나리오와 보호조치의 업무 의미가 필요해지면 OpenMetadata Glossary 와 Teoria Business Ontology 를 연결한다. 위험등급 및 적용 조건은 승인된 정책 규칙으로 관리하고 법령 변경은 재검토를 유발한다. 자동 탐지 결과는 Suggestion→Review→Application 절차를 거쳐 OpenMetadata 에 적용해야 한다.

참고 계약

공식 시행일 목록 가이드 [참고 1]

공식 시행일 본문 가이드 [참고 2]

참고한 korean-law-mcp 계보 조회 [참고 3]

Connector [pipelines/connectors/moleg_law_api.yaml]

Provider Reference [pipelines/references/providers/moleg/moleg_law_api/metadata.yaml]

조회 기능과의 연결

수집된 snapshot 을 사용하는 Admin 진단 API 와 기준일별 검색 규칙은 법령 근거 검색 [§ 16]에 설명한다. Pipeline 은 원문·버전· 검색용 read view 를 소유하고 Platform 은 조회·진단을 소유한다.

외부 참고 주소

[1] https://open.law.go.kr/LSO/openApi/guideResult.do?htmlName=lsEfYdListGuide

[2] https://open.law.go.kr/LSO/openApi/guideResult.do?htmlName=lsEfYdInfoGuide

[3] https://github.com/chrisryugj/korean-law-mcp/blob/c6feff0cbd10cc5b907cdd4f4b3fcf8c59dcc7f7/src/lib/law-lineage.ts

31. public_procurement.md — 공공조달 데이터 수집

원문 파일: docs/pipelines/domains/public_procurement.md

공고·계약·낙찰 목록과 업종 사전을 수집한다. 수집 실행은 `pipelines/src/teoria_pipelines/domains/public_procurement/`가 소유한다. 집계 의미와 조회 기능은 Platform 이 담당한다.

		수집 기능과 저장 흐름

기능

구현 위치

주요 처리

계약

`contracts/flows.py`, `tasks.py`

최신·보정·과거 수집, 결손 재시도

공고

`bid_notices/flows.py`, Task 모듈

기본 공고, API 면허·지역 보강

낙찰·개찰

`bid_results/flows.py`, `tasks.py`

낙찰 목록, 공고별 참여업체 보강

업종

`industries/flows.py`, `tasks.py`

업종 사전

나라장터 계약은 `pps_contract_incremental` 과 `pps_contract_backfill` 이 독립 checkpoint 를 사용한다. 일별 하위 Flow 에서 상품·공사·용역·외자를 조회한 뒤 Raw 저장·정규화·적재를 수행한다. 원본과 정규 적재가 모두 성공한 뒤에만 진행 위치를 갱신한다. Raw 본문은 hash 기준으로 한 번 보관하고 실행별 관찰은 별도 기록한다. 테이블과 감사시각 규칙은 Data DB [§ 27]를 참고한다.

보강과 원본 보존

공고별 면허·지역 보강은 기본 1 RPS 의 완전 순차 호출로 제한한다. 최신 공고 수집은 변경 공고를 즉시 보강하지만, 과거 공고 Backfill 은 기본 공고만 적재하고 체크포인트를 독립적으로 갱신한다. 과거 참가제한은 `pps-bid-notice-enrichment-backfill` 이 7 일 이상 지난 미처리 공고를 배포 파라미터의 처리량으로 선점하고 5 건 단위로 Raw 저장과 정규 적재를 완료한다. 일부 청크가 429 로 실패해도 완료된 공고의 `enrichment_checked_at` 은 보존되며, 실패한 선점은 60 분 뒤 다시 처리할 수 있다.

계약 Backfill 은 날짜별 Operation 완료 상태를 보존한다. 제공기관 오류가 반복되는 날짜·Operation 은 `ingestion.pipeline_backfill_gaps` 에 기록하고 다음 날짜로 계속 진행한다. 별도 Deployment 인 `pps-contract-backfill-gap-retry` 가 결손을 페이지 크기 20, 10, 1 순서로 재시도하며 성공한 결손은 해결 시각을 기록한다. 따라서 단일 제공기관 오류가 전체 역사 수집 체크포인트를 막지 않는다.

첨부파일 다운로드·파싱·입찰요건 추출과 참가 가능성 판단은 보류했다. 코드·계약·검증 기록은 archive [archive/bid_document_processing/README.md]에 보관하며 현재 배포와 수집 큐에 포함하지 않는다. 기존 DB 와 객체는 보존한다.

최신·보정·과거 수집의 구분

입찰공고는 공고게시일시, 계약은 계약체결일자, 낙찰은 개찰일시를 조회 기준으로 사용한다. 공고 API 의 목록 Operation 은 공고번호 단건 조회를 지원하지 않는다. 최근 3 일 보정은 지연 반영과 정정을 보완하기 위한 별도 Deployment 이며 예약 여부는 배포 목록에서 확인한다. 낙찰은 개찰 후 최종낙찰 확정이 늦어질 수 있어 30 일·90 일 보정을 별도로 실행한다. 이 주기와 2021 년 이후 과거 데이터를 채우는 Backfill 은 서로 독립적이다.

계약 수집

계약은 최신 수집·30 일 및 90 일 보정·역방향 Backfill·결손 재시도로 나눈다. 구체적인 예약 상태와 파라미터는 Deployment 목록 [§28]을 확인한다.

증분 수집과 Backfill 은 독립 checkpoint 를 사용한다. 기본 계약 Backfill checkpoint 는 `pps_contract_backfill_2021_2026` 이고 `2026-09-13` 부터 `2021-01-01` 까지 역방향으로 진행한다. 범위를 완료하면 이후 예약 실행은 API 를 호출하지 않는다.

별도 범위를 적재할 때는 UI 에서 같은 Flow 의 Custom Run 이나 Schedule 파라미터에 고유한 `checkpoint_id`, `start_date`, `end_date` 를 지정한다. 서로 다른 Backfill 에 같은 `checkpoint_id` 를 사용하면 진행 상태가 충돌하므로 사용하지 않는다.

공고 수집

입찰공고는 최신 수집·보정·기본 공고 Backfill·과거 참가제한 보강으로 나눈다. 구체적인 예약 상태와 파라미터는 Deployment 목록 [§28]을 확인한다.

입찰공고 Backfill 은 `pps_bid_notice_backfill_2021_2026` checkpoint 를 사용한다. 기본 공고 원본과 정규 공고 저장이 성공하면 날짜를 과거로 이동한다. 면허·지역 제한은 독립 보강 큐가 처리하며 과거 첨부파일은 등록하지 않는다.

낙찰·개찰 수집

낙찰 목록 수집과 개찰 참여업체 보강은 독립 Flow 다. 증분·보정·Backfill·보강의 예약 상태와 범위는 Deployment 목록 [§28]을 확인한다.

낙찰 목록 수집은 정규화·적재와 개찰 보강 대상 등록이 성공하면 체크포인트를 갱신한다. 공고별 개찰조회는 별도 `pps-bid-opening-enrichment` Flow 가 처리하며, 실패 건은 30 분·2 시간·12 시간·1 일 간격으로 재시도한다. 따라서 제공기관 개찰 API 지연은 낙찰 증분 수집이나 Backfill 체크포인트를 막지 않는다. Backfill 의 `checkpoint_id`, `start_date`, `end_date` 는 필수이며 `end_date` 는 오늘보다 이전이어야 한다.

공공데이터포털은 데이터 갱신주기를 실시간으로 안내하지만 정확한 반영 지연은 보장하지 않는다. Incremental 은 지연 반영을 고려해 최근 3 일을 중복 조회하며 정규 테이블에 upsert 한다.

개찰 보강 완료 판정

개찰 원본은 모든 페이지의 totalCount·행 수·중복·공고 키와 알려진 낙찰자를 검증한 뒤 저장한다. 일부 페이지만 조회하거나 응답이 잘린 경우 기존 참여업체를 교체하지 않고 해당 공고만 재시도한다. 정규 저장 대상은 순위 1~10 위와 낙찰자다. 전체 참가자 수와 정규 저장 수를 비교하지 않는다. 수집 상태의 `participant_count` 는 원본 조회 행 수를 유지하고, `selected_business_numbers` 는 검증한 저장 대상 업체 집합을 보관한다. 같은 낙찰 원본 hash 에 대해 정규 테이블의 업체 집

합이 정확히 일치해야 queue 를 완료한다. 기존 상태의 NULL 집합은 미검증이며 재조회 없이 일괄 완료로 바꾸지 않는다. 전체 조회는 기존 timeout·호출 속도 제한을 따르므로 참가자가 많은 공고는 제한 시간 내에 완료하지 못해 재시도할 수 있다.

응답 불완전은 최대 3 회 시도 후 `partial_completed`, 조회 실패는 최대 5 회 시도 후 `permanent_failure` 로 종결한다. 두 상태는 Runtime 에서 `failed` 이며 전체 성공으로 합산하지 않는다. 원인을 해결한 뒤 운영자가 재처리 여부를 판단한다. 처리 제한은 `087_bound_bid_opening_retries.sql` 과 해당 repository 가 소유한다.

성공한 낙찰·개찰 응답은 정규화 후 보존하지 않는다. 수집 계약의 `retain_raw_records: false` 에 따라 정규 원천 hash·실행 이력·검증한 업체 집합으로 추적하며 Raw 건수 대신 정규 처리 건수와 보강 완료 상태를 확인한다.

구현을 수정할 때

입찰공고 Task 는 `ingestion_tasks.py` 와 `enrichment_tasks.py` 가 수집·API 보강을 담당한다. 수집 날짜와 Backfill 경계는 `public_procurement/calendar.py` 의 한국시간 날짜를 사용한다. 감사시각은 UTC 로 저장한다. 계약 집계 갱신은 요청 버전별로 완료 처리하므로 갱신 중 들어온 새 요청은 다음 실행까지 큐에 남는다. 같은 분야의 동시 갱신은 DB advisory lock 으로 제한한다.

실행·수동 재시도는 Prefect [§33], 의미 기준은 Platform 조달 집계 [§10]를 참고한다. 과거 데이터 결손의 시점별 점검은 수집 점검 기록 [§54]에 보존한다.

32. openmetadata.md — OpenMetadata 메타데이터 수집

원문 파일: docs/pipelines/openmetadata.md

Teoria Data DB 의 schema metadata 를 OpenMetadata 에 동기화하는 작업이다. Prefect 가 일정·재시도·timeout 을 관리하고, 공식 ingestion image 기반의 일회성 컨테이너가 수집과 OpenMetadata REST 쓰기를 수행한다. Server 설치는 OpenMetadata 배포 [§ 39]를 먼저 완료한다.

	구성과 수집 범위

위치

역할

`pipelines/prefect.metadata.yaml`

배포 이름·이미지·network·timeout·스케줄

`pipelines/src/teoria_pipelines/metadata/openmetadata.py`

Flow 와 Job 실행 Task

`deploy/compose/openmetadata/Dockerfile`

공식 ingestion image 에 설정을 넣는 이미지

`deploy/compose/openmetadata/ingestion/teoria-postgres.yaml.example`

PostgreSQL connector 와 REST sink 설정

현재 설정은 `DatabaseMetadata` 로 `public_procurement` 의 테이블·view·설명을 수집한다. DataOps 원본·checkpoint·재시도 상태를 담는 `ingestion` schema 는 제외한다. Profiler, 샘플, usage 와 query lineage 는 이 작업에 포함되지 않으며 별도 workflow 가 필요하다. `pg_stat_statements` 는 schema metadata 수집의 필수 조건이 아니다.

일반 수집 Worker 에는 `openmetadata-ingestion` 을 설치하지 않는다. 전용 `prefect-metadata-worker` 는 Docker SDK 로 컨테이너를 실행하고 종료 후 제거한다. Job 에는 read-only Data DB credential 과 OpenMetadata 쓰기 bot token 만 전달한다.

이미지와 인증정보 준비

Server 와 동일한 `OPENMETADATA_VERSION` 으로 이미지를 빌드한다. 현재 저장소 기본값은 2.0.5 다. 기존 `.env` 값은 기본값보다 우선하므로 배포 전에 실제 값을 확인한다.

```
docker compose --env-file deploy/compose/.env -f deploy/compose/compose.yaml \
  --profile metadata --profile metadata-ingestion build openmetadata-ingestion-job
```

`TEORIA_METADATA_DB_PASSWORD` 와 `OPENMETADATA_INGESTION_BOT_TOKEN` 을 배포 환경에 설정한다. Prefect deployment 의 `image`, Flow 기본 이미지와 Worker 의 `TEORIA_OPENMETADATA_INGESTION_ALLOWED_IMAGE` 가 실제 이미지에 정확히 일치해야 한다. Worker 는 allowlist 가 다른 이미지를 실행하지 않는다.

한 번 실행해 확인하기

```
docker compose --env-file deploy/compose/.env -f deploy/compose/compose.yaml \
  --profile metadata --profile metadata-ingestion \
  run --rm openmetadata-ingestion-job
```

완료 후 OpenMetadata 에서 `teoria_postgresql` 서비스의 `public_procurement` schema 와 테이블을 확인한다. 재수집 전후 사용자 설명·Glossary 태그·소유자 보존도 확인한다. 샘플·프로파일이 없는 것은 이 workflow 의 실패가 아니다.

Prefect 로 실행하기

```
docker compose --env-file deploy/compose/.env -f deploy/compose/compose.yaml \
  --profile metadata --profile metadata-orchestration \
  up -d --build prefect-metadata-worker

docker compose --env-file deploy/compose/.env -f deploy/compose/compose.yaml \
  --profile metadata --profile metadata-orchestration \
  exec prefect-metadata-worker prefect deployment run \
  'OpenMetadata PostgreSQL 메타데이터 동기화/openmetadata-postgres-metadata-sync' --watch
```

Work pool 생성·Deployment 등록 서비스도 같은 metadata profile 의 코드와 이미지를 사용한다. 예약 시각·활성 상태는 배포 목록 [§28], 운영 등록 상태와 실패 Task 는 Prefect UI 에서 확인한다. OpenMetadata UI 에서 ingestion 을 생성·예약·재시도하지 않는다.

Task 는 실패 시 두 번 재시도하며 컨테이너 timeout 과 종료 코드를 확인한다. 실패하면 image allowlist, Docker network, Server 상태, read-only DB 접속과 bot 권한 순으로 확인한다. Job 로그에는 인증정보나 민감한 샘플을 남기지 않는다.

Compose 의 전용 Worker 는 host Docker socket 을 사용하므로 일반 수집 Worker 와 격리한다. K3s 는 socket mount 없이 전용 Metadata Worker 가 namespaced Kubernetes Job 을 생성한다. 실행 backend·Secret 과 최초 bot 설정은 K3s 배포 [§38]를 따른다. 실제 EC2 실행 검증은 서버 설정 후 수행한다.

33. prefect.md — Prefect 실행과 장애 대응

원문 파일: docs/pipelines/prefect.md

Prefect 는 수집 Flow 와 OpenMetadata Job 의 스케줄·재시도·실행 상태를 관리한다. 수집 계약과 정규화·저장 로직은 Pipelines 가 소유한다. 이 문서는 실행을 시작하고 실패 지점을 찾아 재개하는 절차를 설명한다.

Flow·Task 가 낯설다면 간단한 PostgreSQL ETL 예제 [§34]부터 따라 해 본다. 이 문서는 등록된 Deployment 의 실행과 장애 대응을 설명한다.

동일한 Flow 를 Compose 에서는 Process Worker 로, K3s 에서는 실행별 Kubernetes Job 으로 실행한다. K3s 의 work pool 변환·작업별 자원·전체 예산과 동시 실행 상한은 K3s 배포 [§38]를 따른다.

시작 전 준비

Compose 배포 [§35]에 따라 `deploy/compose/.env` 를 준비한다. Connector 인증정보, Data DB writer URL 과 Prefect Basic Auth 계정이 필요하다. Compose 밖에서 실행할 때는 `TEORIA_ENV_FILE=deploy/compose/.env` 를 지정한다.

```
docker compose --env-file deploy/compose/.env \
  -f deploy/compose/compose.yaml up --build -d
```

	UI 는 `http://localhost:8081/prefect/`다. Compose 의 Prefect 4200 포트는 내부 통신용이며, init·배포 등록·Worker 가 같은 Basic Auth 계정으로 접속한다.

서비스

역할

`postgres`, `postgres-migrate`

Data DB 와 schema migration

`platform-postgres`

DB·role 을 분리한 Application·Prefect·OpenMetadata DB

`prefect-server`, `prefect-services`, `prefect-redis`

API·UI·스케줄과 메시징

`prefect-deploy`

공통 등록 명령으로 Work pool·Queue 정책·Deployment 등록

`prefect-worker`

일반 수집 실행

Metadata 전용 Worker 와 ingestion Job 은 OpenMetadata 수집 [§32]을 따른다.

실행할 Deployment 찾기

배포 목록 [§28]에서 Flow, 파라미터, pool 과 예약 상태를 확인한다. 목록은 저장소 YAML 에서 생성한 선언값이다. 운영 UI 의 등록값·활성 상태·최근 실행과 대조해야 실제 운영 상태를 알 수 있다.

`pipelines/prefect.yaml` 은 수집, `prefect.metadata.yaml` 은 Metadata Job 을 정의한다. Deployment 의 `pipeline:<definition_id>`와 `domain:<domain_id>` tag 로 수집 계약과 소유 Domain 을 찾는다. YAML 변경 후에는 재등록하고 목록도 재생성한다.

```
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs >
docs/pipelines/deployments.md
uv run --locked --package teoria-pipelines teoria-pipelines deployment-docs --check
docs/pipelines/deployments.md
```

수동 실행과 진행 확인

			Compose 와 K3s 는 `teoria-pipelines register` 를 공통 진입점으로 사용한다. Compose 는 `--backend process`, K3s 는 `--backend kubernetes` 를 지정한다. `--kind ingestion|metadata|all` 로 등록 범위를 선택한다. 배포별 entrypoint·스케줄·입력은 원본 YAML 을 공유하며 환경별 사본만 생성한다. 실행 정책 구현은 `pipelines/src/teoria_pipelines/deployment.py` 가 소유한다.

Queue

대상

우선순위

기본 동시 실행 상한

`latest`

최신 수집·주기적 API 보강·사전 동기화

1

2

`backfill`

과거 수집·보정·Backfill 결손 재처리

10

1

전체 수집 상한은 `TEORIA_PIPELINE_CONCURRENCY=2`, Backfill·보정 상한은 `TEORIA_PIPELINE_BACKFILL_CONCURRENCY=1` 이다. 두 배포에서 같은 정책을 적용하며 Metadata 는 별도 pool 의 동시 실행 1 개로 관리한다. 우선순위는 다음 제출 순서에 적용되고 실행 중인 작업을 선점하지 않는다. 같은 Deployment 의 `CANCEL_NEW` 중복 방지 정책은 유지한다. 동시 실행 상한 변경 후

에는 등록 명령을 다시 실행한다. 한 Prefect Server 에 두 환경을 같은 Deployment 이름으로 중복 등록하면 서로 갱신하므로 운영 환경별 Prefect Server 를 사용한다.

예를 들어 아래 명령은 지정한 기간의 계약 Backfill 을 요청한다. 같은 진행 위치를 재개하려면 같은 checkpoint 를 사용하고, 독립 범위에는 새 `checkpoint_id` 를 사용한다.

```
docker compose --env-file deploy/compose/.env \
  -f deploy/compose/compose.yaml run --rm prefect-deploy \
  prefect deployment run '나라장터 계약정보 Backfill/pps-contract-backfill' \
  --param checkpoint_id=pps_contract_backfill_2021_2026 \
  --param start_date=2021-01-01 \
  --param end_date=2026-09-13 \
  --param batch_days=1
```

UI 에서 일별 하위 Flow 와 API 조회→Raw 저장→정규화→Upsert→완료 기록 Task 를 확인한다. 재실행은 완료 기록이 있는 Operation 을 건너뛰고 모든 Operation 이 완료돼야 날짜 checkpoint 를 이동한다. Backfill 종료일은 오늘 이전 날짜를 사용한다. 수집·보강·과거 결손의 구체적인 정책은 공공조달 수집 [§31]을 따른다.

실패 지점 찾기와 재개

Flow 에서 실패 Task 를 확인한다. Provider 요청, Raw 저장, 정규 적재 중 어느 단계인지 구분한다.

인증·응답 오류·속도 제한·DB 연결를 해당 단계의 로그와 실행 기록으로 확인한다.

성공한 Raw·정규 결과와 checkpoint 를 확인한 뒤 같은 입력으로 재개한다. 진행 위치를 임의로 앞당기지 않는다.

완료 후 원본 관찰과 정규 적재를 확인한다. 최근 성공 시각만으로 전체 기간의 수집 완전성을 판단하지 않는다.

API·DB 쓰기는 Task 단위로 재시도하고 Raw·정규 적재는 중복 실행을 고려한다. 실패한 단위는 checkpoint 를 진행시키지 않는다. 일부 성공을 보존하는 chunk 와 결손 큐의 처리는 Domain 별 정책을 따른다. 재조회 중첩 기간도 Flow·배포 파라미터별로 확인한다.

Process Worker 재시작은 다른 Worker 나 Kubernetes Job 의 종료 증거가 아니다. Compose 시작 시 Data DB 의 `running` 기록 중 Prefect 가 종료 상태로 확인한 실행만 미완료로 정리하며, Prefect 상태·부모 Flow·checkpoint 를 강제로 변경하지 않는다. Prefect 가 Completed 여도 DB 완료 기록이 없으면 적재 성공으로 간주하지 않는다. 아직 Running 이거나 Prefect ID 연결을 확인할 수 없는 기록은 그대로 두고 실제 실행 프로세스·Job 종료를 확인한 뒤 운영자가 Prefect 에서 상태를 정리한다. K3s Dispatcher 는 시작 시 실행 중인 Job 을 일괄 취소하지 않는다.

```
docker compose -f deploy/compose/compose.yaml logs -f prefect-worker
docker compose -f deploy/compose/compose.yaml down
```

`down -v` 는 수집 데이터와 실행 이력을 삭제하므로 데이터를 폐기할 때만 사용한다. Worker 에는 `.env` 전체가 아니라 Compose 가 선언한 변수만 전달한다.

수집이 실제로 진행되는지 확인하기

Worker 의 `ONLINE` 과 Flow 의 `Completed` 만으로 데이터가 모두 채워졌다고 판단하지 않는다. 다음 항목을 함께 확인한다.

배포의 예약 시각·활성 상태, 최근 성공·실패·재시도와 10 분 이상 늦은 실행을 확인한다.

`ingestion.pipeline_runs` 의 완료 시각과 기능별 처리 건수를 확인한다. 재조회·갱신이 포함되므로 처리 건수는 신규 고유 레코드 수와 다를 수 있다.

`ingestion.pipeline_checkpoints` 를 조회해 성공한 범위까지만 진행했는지 확인한다. 역방향 Backfill 은 날짜가 과거로 이동하며, 시작 날짜에 도달한 예약은 처리할 구간이 없을 수 있다.

개찰 보강의 `pending·processing·retry_wait`, `partial_completed·permanent_failure` 와 계약 `pipeline_backfill_gaps` 의 미해결 구간을 따로 확인한다.

현재 예약 간격과 비교한다. 4 시간마다 실행하는 계약·낙찰의 마지막 성공이 시간 단위 공고보다 오래됐다는 이유만으로 장애로 판정하지 않는다.

`CANCEL_NEW`에 따른 중복 예약 취소는 실행 중인 기존 수집을 취소한 것과 구분한다. 부분 응답이나 조회 실패를 보강 큐에 남긴 Flow 는 `Completed`일 수 있으므로, 실행 실패 0 건을 전체 데이터 완전성의 증거로 사용하지 않는다. 원본 비보존 경로의 `raw_record_count=0` 해석은 Data DB [§27]를 따른다. 시간대·건수·남은 결손을 포함한 최근 확인은 2026-10-16 배포·수집 검증 [§48]에 있다.

34. writing_pipelines.md — PostgreSQL 변경분을 DW 에 반영하는 ETL 만들기

원문 파일: docs/pipelines/writing_pipelines.md

매일 저녁, 원천 PostgreSQL 에서 **전날 수정된 주문**을 읽어 데이터웨어하우스(DW)에 반영해 보자. 새 주문뿐 아니라 과거 주문의 금액 수정, 취소, 삭제 표시도 반영한다. 원천과 DW 는 이미 있다고 가정하며 DB 나 테이블을 만드는 코드는 포함하지 않는다.

예제 파일은 flows.py [pipelines/examples/db_sales/flows.py]와 prefect.yaml [pipelines/examples/db_sales/prefect.yaml] 두 개다.

```
처리 날짜 확정 → Extract          → Transform    → Validate     → Load
                 updated_at 조회   상태 표기 통일  품질 검사      DW 주문 갱신
                                                    실패하면 중단
```

ETL 은 Extract(추출), Transform(변환), Load(적재)의 약자다. 여기에 **적재 전 검증**을 넣는다. Python 에는 처리 순서·SQL·검증 규칙을, `prefect.yaml` 에는 실행 입력값과 배포 설정을 둔다.

1. 기존 테이블과 삭제 표시 이해하기

	원천은 `public.orders`, 대상은 PostgreSQL 기반 DW 의 `warehouse.orders` 를 사용한다. 두 테이블에는 다음 컬럼이 이미 있어야 한다. 실제 이름이 다르면 예제의 SQL 과 검증 규칙을 바꾼다.

컬럼

가정하는 타입·역할

`order_id`

비어 있지 않은 주문 ID. DW 의 고유 키

`order_date`

주문한 날짜, `date`. 과거 날짜여도 처리 대상이 될 수 있음

`amount`

비어 있지 않은 0 이상의 숫자

`status`

`pending`, `paid`, `cancelled` 중 하나

`is_deleted`

비어 있지 않은 `boolean`. 정상은 `false`, 삭제 표시는 `true`

`updated_at`

비어 있지 않은 최종 변경 시각, `timestamptz`

원천은 행을 물리적으로 지우는 대신 `is_deleted=true` 로 바꾼다. **추가·수정·취소·삭제 표시·복구 때마다 `updated_at` 도 이전 값보다 큰 시각으로 갱신**해야 한다. 삭제 표시를 해도 나머지 컬럼은 유지된다고 가정한다. ETL 이 읽기 전에 행을 제거하면 이 예제로는 삭제를 알 수 없다.

			예를 들어 10 월 15 일에 다음 변경이 있었다면 세 주문을 모두 가져온다.

주문

원래 주문일

10 월 15 일 변경

DW 에 반영하는 내용

A001

10 월 15 일

새 결제 주문

새 행 추가

A002

9 월 1 일

주문 취소

`status=cancelled` 로 갱신

A003

8 월 10 일

삭제 표시

`is_deleted=true` 로 갱신

DW 에서도 삭제된 행을 남겨 상태를 확인할 수 있게 한다. 결제 매출을 조회할 때 `is_deleted=false AND status='paid'` 조건으로 제외한다. 이 예제는 주문의 최신 상태를 갱신하며, 모든 변경 이력을 별도 행으로 보관하는 모델은 아니다. 새 DW 의 최초 전체 적재도 별도 단계다.

2. 전날의 조회 범위 고정하기

한국 시간 매일 **22:00** 에 실행한다. 10 월 16 일에 예약된 Run 은 다음 범위를 처리한다.

```
business_date = 2026-10-15
시작: 2026-10-15 00:00:00+09:00  — 포함
종료: 2026-10-16 00:00:00+09:00  — 제외
SELECT order_id, order_date, amount, status, is_deleted, updated_at
FROM public.orders
WHERE updated_at >= %(window_start)s
  AND updated_at <  %(window_end)s
ORDER BY updated_at, order_id;
```

종료 시각을 제외하면 정확히 자정에 수정된 행은 다음 날짜에 포함된다. `updated_at` 을 날짜로 변환하지 않고 시각 범위로 비교하며, Python 에서 시간대가 있는 값을 전달한다. 조회량이 많으면 원천의 `updated_at` 인덱스와 쿼리 실행 계획을 확인한다.

날짜를 생략하면 실제 시작 시각 대신 **Prefect 예약 시각의 한국 날짜 기준 전날**을 사용한다. Worker 가 늦게 실행돼도 원래 처리하려던 날짜를 유지하기 위해서다. 과거 날짜를 새 Run 으로 재처리할 때는 `business_date` 를 반드시 직접 지정한다.

3. 실행 환경 연결하기

uv, 기존 PostgreSQL 원천·DW, 접근 가능한 Prefect Server 또는 Prefect Cloud 가 필요하다. 이번 예제는 **같은 머신의 저장소를 읽는 Process Worker** 로 실행한다. Worker 는 Prefect 에 등록된 작업을 받아 Python 프로세스로 실행하는 프로그램이다.

저장소 루트에서 다음 환경변수를 설정한다. 아래 주소와 계정은 자신의 접속 정보로 바꾼다. Secret 관리 도구로 값을 주입해도 된다.

```
export PREFECT_API_URL='https://your-prefect-host/api'
export ETL_SOURCE_DATABASE_URL='postgresql://reader:password@source-host:5432/source_db'
export ETL_TARGET_DATABASE_URL='postgresql://writer:password@warehouse-host:5432/warehouse_db'
export ETL_EXAMPLE_DIRECTORY="$PWD/pipelines/examples/db_sales"

uv sync --locked --package teoria-pipelines --group orchestration --group etl-example
```

원천 계정에는 조회 권한, DW 계정에는 조회·추가·갱신 권한이 필요하다. Prefect Cloud 는 Workspace 의 API URL 과 `PREFECT_API_KEY`, Basic 인증을 사용하는 Server 는 `PREFECT_API_AUTH_STRING` 도 설정한다. 이미 구성된 접속 설정이 있으면 그대로 쓴다. DB 비밀번호는 YAML 의 `parameters` 에 넣지 않는다. 이 입력값은 Prefect 에 저장된다.

Great Expectations 와 pandas 는 `etl-example` 의존성 그룹에만 추가되어 있다. 기본 운영 Worker 이미지는 이 그룹과 예제 파일을 포함하지 않는다. 이 문서의 Worker 는 아래 명령으로 따로 실행한다.

4. Flow 의 실행 순서 읽기

Flow 는 작업 전체, Task 는 UI 에서 실행 결과와 실패 지점을 확인할 수 있는 한 단계다. `orders_etl` 의 핵심 순서는 다음과 같다.

```
window_start, window_end = resolve_window(business_date)
orders = await extract_orders.with_options(**db_policy)(window_start, window_end)
rows = transform_orders(orders)
if not rows:
    return 0
validate_orders(rows)
await load_orders.with_options(**db_policy)(rows)
```

DB 를 읽고 쓰는 Task 와 Flow 는 비동기 함수이며 `await` 로 완료를 기다린다. 변환·검증 Task 는 일반 함수로 유지한다.

`extract_orders` 는 변경 시각으로 조회한다. 결제 상태나 삭제 여부로 원천을 필터링하지 않는다. `transform_orders` 는 `" PAID "` 를 `"paid"` 로 바꾸는 등 상태 표기를 통일한다. 취소·삭제 행도 그대로 검증과 적재에 전달한다. 변경 행이 0 건이면 저장 없이 성공으로 종료한다. 데이터 전체를 메모리에 읽는 작은 예제이므로 대량 데이터는 배치 단위로 나누어 처리한다.

Load 전에 Great Expectations 로 검증하기

	Great Expectations(GX)는 데이터가 정해 둔 조건을 만족하는지 검사하는 라이브러리다. 변환 결과를 pandas DataFrame 으로 바꿔 다음 조건을 확인한다.

검사

막으려는 문제

여섯 컬럼이 모두 비어 있지 않음

식별자·업무 값·변경 시각 누락

주문 ID 가 중복되지 않음

한 배치에 같은 주문이 여러 번 등장

금액이 0 이상

음수 금액 적재

상태가 `pending`, `paid`, `cancelled` 중 하나

알 수 없는 상태 적재

삭제 표시가 boolean

`"true"` 같은 문자열을 삭제 값으로 오인

검사 한 개는 다음처럼 작성한다.

```
check = gx.expectations.ExpectColumnValuesToBeBetween(column="amount", min_value=0)
result = batch.validate(check, result_format="BOOLEAN_ONLY")
if not result.success:
    raise ValueError("적재 중단: 금액 검사 실패")
```

GX 가 실패 결과를 반환하는 것만으로 Flow 가 중단되지는 않는다. **예외를 발생시켜야** 뒤의 `load_orders` 가 호출되지 않는다. 예제에서는 실패한 규칙과 컬럼 이름만 오류에 남기고, 검사 결과의 원본 행을 출력하지 않는다. 삭제 표시 행도 같은 검사를 받는다.

DW 에 최신 상태 반영하기

`load_orders` 는 주문 ID 가 없으면 추가하고, 있으면 최신 값으로 갱신한다. 취소 상태와 삭제 표시도 함께 저장한다. 핵심 SQL 은 다음과 같다.

```
INSERT INTO warehouse.orders AS target (...)
VALUES (...)
ON CONFLICT (order_id) DO UPDATE SET
    -- 날짜·금액·상태·삭제 표시·변경 시각 갱신
    ...
WHERE target.updated_at < EXCLUDED.updated_at;
```

위 코드는 구조 설명용이며 실행할 전체 SQL 은 `flows.py` 에 있다. DW 에 더 최신 값이 있으면 과거 데이터로 덮어쓰지 않는다. 같은 버전을 재실행하면 저장을 생략한다. 전체 배치를 한 트랜잭션으로 반영하므로 중간에 SQL 이 실패하면 rollback 된다. 완료 로그의 건수는 입력 건수이며, 실제 추가·갱신된 행 수와 다를 수 있다.

5. prefect.yaml 에서 실행 정책 정하기

예제 YAML [pipelines/examples/db_sales/prefect.yaml]의 주요 부분이다.

```
parameters:
  business_date: null
  db_retries: 2
  db_retry_delay_seconds: 5
  db_task_timeout_seconds: 60
concurrency_limit:
  limit: 1
  collision_strategy: ENQUEUE
schedules:
  - cron: '0 22 * * *'
    timezone: Asia/Seoul
    active: false
```

	`parameters` 의 재시도·timeout 항목은 **이 예제 Flow 가 정의한 입력값**이다. Python 에서

	`.with_options(...)`로 Extract와 Load Task에 적용한다. 배포 항목 바로 아래에 `retries: 2`를 추가한다고 Task 재시도가 설정되는 것은 아니다.

정책

예제 값과 적용 위치

DB 작업 재시도

`db_retries: 2`: 최초 시도 후 최대 2회 추가 시도

재시도 간격

`db_retry_delay_seconds: 5`: 재시도 전 5초 대기

DB Task 제한 시간

`db_task_timeout_seconds: 60`: 시도 한 번에 60초

날짜 확정·검증·변환 재시도

Python의 `@task(retries=0)`

전체 Flow 재시도·제한 시간

Python의 `@flow(retries=0, timeout_seconds=300)`

같은 배포의 동시 실행

한 번에 하나, 추가 실행은 대기

정기 실행

한국 시간 매일 22:00, 처음에는 비활성화

DB Task는 native async 연결을 사용한다. Prefect timeout 외에도 DB 자체의 실행 제한을 둔다. 코드에도 연결 제한 10초, SQL 실행 제한 30초, 적재 잠금 대기 제한 5초를 둔다. 실제 쿼리 시간에 맞춰 조정한다.

6. 배포하고 한 번 실행하기

먼저 Work pool을 만든다. Work pool은 어떤 Worker가 작업을 받을지 정하는 실행 대기 공간이다. 같은 이름의 pool이 이미 있다면 생성 명령은 생략한다.

```
uv run --locked --package teoria-pipelines --group orchestration --group etl-example \
  prefect work-pool create --type process example-etl

cd pipelines/examples/db_sales
uv run --locked --package teoria-pipelines --group orchestration --group etl-example \
  prefect deploy --all --prefect-file prefect.yaml
cd ../../..
```

이 명령은 Flow 입력값·스케줄·코드 위치를 Prefect에 등록한다. 코드를 서버로 업로드하지는 않는다. YAML의 `pull`은 Worker가 실행 전에 `ETL_EXAMPLE_DIRECTORY`로 이동하도록 한다. 배포 시 이 값이 실제 절대 경로로 들어가므로 동일한 머신과 저장소 경로를 사용한다.

같은 터미널에서 Worker를 시작한다. 앞서 설정한 DB·Prefect 환경변수를 이 Worker가 상속한다.

```
uv run --locked --package teoria-pipelines --group orchestration --group etl-example \
  prefect worker start --pool example-etl
```

Worker를 켜 둔 채 Prefect UI의 Deployments → `orders-daily` → Run에서 `business_date`에 변경 데이터가 있는 날짜(예: `2026-10-15`)를 입력한다. API 접속 설정을 한 별도 터미널에서 실행해도 된다.

```
uv run --locked --package teoria-pipelines --group orchestration --group etl-example \
  prefect deployment run 'orders-etl/orders-daily' --param business_date=2026-10-15
```

UI에서 처리 날짜 확정 → Extract → Transform → Validate → Load가 성공했는지 확인한다. DW에서는 알고 있는 주문 ID로 상태·삭제 표시·변경 시각이 반영되었는지 확인한다. `order_date`는 원래 주문한 날짜이므로 이번 배치 날짜와 다를 수 있다.

```
SELECT order_id, order_date, amount, status, is_deleted, updated_at
FROM warehouse.orders
```

```
WHERE order_id IN ('A001', 'A002', 'A003');

-- 분석에서 사용하는 정상 결제 주문
SELECT order_id, amount
FROM warehouse.orders
WHERE is_deleted = false AND status = 'paid';
```

Validate 가 실패했다면 Load 실행 기록이 없어야 한다. 원본 값이나 변환·검증 규칙을 확인하고 고친 뒤 **같은 `business_date`**를 지정해 다시 실행한다.

7. 체크포인트와 재처리 범위

이 예제는 날짜를 지정하는 최소 배치이므로 별도 체크포인트 테이블을 만들지 않는다. Prefect 에서 Run 별 조회 범위 로그와 성공·실패를 확인한다. **실패한 날짜와 실행되지 않은 날짜를 자동으로 찾아 보충하지는 않는다.** 빠진 날짜는 직접 지정해 실행해야 한다. 자동 복구가 필요하면 날짜별 완료 기록이나 마지막 성공 위치를 관리하는 단계를 추가한다. 체크포인트는 검증과 최종 적재가 모두 성공한 뒤에만 갱신한다.

원천 테이블에는 현재 행만 있으므로, 이후 다시 수정된 행은 과거 날짜 범위를 재조회할 때 나오지 않을 수 있다. 원천의 `updated_at` 소급 변경이나 지연 커밋에도 별도 대응이 필요하다. 이 예제는 날짜별 변경 이력을 완전히 복원하는 방식이 아니다. 동일한 입력으로 재처리해야 하면 추출 결과를 보관하고, 모든 변경을 놓치지 않아야 하면 CDC·변경 이력을 사용하는 방식으로 확장한다.

8. 수정한 내용을 반영하기

예를 들어 YAML 의 `db_retries` 를 `3` 으로 바꿨다면 6 절의 `prefect deploy` 를 다시 실행한다. 정기 실행은 결과를 확인한 뒤 `active: true` 로 바꾸고 다시 배포한다. 이전 예제의 `paid-orders` 배포가 남아 있다면 UI 에서 해당 배포의 스케줄을 중지하고 새 배포를 사용한다.

Python 을 바꿨다면 Worker 가 읽는 같은 경로의 파일을 갱신하고 `version` 도 올려 다시 배포한다. 실행 중인 Run 이 끝난 후 변경한다. Process Worker 는 다음 Run 에서 그 파일을 읽는다. 운영 배포의 이미지 빌드·코드 전달·Worker 갱신은 Prefect 운영 안내 [§33]를 따른다.

API 수집을 추가할 때 필요한 Connector 와 Definition, 실제 기능 코드 배치는 개발 안내 [§29]에서 이어서 확인한다. 이 DB 예제를 위해 API Connector 나 별도 업무 정책 YAML 을 만들 필요는 없다.

공식 참고: Prefect 배포 YAML [참고 1], 예약 시각 조회 [참고 2], GX DataFrame 검증 [참고 3], PostgreSQL UPSERT [참고 4].

외부 참고 주소

[1] https://docs.prefect.io/v3/how-to-guides/deployments/prefect-yaml

[2] https://docs.prefect.io/v3/api-ref/python/prefect-runtime-flow_run

[3] https://docs.greatexpectations.io/docs/core/introduction/try_gx/

[4] https://www.postgresql.org/docs/17/sql-insert.html

35. compose.md — Compose 배포와 운영

원문 파일: docs/deployment/compose.md

`deploy/compose/compose.yaml` 은 지원되는 로컬 개발·통합 배포의 단일 진입점이다. 환경변수는 환경 설정 [§36], 수집 실행은 Prefect 운영 [§33], 선택적 metadata 서비스는 OpenMetadata 배포 [§39]를 따른다. EC2 운영 배포는 K3s [§38]를 따른다.

Docker Compose

`deploy/compose/compose.yaml` 이 전체 로컬 Teoria 스택과 선택적 검증 작업의 단일 진입점이다.

```
deploy/
├── compose/           로컬 전체 스택 + 선택 profile
│   ├── compose.yaml
│   ├── .env.example
│   └── nginx/         공개 HTTP nginx 이미지와 설정
└── k3s/              EC2 단일 노드와 GitHub 자동 배포
```

Compose project name 은 `teoria` 다. 별도 `container_name` 없이 다음 이름이 자동 생성된다.

```
teoria-postgres-1
teoria-prefect-worker-1
teoria_default
teoria_data-db # 기존 데이터를 보존하기 위해 유지하는 물리 volume 이름
```

환경 분리는 `-p teoria-dev`, `-p teoria-staging` 처럼 project name 을 덮어쓴다. DB·Runtime artifact·OPA 를 포함한 모든 관리 볼륨은 project name 에 따라 분리된다. 기본 `teoria` 의 기존 `teoria_data-db`, `teoria_platform-db` 등은 그대로 사용한다. 기존 배포의 project name 을 바꾸면 새 빈 볼륨을 사용하므로 데이터 이관 없이 이름을 변경하지 않는다. 같은 호스트에서 환경을 함께 실행할 때 공개 HTTP 포트도 다르게 지정한다.

기본 스택

`deploy/compose/.env` 는 Compose 입력값의 기준이다. 컨테이너 내부 경로·주소와 운영 정책은 `compose.yaml` 이 정의한다. 호스트에서 직접 실행할 때의 주소·인증 설정은 환경 설정 [§36]을 따른다.

```
TEORIA_PREFECT_USERNAME=admin
TEORIA_PREFECT_PASSWORD=충분히-긴-비밀번호
TEORIA_RUNTIME_API_TOKEN=충분히-긴-임의-토큰
TEORIA_ADMIN_API_TOKEN=Runtime 과-다른-충분히-긴-임의-토큰
TEORIA_OPA_CONTROL_PLANE_TOKEN=Runtime 과 다른-충분히-긴-임의-토큰
TEORIA_OPA_BUNDLE_REVISION=배포-commit-또는-policy-release-id
TEORIA_LOCAL_RUNTIME_DB_PASSWORD=충분히-긴-로컬-비밀번호
# 비워 두면 Compose 내부 PostgreSQL 을 사용한다.
TEORIA_PIPELINE_DATA_DATABASE_URL=
TEORIA_RUNTIME_DATA_DATABASE_URL=
```

Prefect 계정, Admin·Runtime·OPA Control Plane 비밀값이 없으면 Compose 는 시작하지 않는다. Admin API 는 Compose 에서 항상 Bearer 인증을 사용한다. 외부 HTTP 요청은 nginx 의 단일 포트로만 받고, Admin UI/API, Runtime API, OPA 와 Policy Control Plane 은 Compose 내부 네트워크에만 노출한다.

Data DB 는 기본적으로 Compose 의 `postgres` 를 사용한다. 외부 PostgreSQL 을 사용할 때는 `.env` 에 writer 용 `TEORIA_PIPELINE_DATA_DATABASE_URL` 과 read-only Runtime 용 `TEORIA_RUNTIME_DATA_DATABASE_URL` 을 각각 설정한다. 두 URL 은 동일한 database 를 가리키되 DB role 은 분리해야 한다. 외부 DB 의 role·database 생성, TLS 와 방화벽은 외부 DB 운영 영역이며 Compose 의 `postgres-init` 이 외부 DB 계정을 생성하지 않는다. 아래 백업 스크립트는 실제 설정된 Data DB 를 논리 백업하며, 외부 DB 서비스의 백업·복구 정책을 대신하지 않는다.

Data DB 컨테이너는 병렬 쿼리의 동적 공유 메모리를 위해 `shm_size: 1gb` 를 사용한다. Docker 기본값인 64MB 가 부족하면 디스크 여유와 무관하게 `could not resize shared memory segment` 와 `No space left on device` 오류가 발생할 수 있다. 이 설정 변경은 기존 Data DB volume 을 유지한 채 `postgres` 컨테이너를 재생성해야 적용된다.

```
TEORIA_PIPELINE_DATA_DATABASE_URL=postgresql://teoria_pipeline:비밀번호@db.example.com:5432/teoria_data?
```

```
sslmode=require
TEORIA_RUNTIME_DATA_DATABASE_URL=postgresql://teoria_runtime:비밀번호@db.example.com:5432/teoria_data?
sslmode=require
docker compose --env-file deploy/compose/.env \
  -f deploy/compose/compose.yaml \
  up --build -d
```

기본 `up` 은 다음 순서로 실행한다.

```
PostgreSQL → migration ┐
                       ├→ prefect-worker
Prefect DB ─┬→ Server → work pool → prefect-deploy ┘
Redis ──────┴→ Background Services
```

OPA 는 `opa-bundle-build` 가 Rego 를 검사하고 RSA 서명 Bundle 을 생성한 뒤 시작한다. `policy-control-plane` 은 인증된 Bundle download, gzip decision log 와 status 수집만 제공한다. OPA healthcheck 는 `/health?bundles=true&plugins=true` 를 사용하므로 최초 Bundle 활성화나 management plugin 초기화가 실패하면 Runtime 과 Admin API 가 시작되지 않는다. signing private key, Bundle 과 OPA last-known-good state 는 각각 이름 있는 volume 에 보존된다.

수집 Worker 는 기본 CPU 4 · 메모리 8GiB 안에서 Flow 를 최대 2 개 실행한다. `TEORIA_PIPELINE_WORKER_CPUS`, `TEORIA_PIPELINE_WORKER_MEMORY` 로 컨테이너 전체 상한을 바꾸고, `TEORIA_PIPELINE_CONCURRENCY` 로 동시 실행 수를 조정한다. Process 실행이므로 Flow 별 CPU · 메모리 격리를 제공하지 않는다. 최신 수집 우선 · Backfill 1 개 정책은 K3s 와 공유하며 Prefect 운영 [§ 33]을 따른다. 등록 정책 변경 시 `prefect-deploy` 를 다시 실행하고 Worker 환경변수 변경 시 Worker 를 재생성한다.

실행 중인 수집을 보존하는 Worker 재배포는 이미지 빌드 → 수집 · Metadata work pool 일시정지 → 기존 실행 완료 대기 → 등록 서비스 성공 확인 → Worker 재생성 → 자원 상한 · 동작 확인 → 원래 실행 상태 복원 순서로 진행한다. pool 일시정지는 새 실행 제출만 막으며 기존 실행을 중단하지 않는다. 처음부터 일시정지돼 있던 pool 은 재배포 후에도 일시정지를 유지한다. `--no-deps` 로 대상 서비스만 갱신하면 이 변경과 관계없는 DB · API 컨테이너를 유지할 수 있다.

현재 DB I/O 는 native async 이며 Platform 과 Pipeline 의 연결 상한은 각각 `TEORIA_DATABASE_CONCURRENCY_LIMIT`, `TEORIA_PIPELINE_DATABASE_CONCURRENCY_LIMIT` 로 설정한다. 둘 다 기본값은 10 이며 event loop 별 · 접속 URL 별 제한이다. Prefect 의 Flow 동시 실행 제한과 다른 값이고 여러 프로세스의 연결 예산은 합산해서 관리한다.

API 재생성 직후에는 healthcheck 완료와 nginx 의 DNS 갱신을 기다린 뒤 공개 주소로 검증한다. 등록 서비스 exit code 0, 새 Worker heartbeat, 인증 차단, 실제 DB 조회와 Task 재시도 · checkpoint 저장을 확인하고 미대기 코루틴 경고가 없는지 로그를 검사한다. 이미지 교체만 할 때는 Runtime artifact 를 새로 발행하지 않고 기존 version 을 확인한다. 확인한 범위와 남은 결손은 2026-10-16 검증 기록 [§ 48]에 있다.

`prefect-redis` 는 Prefect 메시징 전용이고 `runtime-redis` 는 Runtime Capability 결과 캐시 전용이다. Runtime API 를 여러 replica 로 실행해도 Registry 버전과 Capability 입력이 같은 결과를 공유한다. Redis 조회 · 저장 장애는 Capability 실행을 중단시키지 않고 캐시 미사용으로 전환된다.

첨부파일 · 문서 요건 기능은 보류되어 AI Worker 와 Object Storage 설정을 기본 배포에서 제거했다. 과거 보관 객체와 인증 volume 은 자동 삭제하지 않는다.

	공개 주소는 다음과 같다.

대상

주소

Platform Admin UI

`http://localhost:8081/`

Admin API docs

`http://localhost:8081/admin-api/docs`

Runtime API docs

`http://localhost:8081/runtime-api/docs`

Prefect UI

`http://localhost:8081/prefect/`

nginx health

`http://localhost:8081/health`

공개 포트는 `TEORIA_HTTP_PORT` 로 변경할 수 있으며 기본값은 `8081` 이다. Teoria PostgreSQL 에 로컬 SQL client 로 접근할 때는 포트를 공개하지 않고 `docker compose exec postgres psql` 을 사용한다. MCP 는 STDIO 이므로 기본 백그라운드 서비스에 포함하지 않는다.

		선택 profile

Profile

서비스

실행 방식

`tools`

`platform-check`, `pipelines-check`

일회성 검증

`mcp`

`mcp`

STDIO 실행

```
docker compose -f deploy/compose/compose.yaml --profile tools run --build --rm platform-check
docker compose -f deploy/compose/compose.yaml --profile tools run --build --rm pipelines-check
docker compose -f deploy/compose/compose.yaml --profile mcp run --rm mcp
```

검증 서비스는 로컬 Registry 와 Reference 를 read-only 로 mount 한다.

```
docker compose \
  -f deploy/compose/compose.yaml \
  --profile tools run --rm platform-check
```

상태와 종료

```
docker compose -f deploy/compose/compose.yaml ps
docker compose -f deploy/compose/compose.yaml logs -f prefect-worker
docker compose -f deploy/compose/compose.yaml down
```

`down -v` 는 수집 데이터와 Prefect 이력을 삭제하므로 데이터를 폐기할 때만 사용한다. Worker 에는 `.env` 전체가 아니라 Compose 에 선언한 Pipeline 변수만 전달한다.

PostgreSQL 백업과 복원 확인

실행 중인 Compose 스택에서 다음 명령을 사용한다. Bash, Python 3, Docker Compose 가 필요하다.

```
bash deploy/compose/postgres/backup.sh
```

기본 환경 파일은 `deploy/compose/.env` 이며 `TEORIA_ENV_FILE` 로 변경한다. Compose 와 같은 환경변수 우선순위를 사용하고, `COMPOSE_PROJECT_NAME` 으로 대상을 선택한다. 백업 기본 경로는 `backups/<project>/<UTC timestamp>/` 다.

```
TEORIA_ENV_FILE=/path/to/staging.env \
```

```
COMPOSE_PROJECT_NAME=teoria-staging \
TEORIA_BACKUP_ROOT=/secure/backups \
  bash deploy/compose/postgres/backup.sh
```

`teoria_data.dump`: `postgres-migrate` 에 설정된 실제 Data DB URL 을 사용한다. 외부 DB 를 설정하면 내부 DB 대신 외부 DB 를 백업한다.

`teoria_app.dump`, `prefect.dump`, `openmetadata_db.dump`: 해당 project 의 `platform-postgres` 에서 백업한다.

`SHA256SUMS`, PostgreSQL 버전과 project 정보: 백업 파일 검증과 복원 환경 선택에 사용한다. 연결 URL·암호는 기록하지 않는다.

Data DB 클라이언트는 해당 project 의 실행 중인 `postgres` 컨테이너를 사용한다. 외부 서버로 접속할 네트워크·권한이 필요하고, 서버 버전에 대응하는 `pg_dump` 여야 한다. 외부 DB 용 파일 기반 TLS 인증을 쓴다면 클라이언트 컨테이너에서도 해당 파일에 접근할 수 있어야 한다.

진행 중이거나 실패한 백업은 `.incomplete` 디렉터리에 남는다. 모든 덤프의 archive 목록과 checksum 검사가 성공한 후에만 완료 이름으로 변경한다. 디렉터리·파일은 기본적으로 현재 사용자만 접근하며, 실패 상세는 해당 백업의 `*.stderr.log` 에서 확인한다.

`pg_restore --list` 와 checksum 성공은 복원 성공을 보장하지 않는다. 운영과 분리한 PostgreSQL 에서 덤프를 복원하고 중요 테이블을 조회해 확인한다. DB 초기화·role 생성은 복원 전에 수행하며, 별도 검증 DB 에서는 `pg_restore --no-owner --no-acl` 로 복원할 수 있다. Runtime artifact·OPA 키·객체 저장소·Codex 인증정보는 이 DB 백업에 포함하지 않는다.

36. configuration.md — 환경 설정과 인증정보

원문 파일: docs/deployment/configuration.md

```
cp deploy/compose/.env.example deploy/compose/.env
uv sync --locked --all-packages --all-groups
```

의존성은 사용하는 프로젝트의 `pyproject.toml` 에만 추가한다. 루트 `uv.lock` 은 개발·CI 재현성을 위해 공유한다.

	설정을 읽는 위치

대상

기준 코드·설정

Platform·Runtime·Admin

`platform/src/teoria/settings.py`

Pipelines

`pipelines/src/teoria_pipelines/settings.py`

DB 연결 상한

Platform `persistence/postgres.py`, Pipelines `persistence/connection.py` 의 환경변수

MCP

`mcp/src/teoria_mcp/settings.py`

Compose 서비스·포트·이미지·필수 인증

`deploy/compose/compose.yaml`, `.env.example`

EC2 K3s·GitHub CD

`deploy/k3s/`, `.github/workflows/deploy_k3s.yaml`

쉘 환경변수가 env 파일보다 우선한다. Compose 밖의 실행은 `TEORIA_ENV_FILE` 로 파일을 선택한다.

`TEORIA_ENVIRONMENT=production` 에서 파일을 명시하지 않으면 작업 디렉터리의 `.env` 를 자동으로 읽지 않는다. 아래는 주요 변수의 참조이며 전체 기본값은 위 코드가 기준이다.

프로세스 기본값과 Compose 주입값

		아래 표의 기본값은 Python 설정 모델 기준이다. Compose 전용 변수는 배포 정의의 기본값을 표시한다. `.env.example` 은 설정 예이며 실제 환경변수와 Compose 의 고정값·fallback 이 최종 실행값을 결정한다. 다음 차이를 먼저 확인한다.

설정

Python 기본값

현재 Compose 주입

`TEORIA_ENVIRONMENT`

`development`

Platform 서비스 `production`

`TEORIA_ADMIN_AUTH_MODE`

`disabled`

`bearer`; Admin token 필수

`TEORIA_POLICY_MODE`

`disabled`

Runtime·Admin 에 opa

`TEORIA_OPENMETADATA_ENABLED`

`false`

Admin·진단 Worker 에서 기본 `true`

`TEORIA_OPENMETADATA_BASE_URL`

`http://localhost:8585/api`

기본 `http://openmetadata-server:8585/api`

`TEORIA_OPENMETADATA_AUTH_TOKEN`

없음

명시 token 이 없으면 ingestion bot token fallback

`TEORIA_RUNTIME_ARTIFACT_STORE`

없음

Runtime·Admin 의 `/app/runtime_artifacts`

`TEORIA_CONTEXT_RUNTIME_API_URL`

`http://localhost:8000`

Admin 의 `http://runtime-api:8000`

`TEORIA_CONTEXT_RUNTIME_API_TOKEN`

없음

Runtime service token 주입

Data DB URL

Python 설정값 없음

외부 URL 또는 내부 PostgreSQL 의 역할별 URL

Compose 의 hostname 은 내부 network 용이다. 호스트에서 API 를 호출할 때는 공개 nginx 주소를 사용한다. 토큰과 암호를 포함한 설정 전체를 로그에 출력하지 않는다.

		공통·경로

변수

기본값

의미

`TEORIA_ENVIRONMENT`

`development`

개발·테스트·운영 구분: `development`, `test`, `production`

`TEORIA_ENV_FILE`

개발 시 작업 디렉터리 `.env`

로컬 프로세스가 읽을 환경 파일 경로

`TEORIA_REGISTRY_PATH`

`platform/registries`

Platform Registry

		Runtime·Context·정책

변수

기본값

의미

`TEORIA_SOURCE_TIMEOUT_SECONDS`

`15`

Runtime Source timeout

`TEORIA_SOURCE_MAX_ATTEMPTS`

`3`

Runtime Source 요청 횟수

`TEORIA_SOURCE_MAX_PAGES`

`100`

Capability 최대 페이지

`TEORIA_CAPABILITY_TIMEOUT_SECONDS`

`120`

Capability deadline

`TEORIA_RUNTIME_API_TOKEN`

없음

Runtime API Bearer token

`TEORIA_RUNTIME_ARTIFACT_PATH`

없음

단일 immutable Runtime bundle version 디렉터리

`TEORIA_RUNTIME_ARTIFACT_STORE`

없음

atomic `active.json` 을 가진 Runtime bundle store

`TEORIA_RUNTIME_API_ROOT_PATH`

빈 문자열

reverse proxy 가 Runtime API 앞에 붙이는 URL 경로

`TEORIA_RUNTIME_CACHE_BACKEND`

`memory`

Runtime 공유 캐시 구현: `memory`, `redis`, `disabled`

`TEORIA_RUNTIME_CACHE_URL`

`redis://localhost:6379/0`

Redis Runtime 캐시 접속 URL

`TEORIA_RUNTIME_CACHE_PREFIX`

`teoria:runtime`

Registry 버전과 Capability 키 앞에 붙는 캐시 namespace

`TEORIA_CONTEXT_RUNTIME_API_URL`

`http://localhost:8000`

Context Engine 이 승인된 Capability 를 실행할 Runtime API 내부 주소

`TEORIA_CONTEXT_RUNTIME_API_TOKEN`

없음

Context Engine 전용 Runtime API service credential; Compose 에서는 Runtime token 을 주입

`TEORIA_CONTEXT_RUNTIME_TIMEOUT_SECONDS`

`150`

Context Engine 의 Runtime API 호출 제한시간

`TEORIA_CONTEXT_MAX_PERIOD_YEARS`

`10`

Context 계획이 허용하는 최대 조회 기간

`TEORIA_CONTEXT_MAX_PAGES`

`100`

Context 실행당 Runtime 페이지 상한

`TEORIA_POLICY_MODE`

`disabled`

정책 결정 방식: 개발·테스트용 `disabled`, 운영용 `opa`

`TEORIA_OPA_URL`

`http://localhost:8181`

OPA REST API base URL

`TEORIA_OPA_DECISION_PATH`

`teoria/authz/decision`

OPA Data API 에서 평가할 decision document

`TEORIA_OPA_TIMEOUT_SECONDS`

`3`

OPA 정책 결정 제한시간. 실패·undefined decision 은 fail-closed

`TEORIA_OPA_CONTROL_PLANE_TOKEN`

없음

OPA Bundle·decision log·status 관리 API 전용 bearer credential

`TEORIA_OPA_BUNDLE_PATH`

`/var/lib/teoria/opa/teoria.tar.gz`

Policy Control Plane 이 배포할 서명 Bundle 경로

`TEORIA_OPA_BUNDLE_REVISION`

`development`

Bundle manifest 와 decision log 에 기록할 불변 revision

`TEORIA_REGISTRY_REQUIRE_PUBLISHED`

`false`

주입된 Registry catalog 에 published provenance 가 있는지 검사하는 개발·테스트 안전장치. 운영 Runtime 은 Artifact 설정이 필수

`TEORIA_RUNTIME_DATA_DATABASE_URL`

없음

Runtime 읽기 DB URL; 외부 DB 사용 시 read-only role URL

		Admin·Application DB

변수

기본값

의미

`TEORIA_ADMIN_API_ROOT_PATH`

빈 문자열

reverse proxy 가 Admin API 앞에 붙이는 URL 경로

`TEORIA_ADMIN_AUTH_MODE`

`disabled`

Admin API 인증 방식. 공유·운영 환경은 `bearer` 사용

`TEORIA_ADMIN_API_TOKEN`

없음

`bearer` mode 의 Admin API credential

`TEORIA_ADMIN_METADATA_PUBLIC_READ`

`false`

`true` 면 메타데이터 카탈로그·상세·통합 패널·계보·프로파일의 토큰 없는 조회 허용. 데이터 샘플과 변경 API 의 인증은 유지

`TEORIA_ADMIN_API_ACTOR`

`system:admin`

감사 기록에 남길 인증 principal ID

`TEORIA_ADMIN_API_ROLES`

`metadata_admin,metadata_reviewer,binding_reviewer,ontology_owner`

쉼표로 구분한 Admin 승인 역할

`TEORIA_ADMIN_DATA_DATABASE_URL`

없음

Admin API 가 입찰 데이터 조회에 사용하는 read-only Data DB URL

`TEORIA_DATABASE_CONCURRENCY_LIMIT`

`10`

Platform event loop 별·DB 별 활성 연결 상한. 여러 프로세스는 상한 합산

`TEORIA_APP_DATABASE_URL`

없음

Ontology v2, Binding, Intelligence 를 저장하는 Teoria Application DB URL

		Pipelines

변수

기본값

의미

`TEORIA_PIPELINE_PATH`

`pipelines`

Pipeline 루트

`TEORIA_PIPELINE_CONCURRENCY`

`2`

두 배포의 수집 pool 전체 동시 실행 상한. Compose Worker 자체 실행 상한에도 적용

`TEORIA_PIPELINE_BACKFILL_CONCURRENCY`

`1`

Backfill·보정 Queue 동시 실행 상한. 전체 상한 이하여야 함

`TEORIA_PIPELINE_WORKER_CPUS`

`4`

Compose 수집 Worker 전체 CPU 상한

TEORIA_PIPELINE_WORKER_MEMORY

8g

Compose 수집 Worker 전체 메모리 상한

TEORIA_PLATFORM_REGISTRY_PATH

platform/registries

Pipeline 통합 검증 대상

TEORIA_PIPELINE_SOURCE_TIMEOUT_SECONDS

30

Connector timeout

TEORIA_PIPELINE_SOURCE_MAX_ATTEMPTS

5

Connector 의 멱등 요청 최대 시도 횟수

TEORIA_PIPELINE_SOURCE_RETRY_BACKOFF_SECONDS

60

Connector 재시도 지수 backoff 의 최초 대기시간

TEORIA_PIPELINE_BID_NOTICE_ENRICHMENT_REQUESTS_PER_SECOND

1

공고별 면허·지역 보강 요청의 프로세스당 초당 최대 시작 횟수

TEORIA_PIPELINE_DATABASE_CONCURRENCY_LIMIT

10

Pipeline event loop 별·DB 별 활성 연결 상한. 여러 Worker·프로세스는 상한 합산

TEORIA_PIPELINE_DATA_DATABASE_URL

없음

Pipeline migration·적재용 writer DB URL

		MCP

변수

기본값

의미

TEORIA_MCP_RUNTIME_MODE

remote

MCP 실행 모드

TEORIA_MCP_RUNTIME_API_URL

없음

Remote Runtime URL

`TEORIA_MCP_RUNTIME_API_TOKEN`

없음

MCP 가 사용하는 Runtime API token

`TEORIA_MCP_RUNTIME_TIMEOUT_SECONDS`

`150`

Runtime API 호출 timeout

`TEORIA_MCP_RUNTIME_MAX_ATTEMPTS`

`3`

MCP capability discovery 의 일시 장애 포함 최대 시도 횟수

`TEORIA_MCP_RUNTIME_RETRY_BACKOFF_SECONDS`

`0.25`

MCP discovery 재시도의 최초 지수 backoff 초

		OpenMetadata·배포

변수

기본값

의미

`TEORIA_OPENMETADATA_ENABLED`

`false`

Admin API OpenMetadata integration 활성화 여부

`TEORIA_OPENMETADATA_BASE_URL`

`http://localhost:8585/api`

OpenMetadata REST API base URL

`TEORIA_OPENMETADATA_AUTH_TOKEN`

없음

Backend 전용 OpenMetadata bot JWT. 운영에서는 조회·검토에 필요한 최소권한 별도 토큰 권장

`TEORIA_OPENMETADATA_TIMEOUT_SECONDS`

`10`

OpenMetadata REST 요청 제한시간

`TEORIA_OPENMETADATA_VERIFY_SSL`

`true`

OpenMetadata TLS 인증서 검증 여부

`TEORIA_OPENMETADATA_DATABASE_SERVICE`

`teoria_postgresql`

기본 Teoria PostgreSQL Database Service 이름

`OPENMETADATA_VERSION`

`2.0.5`

Server 와 ephemeral ingestion image 에 공통 적용하는 OpenMetadata 버전

`OPENMETADATA_DB_PASSWORD`

로컬 개발값

OpenMetadata 전용 PostgreSQL 암호

`OPENMETADATA_INGESTION_BOT_TOKEN`

없음

일회성 ingestion Job 이 OpenMetadata REST API 에 쓰는 bot JWT

`TEORIA_METADATA_DB_PASSWORD`

로컬 개발값

OpenMetadata PostgreSQL connector 의 Teoria Data DB read-only 암호

`TEORIA_LOCAL_PLATFORM_DB_PASSWORD`

로컬 개발값

Compose 의 Control/Application PostgreSQL 관리 계정 암호

로컬 Compose 암호 변수는 `deploy/compose/.env.example` 을 따른다. 공유·운영 환경에서는 managed secret 으로 덮어쓴다. `TEORIA_ADMIN_API_TOKEN`, `TEORIA_RUNTIME_API_TOKEN`, `TEORIA_OPA_CONTROL_PLANE_TOKEN` 과 `TEORIA_LOCAL_RUNTIME_DB_PASSWORD` 에는 기본값이 없으며 Compose 실행 전에 반드시 설정한다. Compose 의 Admin API 는 `bearer` 로 고정한다. `TEORIA_ADMIN_AUTH_MODE=disabled` 는 호스트 개발용 설정이며 Compose 인증을 비활성화하지 않는다. `.env.example` 에는 Compose 가 실제로 받는 입력값만 둔다. 컨테이너 내부 OpenMetadata 주소는 `http://openmetadata-server:8585/api` 다.

호스트에서 직접 실행

호스트 프로세스는 각 프로젝트의 Settings 기본값과 명시한 환경 파일을 사용한다. Compose 입력 파일을 재사용하면 컨테이너 전용 주소를 호스트 주소로 덮어써야 한다. 로컬 Python 실행용 `TEORIA_ENVIRONMENT=development`, 경로와 timeout 기본값은 Settings 가 소유한다.

```
TEORIA_ENV_FILE=deploy/compose/.env \
TEORIA_ADMIN_AUTH_MODE=bearer \
TEORIA_OPENMETADATA_BASE_URL=http://localhost:8585/api \
  uv run --locked --package teoria-platform teoria-admin-api
```

이 예시는 주소·인증 차이를 보여준다. DB 를 사용하는 호스트 실행은 별도 접근 경로와 DB URL 도 설정해야 한다. Compose 내부 DB 포트를 기본 공개하지 않는다. 독립적인 호스트 개발 환경 파일을 쓰면 `TEORIA_ENV_FILE` 로 그 파일을 선택한다.

		자산 진단 Worker 설정

변수

기본값

의미

`TEORIA_ASSESSMENT_ENABLED`

`false`

진단 요청 활성화

`TEORIA_ASSESSMENT_MODEL_URL`

없음

gateway 의 전체 chat-completions endpoint

`TEORIA_ASSESSMENT_MODEL_NAME`

없음

사용할 모델 ID

`TEORIA_ASSESSMENT_MODEL_TOKEN`

없음

gateway 인증정보, Worker 에만 전달

`TEORIA_ASSESSMENT_MODEL_TIMEOUT_SECONDS`

`90`

모델 요청 제한시간

`TEORIA_ASSESSMENT_MODEL_MAX_TOKENS`

`6000`

출력 한도, 모델 문맥 크기에 맞게 조정

요청·응답 계약과 시작 방법은 자산 진단 [§ 14]을 따른다.

Secret 규칙

Source: `TEORIA_SOURCE_<SOURCE_ID>_API_KEY`

Connector: `TEORIA_CONNECTOR_<CONNECTOR_ID>_API_KEY`

YAML 에는 실제 값이 아닌 환경변수 이름만 기록한다.

로그, verification case 와 archive 에 키·개인정보·비공개 원본 응답을 남기지 않는다.

Pipeline writer 와 Runtime reader 의 DB 계정·권한을 분리하고 MCP 에는 DB 권한을 주지 않는다.

Runtime bearer token 은 service authentication 이며 최종 사용자 권한을 대신하지 않는다. 사용자 principal 과 role 은 신뢰된 Admin/MCP gateway 가 Runtime 으로 전달하고 OPA 가 Capability 와 Context 정책을 결정한다. 운영 gateway 에 OIDC 를 연결하기 전에는 외부 사용자가 identity 전달 header 를 직접 설정할 수 없도록 한다.

OPA signing private key 는 `opa-signing-keys` volume 또는 외부 secret manager 에만 보관하고 이미지, Git, 일반 로그에 포함하지 않는다. 공유 환경에서는 revision 을 소스 commit 또는 policy release ID 로 고정한다.

K3s 는 Compose `.env` 를 자동으로 읽지 않는다. 역할별 Secret 참조와 GitHub Environment 설정은 EC2 K3s 배포 [§ 38]를 따른다.

37. experimental_k3s.md — K3s 배포 안내 이전

원문 파일: docs/deployment/experimental_k3s.md

기존 빈 scaffold 는 실제 EC2 K3s 배포 구성 [§38]으로 대체했다. 코드는 `deploy/k3s/`, 자동 배포는 `.github/workflows/deploy_k3s.yaml` 에 있다.

38. k3s.md — EC2 단일 서버 K3s 배포

원문 파일: docs/deployment/k3s.md

GitHub `main` push 의 **CI 성공 후** 이미지를 GHCR 에 저장하고 EC2 에 SSH 로 접속하여 K3s 를 갱신한다. 현재 구현은 Linux **x86_64 EC2 1 대**를 대상으로 한다. EC2 주소·도메인과 GitHub 설정은 운영자가 준비한다. 저장소 구성 검증과 실제 EC2 배포 성공은 구분한다. 현재 환경에서 실제 EC2 연결과 배포는 아직 수행하지 않았다.

실행 구성과 기준 파일

	GitHub main → CI → GHCR의 commit SHA 이미지 → EC2 SSH → deploy.sh ↓ K3s 단일 server ├ Teoria API·UI·OPA ├ PostgreSQL·Redis ├ OpenMetadata·검색 └ Prefect·수집 Worker

목적

기준 파일

EC2 K3s 설치 설정

`deploy/k3s/server_config.yaml`

DB·캐시·검색·영속 볼륨

`deploy/k3s/infrastructure/`

OpenMetadata Server

`deploy/k3s/openmetadata/`

API·UI·진단 Worker

`deploy/k3s/application/platform.yaml`

OPA 와 정책 관리

`deploy/k3s/application/policy.yaml`

Prefect 와 수집 Worker

`deploy/k3s/application/prefect.yaml`

HTTPS 와 요청 경로

`deploy/k3s/application/ingress.yaml`

순서가 있는 초기화·migration·등록

`deploy/k3s/jobs/`

배포 순서·대기·실패 처리

`deploy/k3s/deploy.sh`

환경별 인증정보의 키 목록

`deploy/k3s/secrets.env.example`

이미지 빌드·EC2 배포

`.github/workflows/deploy_k3s.yaml`

Kustomize 는 `kubectl` 에 포함된 기능만 사용한다. Helmfile 이나 별도 GitOps 서버는 필요하지 않다. MLflow·Argo CD·Harbor·GPU Operator·MinIO·S3 는 이 초기 구성에 추가하지 않는다. 모델 진단은 외부 모델 gateway 를 호출하며 기본적으로 비활성화한다.

EC2 최초 준비

x86_64 Linux 와 cgroup v2, K3s 요구 커널·네트워크를 준비한다. DB·검색·Prefect 를 함께 실행하므로 K3s 자체의 최소 사양을 전체 서비스 사양으로 사용하지 않는다. YAML 의 CPU·메모리 요청과 실제 데이터 크기를 기준으로 EC2·EBS 용량을 정한다. 기본 server 설정은 시스템·Kubernetes 용도로 합계 CPU 1 개와 메모리 1Gi 를 예약한다. 워크로드에도 CPU·메모리 상한을 두며, 이 값은 실제 부하에 맞춰 조정한다.

데이터 디렉터리를 영속 EBS 에 두고, 운영 데이터 보존·snapshot 정책을 정한다. 같은 EBS 에 DB 와 검색을 함께 두면 I/O 를 공유한다. S3 는 PostgreSQL 의 데이터 디스크가 아니다.

`server_config.yaml` 을 `/etc/rancher/k3s/config.yaml` 에 설치한 뒤 버전을 고정해 K3s 를 설치한다.

```
sudo install -d -m 700 /etc/rancher/k3s
sudo install -m 600 deploy/k3s/server_config.yaml /etc/rancher/k3s/config.yaml
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION='v1.37.1+k3s1' sh -s - server
sudo k3s kubectl get nodes
```

SSH 배포 계정에는 이 클러스터를 관리할 kubeconfig 와 `kubectl`, Python 3, Bash, `flock`, `tar` 가 필요하다. 예를 들어 EC2 의 전용 배포 계정에서 다음을 준비한다. 이 계정은 해당 클러스터를 관리하므로 접속키의 접근 권한을 운영 범위로 제한한다.

```
mkdir -p "$HOME/.kube"
sudo install -m 600 -o "$(id -u)" -g "$(id -g)" \
  /etc/rancher/k3s/k3s.yaml "$HOME/.kube/config"
# EC2 호스트에서만 사용하므로 kubeconfig 의 127.0.0.1 주소를 유지한다.
export KUBECONFIG="$HOME/.kube/config"
kubectl get nodes
```

Security Group 은 사용자 HTTPS, 관리 SSH 와 노드 간 필요한 트래픽을 구분한다. GitHub 호스티드 runner 가 SSH 에 접근할 경로를 준비한다. 그 경로가 없다면 배포 작업은 연결 단계에서 실패한다. Kubernetes API 6443 은 GitHub runner 에 공개할 필요가 없다.

인증정보와 최초 데이터

비밀값을 Git 에 넣지 않는다. 예제 파일을 checkout 밖으로 복사해 `CHANGE_ME` 와 URL 을 채운다. 역할별 암호와 DSN 의 암호는 같아야 하며, DSN 암호는 URL encoding 한다.

```
kubectl create namespace teoria
cp deploy/k3s/secrets.env.example /secure/teoria-secrets.env
chmod 600 /secure/teoria-secrets.env
# 파일을 채운 다음 실행한다.
kubectl -n teoria create secret generic teoria-secrets \
  --from-env-file=/secure/teoria-secrets.env
kubectl -n teoria create secret docker-registry teoria-registry \
  --docker-server=ghcr.io --docker-username="$GHCR_USER" \
  --docker-password="$GHCR_READ_TOKEN"
kubectl -n teoria create secret tls teoria-tls \
  --cert=/secure/fullchain.pem --key=/secure/privkey.pem
```

`GHCR_READ_TOKEN` 은 해당 저장소의 private package 를 읽을 수 있는 credential 이다. 이미지 빌드에 쓰는 GitHub Actions 의 `GITHUB_TOKEN` 을 EC2 의 영구 pull credential 로 쓰지 않는다. 운영 도메인의 DNS 를 EC2 로 연결하고 그 도메인에 맞는 TLS 인증서를 준비한다. 선택적 Provider key 는 사용하는 연동에만 채운다.

빈 클러스터에서는 기존 DB 와 **이미 게시된 Runtime bundle** 을 한 번 이관해야 한다. `deploy.sh` 는 DB 초기화·migration 과 OpenMetadata 기동 이후 artifact 확인에서 멈추므로 이 시점에 DB 복원과 최초 OpenMetadata 계정·ingestion bot 설정을 진행할 수 있다. OpenMetadata 관리는 EC2 에서 `kubectl port-forward service/openmetadata-server 8585:8585`

후 SSH tunnel 을 통해 접속한다. bot JWT 를 얻으면 Secret 의 `OPENMETADATA_INGESTION_BOT_TOKEN`, `TEORIA_OPENMETADATA_AUTH_TOKEN` 을 갱신한다. JWT·연동 token 이 바뀌면 해당 Pod 를 재시작해 새 값을 읽게 한다. DB 암호 변경은 실제 PostgreSQL role 암호와 Secret 을 함께 갱신하는 별도 운영 절차로 수행한다.

DB 복원은 운영과 분리한 대상으로 검증한 뒤 수행한다. 배포 스크립트는 데이터를 자동 복원하거나 기존 DB 를 삭제하지 않는다. Runtime 은 authoring YAML 에서 자동 게시하지 않는다. `active.json` 과 그 파일이 지정한 version 디렉터리를 함께 가져온다. 복사 예시는 다음과 같다.

```
kubectl -n teoria run artifact-import --image=busybox:1.37 --restart=Never \
  --overrides='{"spec":{"containers":[{"name":"artifact-import","image":"busybox:1.37","command":
["sleep","3600"],"volumeMounts":[{"name":"artifacts","mountPath":"/artifacts"}]}],"volumes":
[{"name":"artifacts","persistentVolumeClaim":{"claimName":"runtime-artifacts"}}]}}'
kubectl -n teoria wait --for=condition=Ready pod/artifact-import --timeout=120s
# APPROVED_STORE 와 VERSION 은 이관할 게시 artifact 로 지정한다.
tar -C "$APPROVED_STORE" -cf - active.json "$VERSION" | \
  kubectl -n teoria exec -i artifact-import -- tar -C /artifacts -xf -
kubectl -n teoria exec artifact-import -- chown -R 10001:10001 /artifacts
kubectl -n teoria delete pod artifact-import
```

그 후 동일 커밋의 workflow 를 다시 실행한다. Runtime bundle checksum, 인증 API 와 OpenMetadata 연결 검증까지 성공해야 배포 성공으로 기록한다. Registry·Ontology 변경은 별도 검토·게시하고 새 bundle 을 명시적으로 이관해야 하며, 코드 push 가 이를 대신하지 않는다.

GitHub 설정과 자동 배포

		GitHub Environment `production` 을 만들고 다음을 설정한다. 자동 배포를 원하면 이 Environment 에 수동 reviewer 승인을 필수로 설정하지 않는다.

종류

이름

값

Variable

EC2_HOST

EC2 접속 주소

Variable

EC2_USER

전용 SSH 배포 사용자

Variable

EC2_SSH_PORT

기본 22

Variable

TEORIA_PUBLIC_HOST

TLS 인증서와 일치하는 도메인

Secret

EC2_SSH_PRIVATE_KEY

배포 계정 접속용 키

Secret

EC2_SSH_KNOWN_HOSTS

사전에 확인한 EC2 SSH 호스트 키

저장소의 Actions 권한에서 GHCR package 쓰기를 허용한다. 현재 브랜치는 `main` 이며, 변경하려면 workflow 의 branch 조건과 최신 commit 확인을 함께 바꾼다.

CI 에서 K3s API 서버의 server dry-run 으로 매니페스트를 검증한다.

`CI` 가 성공한 `main` push 만 이미지 빌드·배포한다. PR·fork 실행 결과는 배포하지 않는다.

이미지 9 종은 commit SHA tag 로 저장한다. `latest` 로 배포하지 않는다.

EC2 에는 Git 에 커밋된 `deploy/k3s/`만 전달한다. 비밀값은 GitHub 에서 전송하지 않고 사전 생성된 클러스터 Secret 을 사용하며 수집 namespace 에 필요한 키만 동기화한다.

GitHub concurrency 와 EC2 의 파일 lock 으로 배포를 직렬화한다. 이미 더 최신 main 이 있으면 과거 실행을 건너뛴다.

Secret 사전 확인 → DB·검색 준비 → 계정 초기화 → SQL migration → OPA bundle → 게시 artifact 확인 → API·Worker 갱신 → Prefect 등록 → API 동작 확인 순서다.

실패 시 PVC 와 Job 상태를 남기고 배포 실패로 종료한다. DB migration 을 자동으로 되돌리거나 PVC 를 삭제하지 않는다.

관리 작업은 저장된 커밋 디렉터리에서 같은 스크립트로 실행한다.

```
bash deploy/k3s/deploy.sh ghcr.io/owner/repository COMMIT_SHA teoria.example.com
```

API 와 Worker 는 단일 replica·Recreate 방식이므로 갱신 중 잠깐 중단될 수 있다. 진단 Worker 를 사용하려면 Secret 의 모델 설정을 채우고 `application/platform.yaml` 의 `metadata-assessment-worker` replica 를 1 로 바꾼다. OpenMetadata ingestion schedule 과 retry 는 Prefect 가 소유하고 실제 실행은 한 번짜리 Kubernetes Job 이다. Docker socket·Docker daemon·Airflow 를 요구하지 않는다.

정해진 자원 안에서 파이프라인 병렬 실행

수집 Flow·Task 와 `pipelines/prefect.yaml` 은 Compose 와 K3s 가 함께 사용한다. Compose 는 기존 `teoria-ingestion` Process Worker 내부에서 실행한다. K3s 는 등록 시 work pool 만 `teoria-ingestion-k3s` 로 바꾸고, Kubernetes Worker 가 실행마다 별도 Job 을 만든다. Flow 마다 Docker·Kubernetes 분기 코드를 넣지 않는다.

		기본 정책은 다음과 같다. EC2 의 사양과 서비스에 남길 자원을 고려해 배포 전에 조정한다.

설정

기본값

변경 위치

수집 전체 requests 예산

CPU 2·메모리 4Gi

deploy/k3s/execution/resource_quota.yaml

수집 전체 limits 예산

CPU 4·메모리 8Gi

같은 ResourceQuota

작업당 requests

CPU 1·메모리 2Gi

등록 Job 의 TEORIA_JOB_CPU_REQUEST, TEORIA_JOB_MEMORY_REQUEST

작업당 limits

CPU 2·메모리 4Gi

등록 Job 의 TEORIA_JOB_CPU_LIMIT, TEORIA_JOB_MEMORY_LIMIT

수집 동시 실행 상한

2

등록 Job 의 TEORIA_PIPELINE_CONCURRENCY

Backfill·보정 동시 실행 상한

1

등록 Job 의 TEORIA_PIPELINE_BACKFILL_CONCURRENCY

공통 등록 명령은 `teoria-pipelines register --backend kubernetes --kind all` 이다. 작업당 기본값은 `pipelines/src/teoria_pipelines/deployment.py`, 운영 override 는 `deploy/k3s/jobs/prefect-register.yaml` 의 환경변수로 관리한다. Prefect work pool 의 base job template 에 등록되며, Deployment 의 `work_pool.job_variables` 로 작업별 CPU·메모리를 다르게 지정할 수도 있다. 등록 Job 이 원본 YAML 을 임시 사본으로 변환하므로 Compose 의 Deployment·스케줄·입력 파라미터는 바뀌지 않는다.

최신 수집은 `latest` Queue, Backfill·보정은 `backfill` Queue 를 사용한다. 기본 동시 실행 상한 내에서 최대 2 개 Job 을 생성하되 Backfill·보정은 1 개까지만 실행한다. 나머지는 Prefect 에서 대기한다. 최신 수집을 먼저 선택하며 이미 실행 중인 작업을 선점하지 않는다. 같은 Deployment 중복 실행은 기존 `CANCEL_NEW` 정책이 먼저 적용된다. Kubernetes 는 requests 와 실제 노드의 예약 가능한 자원을 기준으로 Pod 를 배치하고, CPU limit 초과 사용을 제한하며 메모리 limit 초과 시 프로세스가 종료될 수 있다. Quota 는 `teoria-pipelines` namespace 에만 적용되므로 API·DB·Prefect Server 예산과 구분된다. 동시 실행 상한과 작업별 자원 합계가 Quota 안에 들어가도록 함께 조정한다. Quota 초과는 admission 거부, 노드 용량 부족은 Pending 을 유발할 수 있다. Pod 시작을 600 초 기다린 뒤에도 실행되지 않으면 실패로 처리될 수 있으므로 무한 대기를 보장하지 않는다.

Job 종료 후 사용 자원은 반환되고 완료 Job 은 1 시간 뒤 정리된다. 실행 Dispatcher 는 DB·Connector 자격증명을 받지 않으며 Job 에만 필요한 Secret 을 전달한다. 배포 스크립트는 `teoria` 의 Secret 에서 수집용 키와 registry 인증만 `teoria-pipelines` 로 동기화하므로 Secret 회전 후 재배포한다. Kubernetes Dispatcher 재시작은 별도 실행 중인 Job 을 일괄 취소하지 않는다. Compose 시작 시에는 Prefect 가 종료를 확인한 실행의 미완료 DB 기록만 정리한다. 실행 중이거나 Prefect 와 연결을 확인할 수 없는 기록은 자동 취소하지 않는다.

이는 실행 요청에 따라 제한된 범위에서 Job 수를 늘리고 줄이는 구조다. 단일 Flow 의 연산을 자동 분산하거나 Worker replica·EC2 서버를 자동 증설하는 설정은 포함하지 않는다. OpenMetadata ingestion 은 별도 Metadata Worker 가 관리하며 위 수집 namespace 예산에 포함하지 않는다. 실제 EC2 의 병렬 실행·자원 포화 검증은 서버 준비 후 별도로 수행해야 한다.

기준: Prefect Kubernetes Worker 의 Job template [참고 1].

노드 증설과 영속 데이터

처음에는 nodeSelector 나 taint 를 강제하지 않아 server 노드에서 모든 CPU Pod 가 실행된다. 나중에 CPU·GPU agent 를 추가한 뒤 실제 필요한 workload 에 affinity·GPU resource·toleration 을 적용한다. Prefect Server 의 위치와 작업 실행 Worker 의 위치는 독립적이다.

`local-path` PVC 는 생성된 노드의 디스크에 묶인다. agent 를 추가해도 DB·artifact 가 자동으로 다른 노드로 이동하지 않는다. Stateful workload 이동과 디스크·DB 백업은 별도 계획이 필요하다. EC2 한 대 장애는 전체 서비스·Control Plane 장애다. Control Plane HA 는 CPU·GPU agent 추가와 별개의 datastore·server 구성 변경이다.

공식 기준: K3s 설치 [참고 2], 영속 스토리지 [참고 3], GHCR 인증 [참고 4].

기존 scaffold 안내 주소 [§37]는 문서 링크 호환을 위해 새 안내를 가리킨다.

외부 참고 주소

[1] https://reference.prefect.io/prefect_kubernetes/worker/

[2] https://docs.k3s.io/quick-start

[3] https://docs.k3s.io/add-ons/storage

[4] https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

39. openmetadata.md — OpenMetadata Server 배포와 업그레이드

원문 파일: docs/deployment/openmetadata.md

OpenMetadata 는 기술·거버넌스·Semantic Metadata 의 권위 저장소다. Compose 의 `metadata` profile 로 Server, 검색 엔진과 schema migration 을 실행한다. 수집 일정·재시도는 Prefect 가 담당하고 별도 ingestion Job [§32]으로 실행한다. 기본 배포에 Airflow 를 추가하지 않는다.

Server 시작

Compose 환경 설정 [§36]을 준비하고 실행한다.

	`docker compose --env-file deploy/compose/.env -f deploy/compose/compose.yaml \ --profile metadata up -d`

구성

역할

`platform-postgres` 의 `openmetadata_db`

OpenMetadata 전용 DB·role

`openmetadata-search`

검색 인덱스

`openmetadata-migrate`

Server 시작 전 schema migration

`openmetadata-server`

UI 와 REST API, 기본 host port 8585

OpenMetadata DB 와 인덱스는 Teoria Application DB 나 Data DB 가 아니다. 기본 버전은 `OPENMETADATA_VERSION=2.0.5` 다. 기존 `.env` 가 이전 버전을 지정하면 그 값으로 실행하므로 저장소 기본값 변경만으로 기존 catalog 가 업그레이드되지 않는다. 버전은 검증한 값을 고정한다.

수집과 Platform 연결

Server health 와 UI `http://localhost:8585/`를 확인한다.

bot token 을 저장소 밖에서 준비하고 수집 Job [§32]을 실행한다.

OpenMetadata 에서 schema 와 자산을 확인한다.

Admin API 에 `TEORIA_OPENMETADATA_ENABLED=true`, API base URL 과 bot token 을 설정한다.

Admin 데이터 카탈로그에서 동일 자산·설명·참조를 확인한다.

ingestion 은 쓰기 bot 을 사용하고, Teoria Admin 에는 조회·승인된 제안 적용에 필요한 권한을 부여한다. 조회 전용 운영이면 별도 read-only bot 을 사용한다. 인증정보를 이미지·YAML·로그에 기록하지 않는다. 세부 참조는 자산 맥락 [§15]을 따른다.

기존 catalog 업그레이드

새 버전은 운영 DB 를 복원한 격리 환경에서 먼저 검증한다. DB migration, 전체 검색 재색인, 자산 ID·FQN·설명·소유자·태그·Glossary·Binding 참조 보존, Teoria API 연계와 재수집을 확인한다. 기존 버전 복원도 검증한다. 1.12.6 에서 2.0.5 로 전환한 결과는 2026-10-13 검증 기록 [§53]에 있다.

검증한 버전을 운영에 반영할 때는 다음 순서를 따른다.

Metadata ingestion schedule 을 일시 정지하고 진행 중인 Job 과 관리 쓰기를 종료한다.

그 시점의 `openmetadata_db` 를 백업한다. 이전 검증용 백업으로 대신하지 않는다.

Server 를 중지하고 선택한 버전의 migration Job 을 실행한다.

migration 성공 후 Server 를 시작하고 전체 검색을 재색인한다.

동일 버전의 ingestion image, Worker allowlist 와 Prefect deployment parameter 를 반영한다.

자산·Binding 참조·Teoria API·사용자 metadata 보존을 재검증한 뒤 schedule 을 재개한다.

Server·migration·ingestion image, ingestion Dockerfile build argument, Prefect 이미지 파라미터·Flow 기본값 ·Worker allowlist 를 함께 확인한다. 검색엔진 호환 버전도 배포 정의와 선택한 OpenMetadata 버전의 요구사항을 대조한다.

실패 시 복구

쓰기와 ingestion 을 중지하고 변경 전 DB 백업을 복원한 뒤 기존 Server·ingestion·Worker· Prefect 파라미터를 함께 복구하고 검색을 재색인한다. image tag 만 내리는 것으로 DB migration 을 되돌릴 수 없다. 공유 `platform-postgres` 전체 volume 이 아니라 `openmetadata_db` 만 복구해 Prefect·Teoria Application DB 가 함께 되돌아가지 않게 한다.

DB 백업은 인증 설정을 포함할 수 있으므로 접근을 제한하고 Git 에 추가하지 않는다. 백업·상세 로그의 보존 경로와 기간은 작업별로 정하고 일회성 credential·컨테이너·network 는 검증 종료 후 정리한다. EC2 K3s 에서 Prefect 가 Kubernetes Job 을 실행하는 구성은 K3s 배포 [§ 38]를 따른다.

40. gateway.md — Teoria MCP Gateway

원문 파일: docs/mcp/gateway.md

`mcp/`는 Runtime API 의 공개 Capability discovery 를 MCP Tool 로 노출한다. Tool 이름은 Capability ID 이며 입력 schema 는 배포된 Runtime discovery 응답에서 받는다. MCP 프로세스가 Registry 원본이나 DB 를 직접 읽지 않는다.

코드 위치

구현은 `mcp/src/teoria_mcp/`에 있다. `stdio.py` 는 실행 진입점, `tools.py` 는 Tool 등록, `schema.py` 는 MCP 입력 schema 처리, `runtime_client.py` 는 Runtime HTTP 호출, `settings.py` 는 환경변수를 소유한다. Capability 별 실행 로직은 Platform 에 둔다.

실행

저장소 루트에서 `TEORIA_ENV_FILE=deploy/compose/.env` 를 지정하고 Runtime API URL 과 token 을 준비한다. 호스트에서 Compose Runtime 에 접속하면 URL 은 `http://localhost:8081/runtime-api` 다. `http://runtime-api:8000` 은 Compose 내부 주소다. `TEORIA_MCP_RUNTIME_MODE=remote` 만 지원한다.

MCP 는 Runtime HTTP API 를 호출하며 DB 권한과 Source 인증정보를 갖지 않는다.

```
uv run --locked --package teoria-platform teoria validate platform/registries
uv run --locked --package teoria-mcp teoria-mcp
```

Codex 설정:

```toml
[mcp_servers.teoria]
command = "uv"
args = ["run", "--locked", "--package", "teoria-mcp", "teoria-mcp"]
cwd = "/absolute/path/to/teoria"

[mcp_servers.teoria.env]
TEORIA_MCP_RUNTIME_MODE = "remote"
TEORIA_MCP_RUNTIME_API_URL = "http://localhost:8081/runtime-api"
TEORIA_MCP_RUNTIME_API_TOKEN = "development-token"
```

위 URL 은 호스트에서 실행하는 예다. Compose 내부 MCP 컨테이너는 `http://runtime-api:8000` 을 사용하고 token 예시는 실제 배포의 secret 으로 교체한다.

운영 구조는 `AI Client → MCP Gateway → Runtime HTTP API` 다. MCP 는 Registry, Source 키와 DB 권한을 갖지 않는다.

Tool 은 `exposure: public` 인 Capability 만 발견하며 제목과 설명에 active Artifact 의 CapabilityVersion 을 표시한다. Tool 은 Ontology Object·Link 와 provenance 를 반환한다. `_options.include_property_provenance` 와 `_options.max_objects` 로 응답 범위를 조절한다.

오류와 재시도

입력 schema 는 Gateway 시작 시 Runtime discovery 응답에서 가져오며 MCP 코드에 Capability 별 schema 를 복제하지 않는다. 페이지 입력을 제공하는 Capability 는 discovery schema 의 `page`, `page_size` 를 그대로 사용하고, 응답의 `pagination.page`, `pagination.page_size`, `pagination.total_items`, `pagination.total_pages` 를 다음 호출 판단에 사용한다. Gateway 는 결과를 임의로 합치거나 자동으로 다음 페이지를 호출하지 않는다.

Gateway 시작 시 수행하는 Capability discovery 는 network error, timeout 과 HTTP `429`, `502`, `503`, `504` 를 일시 장애로 분류하고 제한된 exponential backoff 로 재시도한다. 최대 시도 횟수와 최초 backoff 는 `TEORIA_MCP_RUNTIME_MAX_ATTEMPTS`, `TEORIA_MCP_RUNTIME_RETRY_BACKOFF_SECONDS` 로 설정한다. `Retry-After` 가 초 단위로 반환되면 그 값을 최소 대기시간으로 사용한다. Discovery 와 실행의 요청 timeout 은 `TEORIA_MCP_RUNTIME_TIMEOUT_SECONDS` 로 제한한다.

Capability 실행은 POST 이며 향후 side effect Capability 가 추가될 수 있으므로 Gateway 가 자동 재시도하지 않는다. 오류는 Runtime 의 `code` 와 `message` 를 보존하고 HTTP status, 시도 횟수와 재시도 가능 여부를 함께 분류한다. 호출자는 `retryable=true` 만으로 재실행하지 말고 해당 Capability 의 effect 와 실행 결과 수신 여부를 확인해야 한다.

Docker 실행:

```
docker compose -f deploy/compose/compose.yaml --profile mcp run --rm mcp
```

검증

```
uv run --locked --package teoria-mcp pytest mcp/tests
```

MCP 는 STDIO protocol 을 사용하므로 애플리케이션 로그를 표준 출력에 추가하지 않는다. 환경변수의 기본값은 환경 설정 [§ 36], 실제 실행 계약은 Platform Runtime [§ 23]을 참고한다.

41. provider_api.md — Teoria Provider API

원문 파일: docs/packages/provider_api.md

`teoria-provider-api` 는 Platform Source 와 Pipeline Connector 가 공유하는 작은 Python 라이브러리다. 서비스가 아니므로 포트, 데이터베이스, 독립 Docker 배포를 갖지 않는다.

책임

포함 범위:

Provider API wire contract schema

입력 검증과 HTTP request 생성

HTTP 실행, credential interface, retry 와 구조화 오류

응답 record path 와 wire type 검증

포함하지 않는 범위:

Semantic Registry, Ontology, Mapping, Capability

Pipeline Definition, Prefect Flow, 정규화와 DB 적재

MCP 또는 Runtime HTTP client

코드와 설정 위치

구현은 `packages/provider_api/src/`, 테스트는 `packages/provider_api/tests/`, 패키지 계약은 `packages/provider_api/pyproject.toml` 에 둔다.

요청이 처리되는 순서

`schema.py` 가 wire 계약을 표현하고 `validator.py` 가 계약을 검증한다. `request_builder.py` 는 입력을 HTTP 요청으로 만들고, `executor.py` 는 인증정보 주입·HTTP 실행·재시도를 수행한다. `response_validator.py` 는 record path 와 응답 타입을 검증한다. 인증정보 경계는 `secrets.py`, 오류와 진단은 `errors.py`·`diagnostics.py` 에서 찾는다.

Platform 과 Pipelines 가 각자의 실행·적재 정책을 유지하면서 같은 Provider wire 처리만 공유한다. 소유 프로젝트의 Registry, Flow, Runtime 을 라이브러리에서 import 하지 않는다.

검증

```
uv run --locked --package teoria-provider-api pytest packages/provider_api/tests
```

상세 문서

호출 프로젝트의 계약 작성은 Source authoring [§ 22]과 Connector 작성 [§ 26]을 따른다.

42. bid-check-service.md — 입찰체크 서비스 연동

원문 파일: docs/integration/bid-check-service.md

입찰체크 서비스는 MCP 나 Data DB 에 직접 연결하지 않고 Runtime HTTP API 를 호출한다.

호출 가능한 Capability 와 입력 계약은 배포된 Runtime 의 `GET /v1/capabilities` 에서 확인한다. 저장소의 Registry 원본과 운영 artifact 는 version 이 다를 수 있으므로 클라이언트가 사용하는 배포의 discovery 와 OpenAPI 를 기준으로 한다. 조달 탐색 Capability 5 개와 기관·업체 관계의 `network_edge` 모드는 Runtime bundle `2026.10.13.1` 부터 제공한다. 배포마다 discovery 를 확인한 뒤 기능을 활성화한다.

빠른 탐색의 신규 이름, 기존 계약 재사용 범위와 인수 조건은 조달 탐색 연동 계약 [§43]에 있다.

```
Bid service → API gateway → Teoria Runtime API → Capability → Source/Data DB
```

계약 발견

배포마다 Registry release 가 달라질 수 있으므로 정적 Capability 목록이나 입력 schema 를 client 코드에 복제하지 않는다.

`GET /health` 로 상태를 확인한다.

`GET /v1/version` 으로 Runtime 과 Registry version 을 기록한다.

`GET /v1/capabilities` 에서 Capability ID 와 `input_schema` 를 조회한다.

필요한 Capability 를 `POST /v1/capabilities/{id}:execute` 로 실행한다.

정확한 request 와 response 는 discovery 응답과 배포된 OpenAPI 가 최종 기준이다.

```
curl -H "Authorization: Bearer $TEORIA_RUNTIME_API_TOKEN" \
  http://localhost:8081/runtime-api/v1/capabilities
```

권장 호출 흐름

입찰 대상 검색, 공고 상세·낙찰·참여 이력 확인, 회사 profile 조회 순서로 실행한다. 첨부문서에서 참가요건을 추출하거나 업체의 참가 가능성을 판정하는 기능은 보류되어 현재 discovery 계약에 포함하지 않는다.

Capability 응답의 object 와 link 에는 type, identity 와 provenance 가 포함된다. 객체가 없다는 사실만으로 자격 상태를 추론하지 말고 Capability 가 정의한 outcome, 오류와 데이터 기준일을 함께 확인한다.

기관·업체 계약 집계 표시

기관·업체 조달 프로필의 계약 통계는 `analyze_organization_procurement_profile` [platform/registries/domains/public_procurement/capabilities/analyze_organization_procurement_profile.yaml]과 `analyze_company_procurement_profile` [platform/registries/domains/public_procurement/capabilities/analyze_company_procurement_profile.yaml]을 사용한다. 산정 규칙의 단일 원본은 공공조달 계약 집계 기준 [§10]이다.

입찰체크는 응답의 `analysis_basis` 와 `summary.amount_completeness`, `summary.missing_first_contract_date_count` 를 함께 해석해야 한다. 계약 추이에는 최근 변경일이 아니라 최초 계약일을 사용하고, 다음 안내를 계약 건수·금액과 함께 표시한다.

계약 건수는 최초 계약일 기준이며, 금액은 조회 종료일 현재의 최신 계약금액입니다.

Ontology 경계

입찰체크 Ontology 는 기존 Published version 을 직접 사용하거나 수정하지 않는다. Published version 에서 새 Draft 를 생성하고 의미가 유지되는 stable concept 를 승계한다. 기관·업체 프로필, 공고 관계 컨텍스트, 유사사업 경험과 같은 집계 응답은 Business Object 로 만들지 않고 Capability 결과 projection 으로 유지한다.

	오류와 pagination

종류

처리

인증/권한

재시도하지 말고 credential 또는 gateway 정책 확인

입력 검증

discovery schema 에 맞춰 요청 수정

timeout/일시 장애

멱등 호출만 제한된 exponential backoff 로 재시도

provider 오류

부분 결과와 provenance 를 보존하고 사용자에게 표시

cursor

응답 cursor 를 그대로 다음 요청에 전달

client timeout 은 Runtime 의 Capability deadline 보다 길게 설정한다. 검색 결과의 `truncated`, `has_more` 와 cursor 의미를 혼용하지 않는다.

보안과 운영

`TEORIA_RUNTIME_API_TOKEN` 은 backend secret 으로 관리하고 브라우저에 노출하지 않는다.

외부 공개 시 gateway 에서 사용자 인증, rate limit, token rotation 과 접근 로그를 적용한다.

사업자번호 등 식별정보를 application log 와 analytics event 에 그대로 남기지 않는다.

응답의 Registry version, artifact checksum 과 provenance 를 감사 가능하게 보존한다.

데이터 수집 완전성과 최신성은 Capability 존재 여부와 별개로 monitoring 한다.

인수 확인

배포 시작 시 discovery 와 필수 Capability 존재 여부를 검사한다.

input schema 변경을 contract test 로 감지한다.

성공, 빈 결과, 부분 결과, validation error, timeout 을 각각 시험한다.

Registry release 전환과 이전 artifact rollback 을 시험한다.

43. procurement_exploration_contract.md — 입찰체크 조달 탐색 연동 계약

원문 파일: docs/integration/procurement_exploration_contract.md

자동완성·네트워크·분야 시장 조회를 연결할 때의 현재 계약과 관측 범위를 설명한다. 배포별 지원 기능과 입력 schema 는 Runtime discovery·OpenAPI 를 확인한다. 저장소 원본 Registry 와 운영 artifact 는 version 이 다를 수 있다. 기본 호출 순서는 입찰체크 연동 [§42]을 따른다.

	기능 선택

Capability

용도

`autocomplete_procurement_entities`

공고·기관·업체·공식 분야 후보 선택

`get_procurement_network`

기관·업체·분야 시작점의 제한된 계약 관계 그래프

`get_organization_company_relationship`

기관–업체 관계와 근거 조회; `network_edge` 모드 지원

`analyze_procurement_field_market`

수집 범위 내 분야·연도별 시장 집계

`get_procurement_classification_catalog`

관측된 공식 분류체계·코드·계층

`get_procurement_region_filter_options`

지원하는 참가 지역제한 코드·명칭

전체 프로필은 노드를 선택한 뒤 별도 Capability 로 조회한다. 네트워크를 그리기 위해 각 노드의 프로필·경쟁사 API 를 반복 호출하지 않는다. 특정 공고 맥락은 `get_bid_notice_relationship_context`, `analyze_bid_participation_context`, `search_bid_related_projects` 의 계약을 유지한다.

자동완성에서 필터로 연결하기

`query`, `types`, `limit`, `cursor` 를 사용한다. 현재 limit 은 기본 8, 최대 20 이며 응답 전체 후보 수에 적용한다. 내부 검색 인덱스에서 prefix·초성·번호를 검색한다. 외부 기업정보 API 나 실적 전체 집계를 호출하지 않는다.

후보의 type·ID·이름·번호와 검증된 `filter_values` 를 사용한다. 누락값은 `null` 로 유지하며 제목으로 분야를 추정하지 않는다. 공고기관·수요기관은 원천 역할을 구분한다.

공고의 참가 지역은 다음 형태다.

```
{"basis": "participation_restriction", "codes": ["11", "26"], "match": "any"}
```

실제 응답의 값을 조회 입력 `region_basis`·`region_codes` 로 전달한다. 위 코드는 형식 예이며 원천 근거 없는 지역을 추가하지 않는다. 단일 입력은 기존 `region_code` 도 지원한다. 기관·업체 소재지를 공고 참가 지역제한으로 바꾸어 사용하지 않는다.

cursor 는 조회 조건 fingerprint 와 offset 을 포함한다. 다른 질의에 재사용하면 `invalid_cursor` 다. 현재 cursor 가 고정된 데이터 snapshot 을 보장하거나 시간 기반 만료를 제공한다고 가정하지 않는다. 검색 중 질의가 바뀌면 이전 응답을 UI 에 반영하지 않는다.

	네트워크 입력과 표시 제한

입력

현재 의미

`root_type`, `root_id`

`organization`, `company`, `field` 시작점과 원천 식별자

`field_code`

분야 시작점 또는 분야 필터; 분야 시작점에서 root ID 대신 사용 가능

`work_type`

업무구분 필터

`region_basis`

현재 `participation_restriction` 만 지원

`region_code`, `region_codes`

구조화된 공고 참가가능지역; 복수 코드는 대안 조건

`period_from_year`, `period_to_year`

최초 계약일의 연도 범위

`depth`

기본 1, 최대 2

`first_hop_limit`

기본 20, 최대 100

`second_hop_limit`

기본 50, 최대 200; 2 단계 조회에만 적용

기관·업체 시작점에는 해당 ID 를 사용하고 분야 필터는 선택적이다. 분야 시작점은 공식 분야 코드로 지정한다. 이름 유사성으로 노드를 합치지 않는다. 현재 2 단계 제한은 2 단계 조회 전체의 edge 목록에 적용하며 노드마다 limit 만큼 확장하는 계약이 아니다.

응답은 nodes·edges·summary·truncation·coverage·analysis_basis 를 포함한다. 선정 기준은 계약 건수 내림차순·업체 귀속금액 내림차순이며, `truncation.selection_basis` 를 함께 확인한다. 상위 제한과 전체 시장 집계를 구분한다. 제외된 정확한 고유 건수를 계산하지 않은 경우 `omitted_node_count`·`omitted_edge_count` 는 `null` 이다.

현재 읽기 모델과 금액 기준

네트워크는 `contract_event_company_ledger` 의 계약 관계를 조회한다. 현재 `coverage` 는 계약만 `available` 이며 낙찰·관측 참여·동시 참여·공동수급 관계는 `not_in_read_model` 로 반환한다. 이러한 관계가 없다고 원천 업무에 존재하지 않는다는 뜻은 아니다. 동시 참여를 공동수급·협력으로 해석하지 않는다.

현재 네트워크의 금액 기준은 `latest_version_at_ledger_refresh` 다. 연도 필터는 최초 계약일에 적용한다. 프로필의 조회 종료일 이전 최신 계약금액과 같다고 가정하지 않는다. 화면은 응답의 `analysis_basis` 와 갱신 시점을 표시한다. 계약사건·공동수급 귀속의 기본 의미는 계약 집계 [§10], 활동·참여 관측은 활동 집계 [§09]를 따른다.

분야 시장의 전체는 수집 범위 내 전체다. 연도별 업체 수를 더해 전체 고유 업체 수로 사용하지 않는다. 공식 분류 catalog 는 원천에서 관측한 범위이며 전체 공식 분류 마스터와 동일하지 않을 수 있다. 범위는 분류 기준 [§13]을 참고한다.

관계 상세와 기준점

기관–업체 관계의 `view_mode=network_edge` 는 네트워크 읽기 모델과 같은 필터로 집계·근거를 조회한다. 생략 시 기본 `history` 계약을 유지한다. history 의 공고 연결 사건은 게시일, 공고 미연결 계약은 최초 계약일 기준이므로 두 모드를 혼용하지 않는다.

`snapshot_id` 는 현재 ledger 갱신 시각을 표현한다. 네트워크·상세가 같은 필터와 갱신 기준인지 확인한다. 현재 반환 ID 를 영속 snapshot 저장·만료·재현 계약으로 해석하지 않는다. 화면에 표시된 상위 edge 의 합계와 전체 관계의 집계, 한 근거 페이지의 합계는 구분한다.

소비자 검증

같은 필터·동일한 읽기 모델 상태에서 edge 와 상세 전체 집계를 비교한다.

없는 데이터, 미지원 필터, 표시 제한, 수집 범위 부족과 서버 오류를 구분한다.

여러 계약·변경본·장기계속계약의 계보·금액을 임의 합산하지 않는다.

응답의 근거·금액 기준·관측 범위·갱신 시각을 화면에 보존한다.

자동완성에서 선택한 다중 지역과 실제 조회 필터가 일치하는지 확인한다.

후속 요구사항과 성능 평가

다음 항목은 현재 보장과 구분하는 후속 검증 범위다.

과거 조회 종료일의 version 을 네트워크 읽기 모델에서도 재현하는 계약

graph·상세·cursor 를 고정된 snapshot 에 결속하고 만료를 구분하는 계약

읽기 모델에 없는 낙찰·참여·동시 참여·공동수급 관계의 근거와 수집 범위

서버·DB 작업까지의 취소 전파와 부하 조건별 성능 측정

기존 제안 P95 목표는 자동완성 200ms, 네트워크 cache hit 1 초, miss 2 초다. 합의·검증한 SLA 가 아니며 데이터량·동시 요청·기간·depth·payload·cache 조건을 고정해 측정해야 한다. 2026-10-13 단회 관측 [§55]은 해당 목표의 달성 증거가 아니다.

인덱스·ledger 생성·갱신과 Data DB migration 은 Pipelines, 조회·집계·캐시·Capability 는 Platform 공공조달 Domain 이 소유한다. 문서 변경만으로 Registry·Published artifact 나 Ontology 가 바뀌지 않는다.

44. bid_check_capabilities.yaml — 계약 참조·템플릿

원문 파일: docs/integration/bid_check_capabilities.yaml

다음은 원문 YAML 파일의 전체 내용이다. 예제·템플릿은 실행 중인 Registry 계약과 구분한다.

```yaml
schema_version: "1.0"
consumer: bid_check
description: 입찰체크 서비스가 직접 사용하는 Runtime Capability 계약
planning_document: procurement_exploration_contract.md
retired_capabilities:
  - id: find_similar_bid_notices
    replacements: [search_bid_related_projects]
    reason: 입찰체크 시장 컨텍스트가 최신 관련 계약사업 검색 결과를 사용하도록 전환했다.
  - id: find_bid_relevant_companies
    replacements: [search_bid_related_projects]
    reason: 관련 사업의 계약업체를 입찰체크 adapter 에서 집계하도록 전환했다.
  - id: analyze_bid_organization_field_companies
    replacements: [analyze_bid_participation_context]
    reason: 기관·분야 업체 화면을 최신 참여 판단 컨텍스트 결과로 전환했다.
  - id: get_bid_result
    replacements: [search_bid_awards, get_bid_notice_participations]
    reason: 공고별 낙찰과 개찰 참여 조회를 하나로 중복 조합하던 구 계약이다.
  - id: get_company_bid_history
    replacements: [search_bid_awards, search_bid_participations]
    reason: 기업별 낙찰과 참여 검색을 하나로 중복 조합하던 구 계약이다.
  - id: get_demand_organization
    replacements: [search_public_organizations]
    reason: Provider 등록일 기반 구 조회이며 최신 기관 검색 계약으로 대체한다.
  - id: get_women_owned_business_qualification
    replacements: [get_company_bid_qualification_profile]
    reason: 기업 입찰자격 프로필이 동일 자격과 직접생산확인을 함께 제공한다.
  - id: get_disabled_owned_business_qualification
    replacements: [get_company_bid_qualification_profile]
    reason: 기업 입찰자격 프로필이 동일 자격과 직접생산확인을 함께 제공한다.
  - id: verify_business_registration
    replacements: [get_business_registration_status]
    reason: 입찰평가는 사업자 상태조회만 사용하며 별도 진위확인 Capability 실행 경로가 없다.
internal_support_capabilities:
  company_data:
    - get_bid_notices_by_ids
    - get_business_registration_status
    - get_company_qualifications
    - get_direct_production_confirmations
    - get_procurement_supplier
    - get_procurement_supplier_industries
    - get_procurement_supplier_products
    - get_procurement_supplier_sanctions
    - verify_innobiz_company
    - verify_mainbiz_company
    - verify_venture_company
  company_identity:
    - get_company_profile
    - get_company_financials
    - get_company_relationships
    - resolve_company_identifiers
  market_context:
    - get_company_similar_project_experience
ontology_decision:
  change_required: true
  strategy: new_draft_from_published
  target_namespace: teoria
  reuse_stable_concepts: true
  rationale: 기존 Published artifact 는 재사용하지 않되 의미가 같은 stable concept 를 승계한 새 Draft 를 검토·게시하고
Capability 를 다시 연결한다.
capabilities:
  - {id: autocomplete_procurement_entities, area: exploration, status: active}
  - {id: get_procurement_classification_catalog, area: classification, status: active}
  - {id: get_procurement_network, area: exploration, status: active}
  - {id: get_procurement_region_filter_options, area: region, status: active}
  - {id: analyze_procurement_field_market, area: market, status: active}
  - {id: search_bid_notices, area: bid_notice, status: active}
  - {id: get_bid_notice, area: bid_notice, status: active}
```

```
- {id: get_bid_notice_participations, area: bid_notice, status: active}
- {id: get_bid_notice_contracts, area: bid_notice, status: active}
- {id: get_bid_notice_relationship_context, area: bid_notice, status: active}
- {id: find_bid_project_lineage, area: bid_notice, status: active}
- {id: analyze_bid_participation_context, area: bid_analysis, status: active}
- {id: search_bid_related_projects, area: bid_analysis, status: active}
- {id: search_public_organizations, area: organization, status: active}
- {id: analyze_organization_procurement_profile, area: organization, status: active}
- {id: search_procurement_activity, area: organization, status: active}
- {id: search_procurement_outcomes, area: organization, status: active}
- {id: search_organization_supplier_entries, area: organization, status: active}
- {id: get_organization_company_relationship, area: organization, status: active}
- {id: search_public_procurement_contracts, area: organization, status: active}
- {id: get_company_procurement_profile, area: company, status: active}
- {id: get_company_bid_qualification_profile, area: company, status: active}
- {id: get_company_detail_context, area: company, status: active}
- {id: analyze_company_procurement_profile, area: company, status: active}
- {id: search_bid_participations, area: company, status: active}
- {id: analyze_company_competitors, area: company, status: active}
- {id: search_companies_by_name, area: company, status: active}
- {id: search_bid_awards, area: award, status: active}
```

45. analyze_metadata_asset.md — 자산 진단 Skills

원문 파일: docs/skills/analyze_metadata_asset.md

		메타데이터 인텔리전스는 OpenMetadata 자산을 대상으로 세 작업을 제공한다. Skill은 Admin UI의 개발용 요청을 복사해 Codex에서 실행한다. 모델 gateway를 설정한 자동 자산 진단 Worker [§14]는 별도 실행 경로이며, 첫 버전은 설명·직접 Binding 제안과 법령 검토 findings를 지원한다.

사용자 기능

Skill

현재 결과

메타데이터 정비

`$enrich-metadata-asset`

테이블·컬럼 설명, Glossary 생성·할당 제안

업무 의미 연결

`$align-metadata-semantics`

기존 Published Ontology Concept와 Binding 제안

개방 적합성 진단

`$assess-data-disclosure`

정보 유형·법령 근거·개방 상태·보호조치 검토 초안

신규 업무 Ontology 제안은 보류한다. 대응하는 Concept가 없으면 연결 공백을 보고한다. 기존 API의 `description`, `glossary_term`, `glossary_term_assignment`, `binding` 유형은 유지한다. 기존 품질·Ontology 적용기도 유지하지만 세 Skill의 생성 범위에 포함하지 않는다.

사용

자산 상세의 자산 진단에서 항목별 요청을 복사하거나 FQN을 지정한다.

```
$enrich-metadata-asset
OpenMetadata 테이블 teoria_postgresql.teoria_data.public_procurement.runtime_contracts의
설명과 Glossary를 검토해줘. 근거와 confidence를 보여주고 제안만 작성해줘.
$align-metadata-semantics
OpenMetadata 테이블 teoria_postgresql.teoria_data.public_procurement.runtime_contracts를
기존 Published Concept와 대조하고 유효한 Binding 제안을 Review Queue에 올려줘.
신규 Ontology는 만들지 말고, 승인하거나 적용하지 마.
$assess-data-disclosure
OpenMetadata 테이블 <table FQN>을 일반 공개하는 상황으로 검토해줘.
평가 기준일을 명시하고 수집된 법령의 조문·버전을 근거로 정보 유형,
내부 처리 검토 상태, 외부 개방 권고, 태그 후보와 보호조치를 보여줘.
개인정보 원본 샘플은 사용하지 말고 태그를 적용하지 마.
```

개방 진단은 현재 보고서까지만 지원한다. 법령·태그 Suggestion 유형과 적용기가 추가되기 전까지 지원하지 않는 유형을 제출하거나 설명 변경으로 우회하지 않는다. 법령 검색은 키워드 방식이며 검색 결과를 적법성 확정으로 사용하지 않는다. 법령 snapshot ID, 시행일, 조문 path, 평가 기준일과 적용 조건·예외를 기록한다. 존재하지 않는 OpenMetadata 태그는 분류체계 공백으로 보고한다.

연결과 검증

두 제안 Skill 은 같은 도구를 사용한다. API 주소 우선순위는 CLI 인자, `TEORIA_ADMIN_BASE_URL`, Compose 기본 주소 순이다.

```
export TEORIA_ADMIN_BASE_URL=http://localhost:8081/admin-api
python3 .agents/skills/enrich-metadata-asset/scripts/fetch_context.py \
  --table-fqn '<service.database.schema.table>' \
  --output /tmp/teoria_metadata_context.json
uv run --locked --package teoria-platform python \
  .agents/skills/enrich-metadata-asset/scripts/submit_proposals.py proposal_bundle.json
```

Compose 네트워크 안에서는 `http://admin-api:8001`, 로컬 API 단독 실행에서는 `http://localhost:8001` 을 지정한다. Bearer 인증은 `TEORIA_ADMIN_API_TOKEN` 을 사용한다. fetch 와 submit 모두 토큰을 환경변수로 받으며 출력하지 않는다.

사용자가 검토 대기열 등록을 요청했으면 검증 후 `--submit` 으로 pending Suggestion 을 등록한다. 기존 `$analyze-metadata-asset` 호환 Skill 과 스크립트는 제거했다. 위의 세 작업별 Skill 을 직접 호출한다.

	검토와 적용

제안 유형

승인 후 효과

`description`

OpenMetadata 테이블·컬럼 설명 변경

`glossary_term`

OpenMetadata Glossary Term 생성

`glossary_term_assignment`

OpenMetadata 컬럼에 Term 할당

`binding`

Binding Draft 생성; 별도 승인 후 활성화

컬럼 설명 제안은 `table_id`, `column_name`, `column_fqn`, `source_version`, `source_description` 을 포함한다. 승인 시 원천 변경을 검사하며 대상이 바뀌면 `stale_evidence` 로 중단한다. 같은 분석에서 생성한 설명은 승인된 의미 근거로 취급하지 않는다. 기본 의미 경로는 `Physical Asset -> Glossary Term -> Ontology` 다.

모든 Skill 은 제안·초안까지만 생성하며 직접 승인·적용하지 않는다. OpenMetadata entity 상태나 원본 샘플을 Teoria DB 에 복제하지 않는다.

실행 계약: 메타데이터 정비 [.agents/skills/enrich-metadata-asset/SKILL.md], 업무 의미 연결 [.agents/skills/align-metadata-semantics/SKILL.md], 개방 적합성 진단 [.agents/skills/assess-data-disclosure/SKILL.md]. 두 제안 Skill 의

bundle 형식은 proposal schema
[.agents/skills/enrich-metadata-asset/references/proposal_bundle.schema.json]를 따른다.

Bundle schema 는 `platform/src/teoria/intelligence/suggestions/contracts.py` 의 `metadata_proposal_bundle_schema()`에서 생성한다. 변경 시 reference JSON 을 재생성하고 `platform/tests/unit/intelligence/test_proposal_contract.py` 로 일치 여부와 검증 동작을 확인한다. Skill 검증은 같은 schema 를 사용하며 제출 시 `warnings` 를 API payload 에서 제외한다.

승인·적용·재시도와 결과 확인은 공통 제안 처리 [§17]를 따른다.

46. source-registry-author.md — Source Registry 작성 Skill

원문 파일: docs/skills/source-registry-author.md

`author-source-registry` 는 Provider Reference 원문에서 Runtime API Source 또는 Ingestion API Connector 와 Operation 별 verification case 를 작성하고, 정적 · Build · 회귀 · Live 검증을 반복하는 저장소 범위 Codex Skill 이다.

사용 방법

Codex 를 이 저장소 루트에서 시작하고 프롬프트에 Skill 이름과 대상 Reference 를 지정한다.

```
$author-source-registry
pipelines/references/providers/pps/pps_contract_api 의 모든 문서를 근거로
Ingestion Connector 와 verification cases 를 만들고 검증해줘.
모든 Operation 에 대해 정적 검증, Build 검증과 실제 API Live 검증을 실행하고,
검증 실패가 Source 계약 문제면 수정한 뒤 전부 다시 실행해줘.
최종 결과에는 Operation 별 PASS, FAIL, BLOCKED 상태를 남겨줘.
```

기존 Source 를 원문과 다시 대조할 때도 사용할 수 있다.

```
$author-source-registry
pipelines/references/providers/pps/pps_contract_api 원문을 기준으로
pipelines/connectors/pps_contract_api.yaml 의 모든 Operation 과 필드를 감사하고,
누락이나 오류를 수정한 뒤 실제 API Live 검증까지 실행하고 전체 결과를 보고해줘.
```

Skill 이름을 생략해도 요청 내용이 설명과 일치하면 Codex 가 자동 선택할 수 있지만, 재현 가능한 작업 지시에는 `$author-source-registry` 를 명시하는 편이 분명하다.

준비 사항

Source 원문은 `platform/references/providers/{provider}/{source}/`, Connector 원문은 `pipelines/references/providers/{provider}/{connector}/`에 둔다.

의존성은 `uv sync --locked --all-packages --all-groups` 로 설치한다.

`.env` 또는 실행 환경에 Source 나 Connector 의 `credential_env` 값을 설정한다. 키는 프롬프트, YAML, verification case 에 넣지 않는다.

credential 이 없더라도 정적 검증, 모든 Build 검증과 전체 테스트는 실행된다. Live 만 `BLOCKED` 로 보고된다.

예를 들어 조달청 Connector 는 다음 환경변수를 사용한다.

```
export TEORIA_CONNECTOR_PPS_CONTRACT_API_KEY='...'
```

Skill 을 사용한 Source 작성 · 검증 요청은 기본적으로 실제 API 호출을 포함한다. Live 호출이 불필요한 경우에만 프롬프트에 `Live 검증은 제외해줘`라고 명시한다.

산출물과 완료 기준

기본 산출물은 다음과 같다.

```
platform/registries/sources/{source_id}.yaml
platform/registries/sources/verification_cases/{source_id}/{operation_id}.yaml
platform/references/providers/{provider}/{source}/metadata.yaml
```

Ingestion 전용 API 라면 다음 위치를 사용한다.

```
pipelines/connectors/{connector_id}.yaml
pipelines/verification_cases/{connector_id}/{operation_id}.yaml
pipelines/references/providers/{provider}/{connector}/metadata.yaml
```

최종 보고에는 원문과 Registry 의 Operation · 필드 대조 결과, 실행한 검증 명령별 상태, 수정한 문제, 모든 Operation 의 Live `PASS` · `FAIL` · `BLOCKED` 상태와 남은 제한이 포함된다. 결과 파일을 `archive/`에 보관하려면 프롬프트에서 별도로 요구한다.

Skill 본문은 SKILL.md [.agents/skills/author-source-registry/SKILL.md]에 있으며, Reference 추출 도구는 같은 Skill 의 `scripts/extract_reference.py` 에 있다.

47. asset_assessment_2026_10_15.md — 자산 진단 실행 검증: 2026-10-15

원문 파일: docs/verification/asset_assessment_2026_10_15.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

이 문서는 당시 환경의 실행 결과다. 현재 설정과 사용 절차는 자산 진단 [§ 14]을 따른다.

검증 환경

현재 운영의 로컬 Qwen3.8-27B 는 8,192 토큰 문맥으로 실행되므로 출력 한도를 2,048 로 두고 소수 컬럼부터 진단한다. 대량 컬럼·긴 법령을 처리하려면 더 큰 문맥의 모델이나 입력 분할이 필요하다. 기존 모델 컨테이너는 `teoria_default` 네트워크의 `metadata-model` 별칭으로 연결한다. 이 컨테이너를 재생성하면 네트워크 별칭도 다시 연결해야 한다.

결과

로컬 Qwen3.8-27B 와 운영 Admin API·Worker 를 연결해 다음을 확인했다.

`be6f555e-3a9a-41a9-885a-46d2203785ec`: `runtime_contracts` 의 `contract_date`, `current_contract_amount` 진단 완료. 설명 제안 2 건은 pending 이며 신뢰도 0.2 인 검토 후보다. 의미 연결은 후보가 모호해 보류했다.

`21f1f682-ae5e-49c8-9349-539c00ec7ef8`: `runtime_bid_opening_participants` 의 `representative_name` 개방 진단 완료. 실제 수집 snapshot 을 인용한 검토 finding 을 생성했다. 법적 적합성 확인은 false 이며 태그와 원천 설명은 변경하지 않았다.

초기 법령 검색·인용 실패 회차는 partial 이력으로 보존한다. 한글 검색어 확장과 짧은 근거 ID 를 적용한 뒤 재검증했다. 이 검증은 실행·저장 경로의 확인이며, 생성 설명·법령 해석의 정확도나 Binding 품질을 보증하는 평가가 아니다.

48. async_deployment_collection_2026_10_16.md — 비동기 배포와 수집 상태 검증: 2026-10-16

원문 파일: docs/verification/async_deployment_collection_2026_10_16.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

현재 로컬 `teoria` Compose 프로젝트의 배포와 실제 수집을 확인한 기록이다. 문서 취합 기준일은 2026-10-16 이며 수집 수치는 **2026-10-16 15:46 KST / 06:46 UTC** 관찰값이다. EC2 K3s 실배포·GitHub CD 성공이나 전체 데이터의 완전성을 뜻하지 않는다.

반영한 실행 기준

Platform API → Service → Repository → PostgreSQL, Runtime reader 와 Pipeline DB Task 를 native async 로 연결했다. 계산·정규화·검증은 일반 함수로 유지한다.

Platform·Pipeline DB 연결 상한은 각각 기본 10 이며 event loop 별·접속 URL 별로 적용한다. 성공 시 commit, 예외·취소 시 rollback 후 연결을 닫는다.

승인·선점·효과 기록·게시·정규 적재와 checkpoint 의 순서를 보존한다.

첨부파일 다운로드·문서 추출·입찰요건 판단은 archive 상태를 유지한다.

	배포와 실제 검증

확인

결과

배포한 서비스

Runtime · Admin · Policy Control Plane · 자산 진단 Worker · 수집 Worker · Metadata Worker 6 개

배포 소스 일치

Platform 서비스별 Python 233 개, Pipeline Worker 별 76 개 파일 대조 일치

Prefect 등록

수집 · Metadata 등록 서비스 모두 exit code 0

기존 수집 보존

새 제출을 일시정지하고 실행 중인 2 개가 Completed 인 것을 확인한 뒤 Worker 교체

API

인증된 조회 HTTP 200, 미인증 HTTP 401, 없는 Ontology version HTTP 404

실제 Runtime 조회

기관 검색 · 분류 목록 · 공고 검색 · 기관 조달 프로필 · 조달 활동 검색 정상

내부 분류 보강

실제 read-only Data DB 에서 누락 분류 계층 조회 경로 실행

Prefect 검증 Flow

6 개 Task Completed, 의도한 1 회 실패 후 2 번째 시도 성공

DB 저장

실행 감사 `completed`, checkpoint 저장과 재시도 전 미갱신 확인

정책 감사

새 OPA decision · status 의 Application DB 저장 확인

최종 프로세스 상태

대상 서비스 실행 · 재시작 0 회, API health 정상, 두 Worker ONLINE

최종 로그

미대기 코루틴 · 잘못된 `asyncio.run` 경고 없음

배포 검증에서 동기 등록 함수를 `asyncio.run()`으로 감싼 CLI 호출과 분류 보강 SQL 의 `await` 누락을 발견해 수정 · 재빌드 · 재배포했다. 관련 회귀 테스트는 시장 분석 64 개, 등록 · entrypoint 16 개가 통과했다. 실제 DB 통합 · 승인 동시성 · 게시 rollback · 취소 검증은 전환 과정에서 별도로 수행했다.

Runtime artifact 는 기존 `2026.10.16.90` 을 사용한다. 이전 · 새 이미지가 같은 checksum 검증 artifact 를 읽는 것을 확인했으며 코드를 배포하기 위해 새 artifact 를 발행하지 않았다. 저장소 계약 46 개 중 Runtime 공개 discovery 는 30 개다.

검증 Flow 는 외부 Provider 를 호출하지 않고 빈 정규 배치와 별도 검증용 실행 ID 를 사용했다. 검증 DB 행 · Flow · Deployment · marker 는 완료 후 정리했고 work pool 의 일시정지 상태를 원래 값으로 복구했다. 실제 Provider 수집은 아래 별도 관찰로 확인했다.

수집 관찰

		수집 · Metadata pool 은 일시정지되지 않았고 각각 ONLINE Worker 를 확인했다. 최근 24 시간 Prefect 의 Failed · Crashed 실행은 0 건이며 10 분 이상 늦은 Scheduled 실행은 0 건이었다. 조회 시점에는 과거 개찰 참여업체 보강 1 개가 실행 중이었다.

수집

마지막 DB 완료 시각 KST

최신 checkpoint

입찰공고

15:17:35

2026-10-16

계약 증분

12:05:51

2026-10-16

낙찰 증분

12:31:33

2026-10-16

최신 개찰 보강

15:45:47

공고별 보강 큐로 추적

계약·낙찰은 4 시간 주기로 실행하므로 마지막 성공 시각을 현재 예약 간격과 비교한다. 계약·낙찰의 위 마지막 Provider 수집은 이번 Worker 교체 이전 실행이다.

새 Worker 배포 후 06:00 UTC 부터 관찰 시점까지 공고 1,002 건, 개찰 참여업체 6,111 건, 공고 결과 561 건을 처리·갱신했다. 개찰 참여업체는 최신 1,500 건·과거 4,048 건· 재시도 563 건의 합계다. 이 수치는 실행별 처리 건수이며 신규 고유 레코드 수가 아니다.

	남아 있는 데이터 결손

항목

관찰값·의미

개찰 보강 pending

633,194 건: 과거 607,818 + 증분 25,376

개찰 보강 retry_wait

199,619 건: 과거 197,384 + 증분 2,235

개찰 보강 processing

180 건

개찰 보강 partial_completed

353 건: 과거 3 + 증분 350

계약 Backfill 미해결 구간

1 개, 재시도 15 회 상한 도달

최근 오류 기록에는 `opening_lookup_failed`, `opening_response_incomplete` 가 있다. 보강 큐의 상태와 오류는 Flow 가 Completed 여도 남을 수 있다. `partial_completed` 는 전체 성공이 아니며 Runtime 수집 상태에서 `failed` 로 노출한다. 근거를 확인하지 않고 완료로 바꾸거나 checkpoint 를 임의로 앞당기지 않는다.

현재 판단은 **수집·적재가 실제로 진행 중이며 미처리·재시도 잔량은 남아 있다**는 것이다. 누적 건수를 전체 기간·모든 공고의 완전 수집으로 해석하지 않는다.

현재 절차와 후속 확인

재배포는 Compose [§35], 실행·지연·큐 점검은 Prefect 운영 [§33], 원본 보존과 감사 건수는 Data DB [§27]를 따른다. 로컬 검증에서 확인한 것과 별개로 EC2 K3s 실배포·자원 포화·장기간 처리량·모델 품질 평가는 추가 확인 대상이다.

49. bid_document_archive_2026_10_16.md — 첨부 문서·입찰요건 기능 보류 검증

원문 파일: docs/verification/bid_document_archive_2026_10_16.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

2026-10-16 사용자 요청에 따라 첨부파일 다운로드·파싱·문서 요건 추출과 업체별 참가자격 판단을 현재 제품에서 제거했다. 보관 범위와 재개 기준은 archive [archive/bid_document_processing/README.md]에 있다.

변경과 배포

전용 코드·Registry·Skill·테스트와 과거 문서는 원래 경로를 유지해 archive 에 보관했다. 공유하던 파일은 변경 전 사본을 보관하고 활성 실행 연결을 제거했다.

문서 다운로드·파싱·요건 추출·보존기간 삭제 Prefect Deployment 4 개를 운영에서 제거했다. AI pool 은 중지하고 AI Worker 컨테이너와 기본 배포 정의를 제거했다.

Runtime bundle `2026.10.16.90` 을 새로 생성·검증·활성화하고 API 와 Worker 를 배포했다. checksum 은 `sha256:2307532eb8b64ea0c2c37df1c4235535cd00ddd9f3fe150c068f5ffdd4333c1b` 다. 이전 게시본과 Published Ontology·승인 이력·Application DB 는 변경하지 않았다. 유지하는 Capability Binding 은 기존 계약과 동일함을 검사한 배포 snapshot 이며, 승인된 계약 target 과 Binding identity 를 보존했다.

migration `088` 은 집계 갱신 큐의 요청 버전을 추가한다. 같은 분야의 동시 갱신은 transaction advisory lock 으로 제한하고 처리한 버전만 삭제한다.

migration `089` 는 공고 조회의 문서 추출 테이블 연결을 제거한다. 종속 view 용 NULL 호환 컬럼은 유지하며 활성 Runtime 응답에는 포함하지 않는다.

조달 수집 날짜·Backfill·날짜 유효성 검사는 한국시간 달력을 사용한다. 저장하는 감사시각은 UTC 를 유지한다.

검증 결과

Pipelines 와 별도 PostgreSQL 통합 검증: 140 개 통과. 갱신 중 새 요청, 동시 Worker, 갱신 실패 시 요청 보존과 한국시간 자정 경계를 검증했다.

Platform: 306 개 통과, 환경 조건이 필요한 24 개 건너뜀.

MCP: 11 개 통과.

Connector·Pipeline·Platform 교차 계약, Registry, 새 Ontology 분류 manifest, Compose config 와 Python 미정의 이름 검사를 통과했다.

운영 공고 수집 Run `5a66fd86-1ed8-4c6c-acf6-70a16d588af2`: `COMPLETED`. 원본 23 건·정규 공고 23 건·문서 0 건이며 checkpoint 는 `2026-10-09` 로 갱신됐다.

기존 첨부파일은 296,812 건으로 수집 전후 동일하며 마지막 수정시각도 동일했다.

Runtime discovery 는 public Capability 30 개를 제공한다. 제거한 요건·판정 Capability 5 개와 관리자 문서 요건·규칙 API 3 개는 404 를 반환한다. 일반 공고 조회는 200 이며 추출 결과 필드가 없다.

이미 적용된 migration, 기존 DB 의 문서·추출 결과, 외부 Object Storage 객체와 인증 volume 은 삭제하지 않았다. API 가 제공하는 공고·면허·지역 정보 수집은 유지한다.

50. bid_notice_vertical_verification.md — Bid notice Context vertical 검증 보고서

원문 파일: docs/verification/bid_notice_vertical_verification.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

검증 시각: 2026-10-07 UTC

		지원 질문과 실제 결과

질문

실행 입력

실제 결과

최근 30 일 동안 게시된 용역 입찰공고

최근 30 일, `work_type=service`

9,334 건

추정가격 1 억 원 이상인 소프트웨어 관련 공고

최근 1 년 기본 범위, `query=소프트웨어`, `estimated_price_min=100000000`

435 건

근로복지공단이 게시한 현재 접수 중인 입찰공고

최근 1 년 기본 범위, 기관 `Z004905`, `bid_status=open`

1 건

기간을 말하지 않은 질문에는 무제한 조회 대신 최근 365 일을 적용하고 `default_period_applied` 경고를 반환한다. 결과는 첫 20 개와 전체 건수를 함께 반환한다.

의미와 실행 계약

Published Ontology: `teoria` 0.1.3

추가 속성: `teoria.BidNotice.estimatedPrice`, `teoria.BidNotice.workType`

Capability: `search_bid_notices`

승인된 Capability Binding: 11 개

Runtime bundle: `2026.10.07.8`

실제 실행 Registry: `2026.10.07.7` 검증 후 metadata binding 을 포함한 `2026.10.07.8` 로 재발행·활성화

OpenMetadata 의 `runtime_bid_notices` 에서 게시일시, 입찰상태, 추정가격, 업무유형과 공고기관코드 컬럼의 존재를 확인했고 각 컬럼을 Ontology property 에 authoritative data asset binding 으로 승인했다.

					데이터 품질 측정

조건

행

고유 ID

게시일 결측

마감일 결측

추정가격 결측

최근 30 일 용역

9,334

9,334

0

1,148

4

최근 1 년 소프트웨어·1 억원 이상

435

435

0

57

0

근로복지공단 접수 중

1

1

0

0

0

마감일과 추정가격은 업무적으로 nullable 하므로 결과에서 누락을 보존한다. OpenMetadata 컬럼 description, Glossary Term 과 Test Suite 는 아직 없으며 quality 상태는 `unavailable` 로 반환한다. 이것은 실행 실패가 아니라 metadata governance 후속 작업이다.

현재 범위

이 vertical 은 정해진 표현을 deterministic parser 로 해석한다. 일반 자연어 AI planner 가 아니며, 지원하지 않는 금액 단위·상대기간·복합 논리는 추측하지 않는다.

51. compose_pipeline_resources_2026_10_16.md — Compose 수집 Worker 재배포 검증: 2026-10-16

원문 파일: docs/verification/compose_pipeline_resources_2026_10_16.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

로컬 `teoria` Compose 프로젝트에 공통 Prefect 등록·Queue 정책과 수집 Worker 자원 상한을 실제 반영했다. 기준은 당시 로컬 작업 트리와 빌드한 이미지이며 GitHub CD 나 EC2 K3s 배포를 검증한 결과는 아니다.

반영과 기존 작업 보존

수집·Metadata work pool 의 새 실행 제출을 일시정지했다.

실행 중이던 공고 수집·최신 개찰 보강·과거 개찰 보강 3 개가 모두 Completed 가 된 뒤 교체했다. 강제 취소하거나 실행 중인 프로세스를 종료하지 않았다.

`prefect-deploy`, `prefect-metadata-deploy` 는 새 등록 명령으로 실행해 exit code 0 을 확인했다.

`prefect-worker`, `prefect-metadata-worker` 만 새 이미지로 재생성했다.

나머지 25 개 컨테이너의 ID·이미지·볼륨 연결이 재배포 전과 동일했다.

	실제 확인 결과

확인

결과

수집 Worker CPU

cgroup quota 400000 / period 100000 = 4 코어

수집 Worker 메모리

cgroup limit 8589934592 bytes = 8GiB

수집 pool 전체 동시 실행 상한

2

`latest` Queue

우선순위 1, 동시 실행 상한 2

`backfill` Queue

우선순위 10, 동시 실행 상한 1

Metadata pool

동시 실행 상한 1

현재 정의의 Deployment 22 개

pool · Queue 배치 일치

새 Worker 실행 검증

임시 Flow 의 첫 실행을 의도적으로 실패시킨 뒤 두 번째 시도에 Completed 확인

UI · nginx health · Admin 인증 session · Runtime health

모두 HTTP 200

검증 Flow 는 Provider API 를 호출하거나 Data DB 를 변경하지 않았다. 완료 후 임시 Flow, Deployment, 실행 기록과 검증 파일을 삭제했다. Metadata Worker 에 OpenMetadata ingestion 라이브러리가 포함되지 않은 것도 확인했다.

기존 월간 업종 수집 `pps-industry-monthly` 는 현재 주간 배포와 중복되므로 예약을 일시정지하고 이력을 보존했다. 현재 수집 배포가 사용하지 않는 `default` Queue 도 일시정지했다. 두 work pool 은 재배포 전과 같은 실행 가능 상태로 복원했고 두 새 Worker 는 Running · 재시작 0 회 상태를 확인했다.

범위와 한계

이는 자원 상한 적용 · 등록 정책 · 실제 Process Worker 실행 · 재시도 검증이다. CPU · 메모리를 한도까지 소모시키는 부하 시험이나 장기간 처리량 측정은 수행하지 않았다. EC2 K3s 실배포 · 노드 장애 복구 검증도 이 결과에 포함하지 않는다. 현재 설정과 재배포 순서는 Compose 배포 [§ 35], Queue 정책은 Prefect 운영 [§ 33]을 따른다.

52. index.md — 검증 기록

원문 파일: docs/verification/index.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

	특정 날짜·데이터·version 에서 확인한 실행 결과와 한계를 보존한다. 문서의 건수·기능 수·환경값은 검증 당시 관찰이며 현재 운영 상태를 뜻하지 않는다. 현재 실행·개발 방법은 문서 목차 [§01]의 해당 프로젝트 문서를 따른다.

기록

검증한 범위

비동기 배포·수집 상태: 2026-10-16 [§48]

API·실제 Worker·재시도·checkpoint·분류 조회·남은 수집 잔량

Context vertical [§56]

대표 계약금액 질문의 의미 연결·실행·DB 대조

공고 Context vertical [§50]

공고 질문·실행 계약·데이터 품질

공공조달 수집 점검: 2026-10-12 [§54]

특정 과거 계약 구간의 원본·적재 결손

OpenMetadata 업그레이드: 2026-10-13 [§53]

DB 복원·migration·연계·재수집·기존 버전 복구

조달 탐색 단회 관측: 2026-10-13 [§55]

소비자가 보고한 검색 시간과 비교 한계

자산 진단: 2026-10-15 [§47]

모델·Worker·API 의 실행·결과 저장 경로

첨부문서 기능 보류: 2026-10-16 [§49]

archive 이동·배포 제거·큐 요청 보존·한국시간 경계

Compose 수집 Worker 재배포: 2026-10-16 [§51]

기존 실행 완료 대기·자원 상한·Queue 정책·실제 실행과 재시도

새 기록에는 기준일·version·입력 범위·검증 방법·결과·한계를 명시한다. 원본 credential·개인정보·비공개 응답은 기록하지 않는다. 실행 검증 성공과 의미·모델 품질 평가는 구분한다. 바뀐 운영 절차는 기록 본문을 복제하지 않고 현재 가이드에 반영한다.

53. openmetadata_upgrade_2026_10_13.md — OpenMetadata 업그레이드 검증

원문 파일: docs/verification/openmetadata_upgrade_2026_10_13.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

검증·배포일: 2026-10-13. 격리 검증 후 운영 Server 와 ingestion 을 1.12.6 에서 2.0.5 로 업그레이드했다. 저장소 배포 기본값과 운영 `deploy/compose/.env` 의 `OPENMETADATA_VERSION` 은 2.0.5 다. 운영 업그레이드는 아래 전환 순서에 따라 별도 수행했으며 기본값 변경만으로 DB migration 을 대신하지 않는다.

공식 안정 릴리스 2.0.5 [참고 1]는 2026-10-08 공개되었다. 사용자 관리 메타데이터 보존과 ingestion 회귀 수정, 보안 의존성 업데이트를 포함한다. 운영은 검증한 명시 버전을 고정하고 `latest` 를 자동 추종하지 않는다.

격리 환경 검증 결과

	운영 `openmetadata_db` 의 custom-format 백업을 별도 PostgreSQL 17 에 복원했다. 별도 Elasticsearch 9.3.0 과 내부 Docker network 를 사용했으며 운영 검색 인덱스를 공유하지 않았다. 메타데이터 ingestion 만 운영 Data DB 의 기존 read-only 계정으로 읽고 결과는 격리 OpenMetadata 에 썼다. 운영 원문 샘플 수집은 수행하지 않았다.

검증

결과

1.12.6 DB 에서 2.0.5 `openmetadata-ops.sh migrate`

중간 Server 를 실행하지 않고 직접 성공

전체 검색 `reindex --force`

성공, Teoria 테이블 검색 응답 확인

테이블 53 개의 ID·FQN·설명·소유자·태그·컬럼

migration 직후 선택한 필드가 변경 전과 일치

Database 1 개·Schema 2 개·DB Service 1 개

기존 ID 전부 보존

Glossary 1 개·Term 1 개·Tag 13 개

기존 ID 전부 보존

Teoria catalog DTO·Table 상세·기존 bearer token

실제 연계 성공

설명 PATCH·Glossary Term 생성·컬럼 용어 할당

실제 연계 성공, 격리 환경에서만 쓰기

Basic Test Suite·Test Case 생성 및 Test Case 조회

실제 연계 성공

샘플·프로파일 미수집 응답

정상 조회 및 미수집 구분

합성 샘플·프로파일

기본 마스킹·명시적 값 조회·DTO 변환 성공

공식 ingestion 2.0.5

schema metadata 수집 성공, 오류 0 건

빌드한 Teoria ingestion 이미지의 재수집

사용자 설명과 컬럼 Glossary 태그 보존

기존 Semantic Binding 의 MetadataTargetRef 17 개

변경 전후 모두 조회 가능한 참조로 유지

기존 버전 복구

원본 백업을 다른 DB 에 복원하고 1.12.6 Server 기동·테이블 53 개 조회 성공

Basic Test Suite 의 `basicEntityReference` 는 Table UUID 가 아니라 FQN 이다. 잘못된 UUID 로 실행한 최초 검증의 404 는 endpoint 제거 또는 버전 호환 결함이 아니었다. 올바른 FQN 으로 기존 Teoria client 의 API 가 성공했다. 2.0.5 프로파일 입력 timestamp 는 epoch milliseconds 이며 Teoria 화면은 초·밀리초를 모두 처리한다. profile 결과가 없는 응답뿐 아니라 합성 profile 로 지표 변환도 확인했다.

현재 운영 데이터에 Domain entity 가 없으므로 기존 Domain 데이터의 migration 보존은 검증하지 못했다. 합성 데이터 검증은 실제 개인정보를 조회하거나 모델에 전달하는 운영 권한·정책 검증을 대신하지 않는다. 복구 검증은 DB 복원과 기존 Server/API 까지이며 운영 검색 인덱스 복구는 별도 재색인이 필요하다.

Pipeline 테스트 324 개, Platform 테스트 274 개가 통과했다. 별도 Application DB 가 필요한 14 개 테스트는 skip 되었다. Connector·Pipeline·Platform 참조 통합 검증과 Compose config 도 통과했다. Compose config 검증에서는 기존 .env 에 누락된 OPA token 에 검증용 placeholder 를 사용했으며 이 값으로 운영 서비스에 배포하지 않았다.

함께 변경한 배포 계약

Compose 의 Server·migration·ingestion image 기본값과 ingestion build argument.

얇은 ingestion Dockerfile 과 `.env.example` 의 버전 기본값.

Prefect metadata deployment 의 image parameter 와 Flow 기본값.

Metadata Worker 의 exact image allowlist.

version 관련 회귀 검증 및 설정 문서.

`teoria-openmetadata-ingestion:2.0.5` 이미지를 실제로 빌드했다. 일반 Pipeline Worker 에는 OpenMetadata ingestion 패키지를 추가하지 않았다. Airflow 도 추가하지 않으며 일정과 재시도는 계속 Prefect 가 소유한다. Prefect 운영 deployment parameter 와 Worker allowlist 는 운영 전환 시 함께 갱신해야 한다.

운영 전환 순서

2026-10-13 운영 전환에서 최신 백업을 확보한 뒤 migration 과 전체 reindex 를 완료했다. 테이블 53 개의 선택한 메타데이터 필드는 migration 직후 변경 전과 일치했다. 2.0.5 Server 와 기존 Teoria Admin 의 catalog status·Table 상세 조회, 검색·샘플·프로파일 endpoint 가 정상 동작했다. 실제 Prefect 실행 `babafb13-4a32-485d-8ad3-e8241f6e25b8` 이 새 ingestion 이미지로 완료됐다. metadata deployment parameter·Worker allowlist 를 함께 갱신하고 기존 매일 05:00 KST schedule 을 재개했다. 배포 등록 Job 도 metadata-worker 이미지와 같은 코드를 사용한다. 조달 수집 워커와 Data DB 는 이 전환에서 변경하지 않았다.

수집 후 컬럼 446 개의 빈 `tags` 표현이 빈 배열과 누락 값 사이에서 정규화되었다. 빈 값을 동일하게 비교하면 실제 태그·컬럼 정보는 보존되며 사용자 설명·소유자· 테이블 태그도 유지됐다. 운영 Binding 대상 17 개가 모두 조회 가능한 참조로 남았다.

정상 동작 확인 후 사용하지 않는 1.12.6 Server·ingestion 이미지와 이전 metadata-worker 이미지, 이번 검증에서 사용한 임시 Platform 이미지 태그를 정리했다. 변경 전 운영 DB 백업은 `.artifacts/openmetadata_upgrade/openmetadata_production_before_2_0_5.dump` 에 권한을 제한해 보존한다. 이미지 정리는 백업 삭제를 포함하지 않는다.

재사용할 업그레이드·복구 절차는 OpenMetadata 배포 [§39]를 따른다. 위 결과는 2026-10-13 의 특정 버전·데이터·환경에 대한 검증이다.

백업과 상세 로그는 `.artifacts/openmetadata_upgrade/`에 권한을 제한하여 보관한다. DB 백업은 인증 설정을 포함할 수 있으므로 Git 에 추가하지 않는다. 일회성 credential 파일과 검증 컨테이너·network 는 검증 종료 후 제거한다.

외부 참고 주소

[1] https://github.com/open-metadata/OpenMetadata/releases/tag/2.0.5-release

54. procurement_collection_2026_10_12.md — 공공조달 수집 점검: 2026-10-12

원문 파일: docs/verification/procurement_collection_2026_10_12.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

이 기록은 당시 관측한 과거 수집 결손이다. 현재 해결 여부는 실행 기록과 원본을 다시 대조해야 한다. 운영 처리 기준은 공공조달 수집 [§31]을 따른다.

2026-10-12 점검에서 과거 용역 계약 `2021-07-29` 는 DB 에 1,661 건이 있었지만, 이를 대조할 원본 보관 이력은 없었다. 제공기관은 같은 날짜의 일부 행에서 오류 99 를 반환한다. API 의 totalCount 와 DB 건수만 같다는 이유로 해당 gap 을 해결로 표시하지 않는다. 해당 gap 은 15 회 재시도 한도에 도달해 자동 재시도가 중단된 상태다. 이 날짜는 제공기관 응답 복구 후 계약 식별자와 원본 hash 를 검증해야 하며, 다른 날짜의 과거 계약 backfill 은 계속 진행한다.

55. procurement_exploration_2026_10_13.md — 조달 탐색 연동의 단회 관측: 2026-10-13

원문 파일: docs/verification/procurement_exploration_2026_10_13.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

	입찰체크 측이 보고한 공고 검색 관측이다. 이 저장소에서 재측정한 결과가 아니다.

조건

보고된 시간

키워드 유지관리, 2024 년부터, `latest_only`

7.49 초

같은 키워드, 최근 90 일, `all`

0.95 초

조회 기간·차수 조건이 다르므로 직접적인 속도 비교나 전체 성능·P95 의 근거로 사용하지 않는다. 현재 계약과 후속 성능 평가 범위는 조달 탐색 연동 [§43]을 따른다.

56. vertical_slice_verification.md — Context vertical 검증 보고서

원문 파일: docs/verification/vertical_slice_verification.md

날짜별 검증 기록 · 기록에 표시된 기준일의 결과와 한계로 읽는다.

검증 시각: 2026-10-07 UTC

이 문서는 대표 질문 하나를 사용해 Ontology, Semantic Binding, OpenMetadata, Capability, Runtime 과 실제 Data DB 가 끝까지 연결되는지 검증한 결과다.

검증 질문

근로복지공단의 최근 5 년 계약금액을 알려줘

기준일은 `2026-10-07` 이며 대상 기간은 `2022-01-01` 부터 `2026-10-07` 까지다.

판정

실제 데이터 실행 경로와 기본 semantic path 가 입증됐다. 컬럼과 Glossary Term 의 연결은 OpenMetadata 의 중첩 컬럼 필드를 명시 조회해 확인했고, Teoria 의 Suggestion → Review → Application 이력도 남겼다. Metadata quality test 는 아직 없으므로 이 vertical 은 `실행·의미 연결 검증 완료`, `quality 검증 미완료`로 판정한다.

		현재 Registry 원본과 Admin API 의 Capability 수는 모두 **46 개**다. Runtime API 는 이 중 public exposure 인 **29 개**를 노출한다. 따라서 이전 화면에서 관찰된 52 개는 현재 Registry 기준 숫자가 아니며 준비도 판단에 사용하면 안 된다.

검증 단계

결과

관찰된 근거

Published Ontology

통과

`procurement.Contract.currentAmount`, 단위 KRW, artifact checksum `3d912a41...95673`

승인된 Capability Binding

통과

analytics route 가 `search_public_procurement_contracts` 로 선택됨

Capability 실행 계약

통과

날짜와 기관코드 입력 후 24 페이지 실행

실제 Data DB 대조

통과

API 와 DB 모두 계약 2,323 건, 결측 0 건, 합계 1,257,472,414,585 원

Registry provenance

통과

Published Registry `2026.10.07.6`, checksum 과 Git commit 반환

OpenMetadata asset 존재

통과

계약 테이블과 `current_contract_amount` 컬럼 조회 성공

OpenMetadata glossary 존재

통과

`Procurement.ContractAmount` 조회 성공

Physical Asset → Glossary Term

통과

컬럼에 `Procurement.ContractAmount`, `Manual/Confirmed` 적용 확인

Review/Application 이력

통과

Suggestion `6bd68784-1427-42e8-9196-40048db1586a` 가 `system:admin` 승인 후 applied

실행 시 metadata 조회

통과

Admin 연동 활성화 후 두 entity 모두 `available=true`

데이터 freshness 계약

통과

Runtime object 의 `source_registered_at` 최댓값을 실행 검증에 기록

Metadata quality test

실패

OpenMetadata table 의 `testSuite` 가 없음

실행 증거

Context Engine 은 질문을 다음과 같이 해석했다.

```
업무 개념: procurement.Contract.currentAmount
```

```
기관: 근로복지공단본부 (Z004905)
기간: 2022-01-01 ~ 2026-10-07
Capability: search_public_procurement_contracts
금액 필드: current_contract_amount
```

Runtime 결과는 다음과 같다.

```
상태: complete
고유 계약 수: 2,323
금액 존재 계약 수: 2,323
합계: 1,257,472,414,585 KRW
실행 페이지: 24
실패 페이지: 없음
```

동일 조건으로 `public_procurement.runtime_contracts` 를 직접 집계한 결과도 다음과 같았다.

```
rows: 2,323
distinct unified_contract_number: 2,323
null current_contract_amount: 0
sum(current_contract_amount): 1,257,472,414,585
latest_update: 2026-10-06 17:49:57.861539+00
```

따라서 이 결과는 UI mock 이나 Capability 선언만으로 만들어진 값이 아니라 실제 Data DB 를 Runtime 이 읽어 계산한 값이다.

의미 연결 증거와 결손

승인된 Binding 에는 아래 두 OpenMetadata reference 가 존재한다.

```
openmetadata://column/
  teoria_postgresql.teoria_data.public_procurement.contracts.current_contract_amount

openmetadata://glossaryTerm/Procurement.ContractAmount
```

OpenMetadata 에서 두 entity 를 실제 조회할 수 있었고 설명도 존재했다.

테이블: 조달 원계약과 변경·차수계약의 최신 계약 상태 및 금액을 관리하는 정규화 테이블

Glossary Term: 계약 사건에서 확정된 통화 기준 계약금액

`fields=columns` 를 포함해 조회한 결과 `current_contract_amount` 컬럼에는 `Procurement.ContractAmount` 가 `Manual/Confirmed` 상태로 적용돼 있었다. 기본 조회 응답에서 중첩 tag 가 생략되는 것을 tag 부재로 판정하지 않아야 한다.

```
Physical Column → Procurement.ContractAmount → procurement.Contract.currentAmount
```

Admin 배포의 OpenMetadata 연동을 활성화했고 Context 응답에서 entity 가용성, metadata 수정 시각과 실제 Runtime object 의 source freshness 를 구분해 반환한다.

다음 완료 조건

이 vertical 을 완결하려면 다음 조건을 모두 충족해야 한다.

`current_contract_amount` 에 대한 OpenMetadata quality test 를 정의하고 실행한다.

같은 검증을 대표 질문 3~5 개에 반복하고 통과한 Capability 만 `verified` 로 구분한다.

이 조건 전에는 전체 Registry 가 준비됐다고 표현하지 않고, **한 Context vertical 의 실행과 의미 연결은 검증됐으며 quality 와 나머지 Capability 는 미검증**이라고 표현한다.

다음 후보 준비도 감사

		대표 후보 5 개를 동일한 binding coverage 기준으로 감사했다. 종단 검증을 통과한 것으로 취급하는 것은 첫 항목뿐이다.

Capability

현재 상태

차단 사유

`search_public_procurement_contracts`

vertical 검증

Context 에 필요한 금액 output 은 승인됐으나 전체 Capability coverage 는 partial

`get_bid_notice`

partial

입력 2 개와 관계 output 1 개가 미연결

`search_bid_notices`

unbound

입력 22 개와 output 3 개가 미연결

`search_public_organizations`

unbound

입력 6 개와 output 1 개가 미연결

`search_companies_by_name`

unbound

입력 1 개와 output 5 개가 미연결

준비도 수치를 억지로 높이기 위해 pagination 이나 free-text 같은 기술 입력을 업무 개념에 연결하지 않는다. 다음 vertical 질문을 먼저 정한 뒤 필요한 의미만 승인한다.